



UNIVERSITY OF HERTFORDSHIRE

FINAL YEAR PROJECT REPORT

**AI-BASED INTERACTIVE SYSTEM IN HUMAN-COMPUTER INTERACTION
USING NATURAL LANGUAGE PROCESSING FOR DISABLED PEOPLE**

MOHAMMAD

SCSJ2000064

A project report submitted in partial fulfilment of the requirements for the course

6COM2017 ARTIFICIAL INTELLIGENCE PROJECT (PART 2)

FACULTY OF COMPUTING AND INFORMATION TECHNOLOGY

SEGi UNIVERSITY

JUNE 2026

2025/2026 ACADEMIC SESSION

UNIVERSITY OF HERTFORDSHIRE

FINAL YEAR PROJECT REPORT

TITLE

**AI-BASED INTERACTIVE SYSTEM IN HUMAN-COMPUTER INTERACTION
USING NATURAL LANGUAGE PROCESSING FOR DISABLED PEOPLE**

Submitted By

Name: Mohammad

Student ID: SCSJ2000064

Course Information

Course Title: 6COM2017 Artificial Intelligence Project (Part 2)

Submitted To

Lecturer: Ms. Anizah

Supervisor: Ms. Diana

Submission Date

Date: 8 June 2026

Institution

SEGi University

FACULTY OF COMPUTING AND INFORMATION TECHNOLOGY

2025/2026 ACADEMIC SESSION

AUTHOR'S DECLARATION

I declare that the work presented in this Final Year Project Report was carried out in accordance with the academic regulations and requirements of SEGi University and the University of Hertfordshire. The work is original and is the result of my own effort, except where otherwise acknowledged through appropriate references and citations.

I further declare that this project report has not been submitted, either wholly or partly, to any other academic institution for the award of any degree, diploma, or other qualification.

I acknowledge that I have complied with the academic integrity policies and regulations governing the conduct of my studies and research.

Name of Student:	Mohammad
Student ID No.:	SCSJ2000064
Course:	6COM2017 Artificial Intelligence Project (Part 2)
Faculty:	Faculty of Computing and Information Technology
Institution:	SEGi University

Project Title: AI-Based Interactive System in Human-Computer Interaction Using Natural Language Processing for Disabled People

Mohammad

Signature of Student:

Date: 8 June 2026

ABSTRACT

Traditional computer interaction relies heavily on physical input devices such as keyboards and computer mice, which may create accessibility challenges for individuals with physical disabilities and mobility impairments. In addition, many speech-recognition systems depend on internet connectivity and cloud-based processing, resulting in privacy concerns and reduced reliability. Therefore, there is a need for an offline voice-controlled system that enables hands-free computer interaction. This research aims to develop and evaluate an Offline Voice-Controlled Desktop Automation System using Vosk, PocketSphinx, and CMUSphinx speech-recognition frameworks integrated with Python-based desktop automation technologies. A prototype development methodology was adopted, and the system was implemented in a Windows environment to support functions such as mouse control, scrolling, browser automation, application launching, camera control, and volume management through voice commands. System evaluation was conducted through functional testing, command-recognition analysis, Word Error Rate (WER) evaluation, environmental-noise testing, and user evaluation involving 17 participants. The results indicate that offline speech-recognition technologies can effectively support real-time desktop automation. Among the evaluated frameworks, Vosk achieved the best overall performance in terms of recognition accuracy, command-detection capability, and robustness against environmental noise. The study demonstrates the feasibility of offline voice-controlled desktop automation and contributes to the development of accessible and privacy-preserving Human-Computer Interaction systems.

ACKNOWLEDGEMENT

First and foremost, I would like to express my deepest gratitude to Almighty God for His blessings, strength, guidance, and mercy throughout the completion of this project. Without His grace and support, this research would not have been possible.

I would also like to convey my heartfelt appreciation to my beloved parents for their unconditional love, continuous encouragement, sacrifices, and unwavering support throughout my academic journey. Their motivation, patience, and belief in my abilities have been a constant source of inspiration and have played a significant role in helping me achieve this milestone.

My sincere gratitude is extended to my project supervisor, Ms Diana, for the invaluable guidance, constructive feedback, encouragement, and support provided throughout the development of this project. Their expertise, patience, and dedication greatly contributed to the successful completion of this research.

I would also like to thank the lecturers and staff of the Faculty of Computing and Information Technology for their assistance and support during my studies. Special thanks are extended to all participants who took part in the testing and evaluation of the proposed Offline Voice-Controlled Desktop Automation System. Their cooperation and feedback contributed significantly to the success of this study.

Finally, I would like to express my appreciation to my friends and everyone who has directly or indirectly contributed to the completion of this project. Their support, encouragement, and assistance are sincerely appreciated.

Thank you.

Table of Contents

AUTHOR'S DECLARATION	3
ABSTRACT	4
ACKNOWLEDGEMENT	5
CHAPTER 1	14
INTRODUCTION.....	14
1.0 Introduction.....	14
1.1 Background of the Study	16
1.3 Problem Statement	17
1.4 Aim of the Research.....	19
1.5 Research Questions	19
1.6 Research Objectives.....	19
1.7 Scope of the Research.....	20
1.8 Significance of the Research.....	22
1.9 Report Structure	23
CHAPTER 2.....	25
LITERATURE REVIEW.....	25
2.1 Introduction.....	25
2.2 Artificial Intelligence and Intelligent Systems.....	26
2.3 Human–Computer Interaction and Accessibility	27
2.4 Voice-Based Interaction in Human–Computer Interaction	28
2.5 Automatic Speech Recognition Technology	30
2.6 Evolution of Automatic Speech Recognition.....	31
2.7 Automatic Speech Recognition Pipeline.....	33
2.8 Cloud-Based and Offline Speech Recognition	34
2.9 Speech Recognition Frameworks	36
2.10 Comparative Analysis of Speech Recognition Frameworks.....	38
2.11 Desktop Automation Technologies	40
2.12 Noise Robustness in Speech Recognition.....	41
2.13 Latency and Real-Time Performance	43
2.14 Research Gap	45

2.15 Chapter Summary	47
CHAPTER 3	49
METHODOLOGY	49
3.1 Introduction.....	49
3.2 Research Design and Development Methodology.....	50
3.3 Requirement Analysis	55
3.4 System Architecture	65
3.5 Operational Workflow.....	80
3.6 System Modules and Functional Components.....	85
3.7 Chapter Summary	100
CHAPTER 4.....	102
IMPLEMENTATION.....	102
4.1 Introduction.....	102
4.2 Experimental Setup and Testing Environment.....	103
4.3 Functional Testing of Desktop Automation Commands	104
4.4 Vosk Recognition Performance Evaluation	114
4.5 PocketSphinx Recognition Performance Evaluation	118
4.6 CMUSphinx Recognition Performance Evaluation.....	123
4.7 Runtime Execution Monitoring Evaluation	127
4.8 Word Error Rate (WER) Evaluation	130
4.9 Command Detection Rate Evaluation.....	133
4.10 Environmental Noise Evaluation	134
4.11 Comparative Analysis of Speech Recognition Frameworks.....	139
4.12 Result Discussion.....	142
4.13 Chapter Summary	144
CHAPTER 5.....	147
ANALYSIS AND DISCUSSION.....	147
5.1 User Evaluation Overview	147
5.2 User Testing Sessions.....	148
5.3 Result Analysis.....	150
5.4 Comparative Analysis	150
5.5 Discussion	151

5.6 Project Schedule and Development Timeline	153
5.7 Overall Discussion	155
CHAPTER 6.....	157
CONCLUSION AND RECOMMENDATIONS	157
6.1 Summary of the Study	157
6.2 Conclusion	158
6.3 Limitation.....	159
6.4 Future Work	161
7.0 References.....	166

LIST OF TABLES

Table No.	Title
Table 2.1	Benefits of Voice-Controlled Interfaces
Table 2.2	Applications of Speech Recognition Technology
Table 2.3	Comparison Between Cloud-Based and Offline Speech Recognition
Table 2.4	Comparative Analysis of Speech Recognition Frameworks
Table 2.5	Desktop Automation Technologies
Table 2.6	Noise Robustness Evaluation from Previous Studies
Table 2.7	Literature Review on Recognition Accuracy and WER
Table 2.8	Summary of Literature Findings
Table 3.1	Research Question–Research Objective–Methodology Mapping
Table 3.2	Functional Requirements of the Proposed System
Table 3.3	Non-Functional Requirements of the Proposed System
Table 3.4	User Requirements
Table 3.5	Hardware Requirements
Table 3.6	Software Requirements
Table 3.7	Comparison of Available Vosk Models
Table 4.1	Experimental Environment Configuration
Table 4.2	Functional Testing Results
Table 4.3	Vosk Recognition Results
Table 4.4	Vosk Recognition Accuracy Summary
Table 4.5	PocketSphinx Recognition Results
Table 4.6	PocketSphinx Recognition Accuracy Summary
Table 4.7	CMUSphinx Recognition Results
Table 4.8	CMUSphinx Recognition Accuracy Summary
Table 4.9	Runtime Execution Observation Results
Table 4.10	Word Error Rate Evaluation Result
Table 4.11	Overall Command Detection Rate
Table 4.12	Detection Results at 50–57 dB
Table 4.13	Detection Results at 68–79 dB
Table 4.14	Comparative Performance Analysis
Table 5.1	User Evaluation Overview
Table 5.2	Comparative Analysis of Different Age Groups

Table 6.1 Risk Register for the Proposed Offline Voice-Controlled Desktop Automation System

LIST OF FIGURES

Figure No.	Title
Figure 1.1	Evolution of Speech Recognition Technology
Figure 2.1	Automatic Speech Recognition Process
Figure 2.2	Human–Computer Interaction Framework for Voice Control
Figure 2.3	Generic Architecture of a Speech Recognition System
Figure 2.4	Literature Review Conceptual Framework
Figure 3.1	Prototype Development Methodology
Figure 3.2	Use Case Diagram of the Proposed System
Figure 3.3	Overall System Architecture
Figure 3.4	Voice Command Processing Flowchart
Figure 3.5	System Development Workflow
Figure 3.6	Desktop Automation Command Flow
Figure 3.7	Voice Input Acquisition Process
Figure 3.8	Vosk Recognition Workflow
Figure 3.9	PocketSphinx Recognition Workflow
Figure 3.10	CMUSphinx Recognition Workflow
Figure 3.11	Runtime Monitoring Workflow
Figure 3.12	WER Evaluation Procedure Flowchart
Figure 4.1	Browser Automation Output
Figure 4.2	Calculator Automation Output
Figure 4.3	Camera Automation Output
Figure 4.4	Volume Increase Output
Figure 4.5	Volume Mute Output
Figure 4.6	User 1 Voice Command Testing Session
Figure 4.7	User 2 Voice Command Testing Session
Figure 4.8	User 3 Voice Command Testing Session
Figure 4.9	User 4 Voice Command Testing Session
Figure 4.10	Vosk Recognition Test – Attempt 1
Figure 4.11	Vosk Recognition Test – Attempt 2
Figure 4.12	Vosk Recognition Test – Attempt 3
Figure 4.13	PocketSphinx Recognition Test – Attempt 1
Figure 4.14	PocketSphinx Recognition Test – Attempt 2

Figure 4.15	PocketSphinx Recognition Test – Attempt 3
Figure 4.16	CMUSphinx Recognition Test – Attempt 1
Figure 4.17	CMUSphinx Recognition Test – Attempt 2
Figure 4.18	CMUSphinx Recognition Test – Attempt 3
Figure 4.19	Runtime Monitoring, Command Logging, and Error Detection Output
Figure 4.20	Runtime Automation Execution Verification Output
Figure 4.21	WER Calculation Procedure
Figure 4.22	Testing Environment at 50–57 dB
Figure 4.23	Testing Environment at 68–79 dB
Figure 4.24	Comparative Performance Analysis Chart
Figure 5.1	User Testing Sessions
Figure 5.2	User Evaluation Participant Categories
Figure 5.3	Project Gantt Chart
Figure 6.1	Proposed Future Architecture for an Enhanced Voice-Controlled Desktop Automation System

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
ASR	Automatic Speech Recognition
HCI	Human–Computer Interaction
NLP	Natural Language Processing
DNN	Deep Neural Network
HMM	Hidden Markov Model
WER	Word Error Rate
GUI	Graphical User Interface
IDE	Integrated Development Environment
OS	Operating System
API	Application Programming Interface
MFCC	Mel Frequency Cepstral Coefficients
SNR	Signal-to-Noise Ratio
IoT	Internet of Things
CPU	Central Processing Unit
RAM	Random Access Memory
USB	Universal Serial Bus
Hz	Hertz
dB	Decibel
GUI	Graphical User Interface
NLP	Natural Language Processing
HMM	Hidden Markov Model
DNN	Deep Neural Network

CHAPTER 1

INTRODUCTION

1.0 Introduction

The rapid advancement of intelligent computing technologies has significantly transformed the interaction between humans and digital systems. Traditional Human-Computer Interaction (HCI) methods primarily depend on physical input devices such as keyboards, touchpads, and computer mice. Although these devices remain effective for general computing tasks, they may reduce convenience, efficiency, and accessibility in situations where hands-free interaction is preferred or required. Consequently, speech-recognition technology has emerged as an important research area within Artificial Intelligence (AI), automation, and HCI (Malik et al., 2021; Latif et al., 2021).

Automatic Speech Recognition (ASR) refers to computational methods that enable computer systems to interpret spoken language and convert speech into textual output or executable commands. Modern ASR systems utilise machine learning, digital signal processing, deep neural architectures, and Natural Language Processing (NLP) techniques to recognise and process human speech with increasing accuracy and lower response latency (Kheddar et al., 2024; Malik et al., 2021). The integration of speech-recognition technologies into modern computing environments has enabled the development of intelligent voice-driven systems capable of supporting more natural and efficient interaction between humans and machines.

Voice-controlled systems are increasingly integrated into virtual assistants, healthcare technologies, accessibility systems, robotics, educational platforms, and smart environments. Applications such as Google Assistant, Siri, and Alexa demonstrate the growing importance of speech-enabled interaction within modern digital ecosystems (Deshmukh & Chalmeta, 2024). These systems allow users to perform tasks such as application launching, multimedia control, information retrieval, and operating-system interaction using spoken commands.

In addition to improving convenience, speech-controlled systems have become increasingly valuable within accessibility-oriented environments. Individuals with mobility impairments,

repetitive strain conditions, or physical disabilities may experience difficulties when relying continuously on traditional physical input devices. Speech-enabled systems therefore improve accessibility by enabling users to interact with computer systems more independently through spoken interaction (Masina et al., 2020; Vieira et al., 2022).

Despite the growing adoption of speech-recognition systems, several practical and technical limitations remain unresolved. Environmental noise, accent variation, microphone quality, speech inconsistency, and command-recognition latency may negatively affect system performance and usability. Furthermore, many modern ASR systems depend heavily on cloud-based processing and internet connectivity, potentially introducing privacy concerns, cybersecurity risks, and inconsistent response latency under unstable network conditions (Gondi & Pratap, 2021; Di Leo et al., 2025).

This research focuses on the development and evaluation of an offline speech-controlled desktop automation system capable of executing real-time computer operations using voice commands. The proposed system integrates speech-recognition technologies with desktop automation frameworks to allow users to control browser platforms, multimedia applications, operating-system functions, and mouse operations through spoken interaction. The study specifically evaluates offline speech-recognition frameworks including Vosk, PocketSphinx, and CMUSphinx within desktop automation environments.

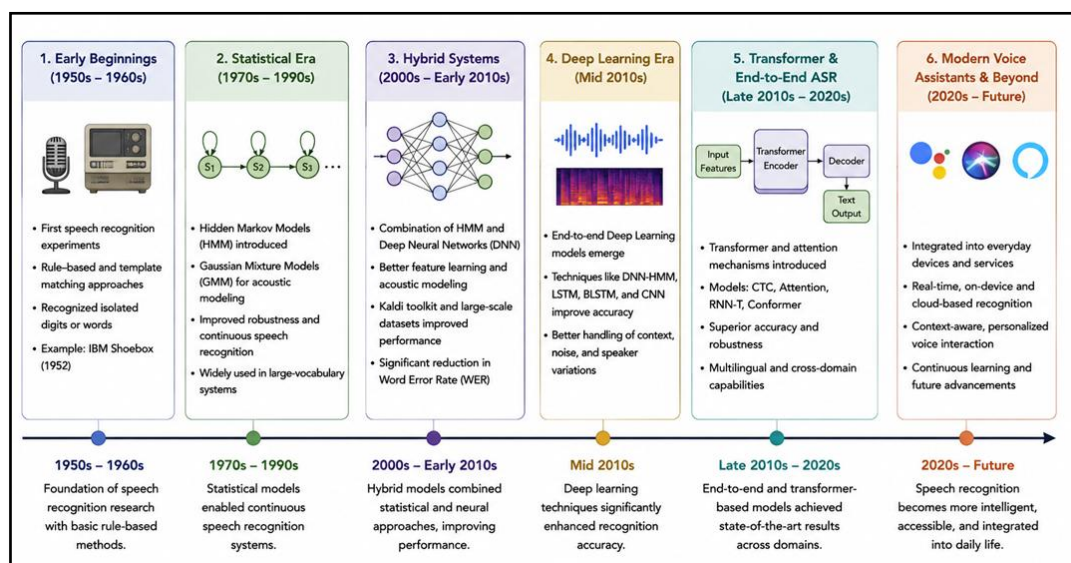


Figure 1.1 Evolution of Speech Recognition Technology

1.1 Background of the Study

Human-Computer Interaction (HCI) has evolved significantly alongside advancements in Artificial Intelligence, automation technologies, and intelligent computing systems. Early computer systems relied heavily on command-line interfaces and physical input devices such as keyboards and computer mice. However, modern computing environments increasingly utilise more natural interaction approaches including gesture recognition, eye tracking, touch interaction, and speech-controlled interfaces (Zhang et al., 2022).

Among these technologies, speech recognition has become one of the most widely researched interaction methods because of its ability to facilitate natural communication between humans and machines. Speech-recognition systems operate by capturing speech signals through microphones, processing acoustic patterns, extracting relevant features, and converting spoken language into meaningful textual outputs or executable instructions (Fendji et al., 2022; Malik et al., 2021).

Earlier ASR systems primarily relied on Hidden Markov Models (HMMs) and Gaussian Mixture Models (GMMs). However, recent speech-recognition systems increasingly utilise deep-learning architectures such as Deep Neural Networks (DNNs), Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) networks, and Transformer-based models to improve recognition performance (Kheddar et al., 2024; Ahlawat et al., 2025).

The increasing popularity of intelligent devices and virtual assistants has accelerated the adoption of speech-based interaction across healthcare, education, accessibility systems, entertainment, and automation platforms. Speech-recognition technologies are particularly beneficial within accessibility-oriented environments because they reduce dependence on conventional physical input devices and support hands-free interaction capabilities (Masina et al., 2020; Núñez Macías et al., 2023).

Desktop automation technologies have also become increasingly important within modern computing environments. Automation systems allow repetitive or routine operations to be executed automatically, thereby improving operational efficiency and reducing manual workload. Integrating speech recognition into desktop automation systems creates

opportunities for users to interact with computer systems more naturally and efficiently through spoken commands.

This research investigates the implementation of an offline speech-controlled desktop automation system developed using Python programming language and automation libraries. The proposed system enables users to execute desktop operations such as mouse movement, clicking functions, scrolling, browser navigation, multimedia access, volume control, and application launching through predefined voice commands.

The speech-recognition frameworks investigated within this research include Vosk, PocketSphinx, and CMUSphinx. These frameworks were selected because they provide offline speech-processing capabilities suitable for lightweight and privacy-preserving desktop automation environments (Di Leo et al., 2025; Gondi & Pratap, 2021).

1.3 Problem Statement

Traditional computer interaction methods rely heavily on physical input devices such as keyboards and computer mice. Although this interaction methods remain widely adopted, they may reduce efficiency, convenience, and accessibility in situations where hands-free interaction is preferred or required. Individuals with mobility impairments, repetitive strain conditions, or physical disabilities may experience difficulties when interacting continuously with conventional computer hardware such as keyboards and computer mice. These challenges may reduce accessibility, increase physical fatigue, and limit independent computer usage. As a result, researchers have increasingly explored natural and accessible human-computer interaction approaches, including speech-based interfaces, gesture recognition, eye tracking, and touch interaction, to improve usability and inclusivity for diverse user groups (Mott et al., 2022; Zhang et al., 2022).

Speech-recognition technologies provide an alternative interaction approach by enabling users to control digital systems through spoken commands. Such technologies have the potential to improve accessibility, reduce physical effort, and enhance user independence, particularly for individuals with physical limitations. However, implementing reliable and responsive speech-controlled systems remains a significant challenge because environmental noise, pronunciation

variation, microphone sensitivity, accent diversity, and speech inconsistency may negatively affect command-recognition accuracy and system usability (Pervaiz et al., 2020; Dua et al., 2023). These limitations may lead to recognition errors, reduced user satisfaction, and decreased system effectiveness in practical computing environments.

Another important limitation involves the dependence of many speech-recognition systems on cloud-based processing and internet connectivity. In cloud-based architectures, speech data must be transmitted to remote servers for processing before a response can be generated. This dependency may result in increased response latency, service interruptions during network failures, and reduced reliability in environments with unstable internet access. Furthermore, transmitting speech data to external cloud infrastructure may expose sensitive user information to privacy and cybersecurity risks, including unauthorized access, data leakage, and third-party storage concerns (Gondi & Pratap, 2021; Vieira et al., 2022; Wang et al., 2020). Consequently, offline speech-recognition approaches have gained increasing research attention because they enable local processing, lower latency, improved reliability, and enhanced privacy protection.

Furthermore, many existing speech-controlled systems focus primarily on virtual-assistant functionality or transcription accuracy rather than practical desktop automation. Existing literature also gives limited attention to command-execution reliability, response consistency, environmental robustness, and real-time operating-system interaction within offline desktop automation environments (Ferraro et al., 2023; Harihar et al., 2025). Although recent advancements in lightweight automatic speech-recognition engines such as Vosk and PocketSphinx have demonstrated the feasibility of offline speech processing on local devices, research investigating their integration with real-time desktop automation remains limited. Therefore, challenges relating to command-recognition accuracy, execution reliability, and usability in offline automation environments continue to require further investigation.

Therefore, there is a need to develop and evaluate a lightweight offline speech-controlled desktop automation system capable of executing real-time computer operations efficiently, accurately, and reliably within practical computing environments. The proposed system addresses this need by integrating offline speech-recognition technologies with predefined command-mapping mechanisms to perform desktop automation tasks such as cursor movement, mouse control, scrolling, application management, media control, and system navigation without requiring internet connectivity. By processing speech locally on the user's

device, the system aims to improve response speed, operational reliability, accessibility, and privacy while providing a practical and cost-effective solution for real-world human-computer interaction applications.

1.4 Aim of the Research

The aim of this research is to develop and evaluate an offline voice-controlled desktop automation system capable of executing real-time computer operations using speech recognition technologies.

1.5 Research Questions

This research seeks to answer the following questions:

- How can Vosk, PocketSphinx, and CMUSphinx offline speech-recognition technologies be integrated with Python-based automation tools to support real-time desktop automation?
- How effectively can the proposed voice-controlled system recognize speech commands and execute desktop automation tasks such as mouse control, scrolling, browser automation, application launching, and multimedia control?
- What is the recognition accuracy, command-execution success rate, and usability level achieved by the proposed offline voice-controlled desktop automation system?
- How does environmental noise influence speech-recognition accuracy and command-execution performance in the proposed system?

1.6 Research Objectives

The objectives of this research are as follows:

- To develop an offline voice-controlled desktop automation system using Vosk, PocketSphinx, and CMUSphinx speech-recognition engines integrated with Python-

based automation technologies to support accessible hands-free human-computer interaction.

- To implement voice-command functionalities for mouse control, scrolling operations, browser automation, multimedia interaction, application launching, and operating-system control using PyAutoGUI, Webbrowser, and Subprocess modules.
- To evaluate the accuracy, reliability, and usability of the proposed system using command-recognition accuracy testing, Word Error Rate (WER) analysis, command-execution validation, and functional testing.
- To analyse the effects of environmental noise on speech-recognition accuracy and command-execution performance through noise-robustness testing, ambient-noise calibration evaluation, and recognition-stability analysis under different operating conditions.

1.7 Scope of the Research

This research focuses on the development and evaluation of an offline voice-controlled desktop automation system using Vosk, PocketSphinx, and CMUSphinx speech-recognition technologies integrated with Python-based automation libraries. The proposed system is designed to support hands-free human-computer interaction by enabling users to perform common desktop operations through predefined English-language voice commands without requiring internet connectivity.

The research is particularly relevant for users who may benefit from alternative interaction methods, including individuals with mobility impairments, repetitive strain injuries, temporary physical limitations, or users seeking more convenient hands-free computer interaction. The proposed system is intended as an accessibility-oriented desktop automation solution rather than a general-purpose conversational virtual assistant.

The scope of the research includes:

- Real-time speech recognition using offline Automatic Speech Recognition (ASR) technologies.

- Offline speech processing without reliance on cloud-based recognition services or internet connectivity.
- Desktop automation through voice-command execution.
- Mouse movement, clicking, and scrolling operations for hands-free desktop navigation.
- Volume-control operations for multimedia interaction.
- Browser automation and website navigation through voice commands.
- Application launching and control within a Windows desktop environment.
- Speech-enabled Human–Computer Interaction (HCI) to improve accessibility and user convenience.

The implemented system supports selected applications and services that represent common desktop activities and are used to evaluate the effectiveness of offline voice-controlled automation. These include:

- Google Chrome for browser automation and web navigation.
- YouTube for multimedia platform access and online content navigation.
- Calculator for productivity-related application control.
- Camera for multimedia-device interaction.
- Volume-control functions for multimedia management.

These applications and operations were selected because they represent common desktop tasks involving web navigation, multimedia interaction, productivity tools, and operating-system control. Their implementation enables evaluation of the proposed system across multiple desktop automation scenarios while maintaining a manageable project scope.

The research is limited to predefined command-based interaction, English-language voice commands, and Windows operating-system environments. The system does not support unrestricted natural-language conversation, cloud-based speech processing, speaker authentication, or cross-platform deployment.

The speech recognition frameworks discussed and evaluated in this research include:

- Vosk

- PocketSphinx
- CMUSphinx

However, the primary implementation of the proposed system utilises Vosk because of its stronger offline recognition performance, lower response latency, and improved environmental robustness compared to lightweight traditional ASR systems (Di Leo et al., 2025; Netinant et al., 2026).

The research is restricted to predefined English-language voice commands for offline desktop automation within a Windows operating-system environment. The focus of the study is on command recognition, desktop automation, and real-time voice-controlled human-computer interaction. Therefore, the implementation scope is limited to offline speech processing and command-based desktop control. Advanced functionalities such as multilingual speech recognition, conversational artificial intelligence, biometric speaker authentication, custom deep-learning model training, and cloud-based speech-processing architectures are beyond the scope of the present study and may be considered in future system enhancements.

1.8 Significance of the Research

This research contributes to the field of speech-enabled HCI by demonstrating the practical implementation of an offline speech-controlled desktop automation system within a real-time computing environment. The findings of this study may provide useful insights for researchers and developers interested in automation technologies, accessibility systems, and intelligent voice-driven interaction platforms.

From an accessibility perspective, the proposed system may assist individuals with mobility impairments or physical disabilities by enabling hands-free interaction with computer systems. This may improve user independence while reducing dependence on conventional physical input devices (Masina et al., 2020; Volochtchuk et al., 2024).

From a technological perspective, the research demonstrates how offline speech-recognition technologies can be integrated with desktop automation frameworks to perform practical operating-system interactions through spoken commands. The study also highlights important

challenges involving environmental noise, command-recognition reliability, response latency, and speech variability within real-time desktop automation environments.

Furthermore, the research contributes towards understanding the role of privacy-preserving offline speech-recognition systems within modern computing environments. The proposed system may serve as a foundation for future development of more advanced desktop assistants, accessibility-oriented systems, and intelligent automation technologies (Gondi & Pratap, 2021; Di Leo et al., 2025).

1.9 Report Structure

This report is organised into six chapters:

- Chapter 1 introduces the research background, problem statement, research objectives, research questions, scope of the study, significance of the research, and overall organisation of the report.
- Chapter 2 presents the literature review using a thematic review approach. The chapter discusses previous studies related to speech-recognition technologies, offline Automatic Speech Recognition (ASR) systems, desktop automation, Human–Computer Interaction (HCI), accessibility-focused computing, and research gaps that motivate the proposed study.
- Chapter 3 explains the research methodology adopted in the study, including the prototype-based development approach, requirement analysis, system architecture design, implementation of Vosk, PocketSphinx, and CMUSphinx, command-processing mechanisms, desktop automation modules, system testing procedures, and evaluation methods.
- Chapter 4 presents the system implementation and experimental evaluation of the proposed offline voice-controlled desktop automation system. The chapter includes implementation results, functional testing, recognition-accuracy analysis, Word Error Rate (WER) evaluation, noise-robustness testing, command-execution validation, runtime monitoring, and performance evaluation findings.
- Chapter 5 discusses the results obtained from the implementation and evaluation process. The chapter interprets the findings, compares the performance of the

implemented speech-recognition frameworks, analyses system effectiveness and usability, and relates the results to the research objectives and existing literature.

- Chapter 6 concludes the research by summarising the major findings, discussing the contributions and limitations of the study, and providing recommendations for future improvements and further research in offline speech-controlled desktop automation systems.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

Artificial Intelligence (AI) has become one of the most influential technologies in modern computing and has significantly transformed the way humans interact with computer systems. Advances in machine learning, deep learning, and Natural Language Processing (NLP) have enabled intelligent systems to perform tasks that traditionally required human cognitive abilities, including speech understanding, decision-making, pattern recognition, and adaptive interaction (Kheddar et al., 2024; Fatehi et al., 2025).

The increasing integration of AI technologies has also contributed to the development of more natural and user-friendly Human–Computer Interaction (HCI) environments. Modern HCI research emphasises interaction methods that reduce dependence on conventional input devices such as keyboards and computer mice. As a result, speech-based interaction, gesture recognition, eye tracking, and touch-based interfaces have gained considerable attention as alternative communication mechanisms between humans and computers.

Among these technologies, Automatic Speech Recognition (ASR) has emerged as a key enabling technology for voice-controlled systems, accessibility-oriented applications, intelligent assistants, and desktop automation platforms. Recent advancements in speech-recognition technologies have improved recognition accuracy, response latency, environmental robustness, and deployment flexibility, making speech-based interaction increasingly practical for real-world computing environments (Ferraro et al., 2023; Fadel et al., 2023).

This chapter critically reviews the literature related to Artificial Intelligence, Human–Computer Interaction, speech-based interaction, Automatic Speech Recognition technologies, speech-recognition architectures, offline and cloud-based ASR systems, desktop automation technologies, environmental robustness, latency, privacy considerations, and accessibility-oriented computing. The chapter further evaluates major speech-recognition frameworks including PocketSphinx, CMUSphinx, Vosk, and Google Speech-to-Text while identifying

research gaps that justify the development of the proposed offline voice-controlled desktop automation system.

2.2 Artificial Intelligence and Intelligent Systems

Artificial Intelligence (AI) refers to the ability of computer systems to perform tasks that typically require human intelligence, such as learning, reasoning, problem-solving, decision-making, and language understanding. AI has become an important component of modern computing and is widely applied in healthcare, education, robotics, cybersecurity, transportation, and intelligent automation systems (Kheddar et al., 2024; Fatehi et al., 2025).

Advances in machine learning and deep learning have significantly enhanced AI capabilities by enabling systems to analyse data, identify patterns, adapt to changing conditions, and improve performance over time. These developments have contributed to the growth of intelligent systems that support more efficient and natural interactions between humans and computers (Ahlawat et al., 2025; Zhang et al., 2025).

Intelligent systems utilise AI techniques to process information, make decisions, and perform actions to achieve specific objectives. One important application of AI is speech recognition, which converts spoken language into machine-understandable commands or text through the use of acoustic modelling, language modelling, and pattern-recognition techniques. Speech recognition has become a key enabling technology for voice-controlled applications, accessibility-oriented systems, and desktop automation environments (Karibayeva et al., 2025; Kheddar et al., 2024).

The integration of AI and speech-recognition technologies has enabled the development of more natural, efficient, and accessible forms of interaction, providing an important foundation for Human–Computer Interaction (HCI), which focuses on effective communication between users and computer systems.

2.3 Human–Computer Interaction and Accessibility

Human–Computer Interaction (HCI) is an interdisciplinary field that focuses on the design, evaluation, and implementation of interactive computing systems for human use. The primary objective of HCI is to improve the effectiveness, efficiency, usability, and user experience of computer systems by facilitating effective communication between users and technology (Poirier et al., 2023; Pulikottil et al., 2026).

Traditionally, interaction between humans and computers relied heavily on physical input devices such as keyboards, computer mice, and touchpads. Although these devices remain widely used, modern computing environments increasingly utilise more natural interaction approaches including gesture recognition, eye tracking, touch interaction, and speech-controlled interfaces. These technologies aim to provide more intuitive and accessible methods of interaction while reducing the physical effort required to operate computer systems (Zhang et al., 2022; Poirier et al., 2023).

Accessibility has become an important area within HCI research because computer systems should be usable by individuals with diverse physical, sensory, and cognitive abilities. Accessibility-oriented computing seeks to remove barriers that may prevent users from interacting effectively with digital technologies. Researchers have therefore explored alternative interaction techniques that support users with mobility impairments, repetitive strain injuries, visual impairments, and other physical limitations (Mott et al., 2022; Matre & Cameron, 2024).

Speech-controlled interaction has emerged as one of the most promising accessibility technologies because it enables users to interact with computer systems through spoken language rather than conventional input devices. Voice-based interfaces can support hands-free interaction, improve user independence, and reduce the physical demands associated with repetitive computer operations. Consequently, speech recognition has become an important component of accessibility-oriented HCI systems and intelligent interactive applications (Volohtchuk et al., 2024; Pulikottil et al., 2026).

Despite these advantages, researchers also identify several challenges associated with speech-enabled interaction, including pronunciation variation, accent diversity, environmental noise, microphone quality, and speech inconsistency. These factors may affect recognition accuracy and overall system usability. Therefore, the effectiveness of speech-controlled systems must be evaluated carefully to ensure reliable operation across different user groups and interaction environments (Sand et al., 2025; Almgren Bäck et al., 2024).

The increasing importance of speech-based interaction within HCI has led to significant research efforts in Automatic Speech Recognition (ASR), which provides the technological foundation for converting spoken language into machine-understandable commands and actions.

2.4 Voice-Based Interaction in Human–Computer Interaction

Voice-based interaction refers to a communication method that enables users to interact with computer systems through spoken language. As one of the most widely adopted Natural User Interface (NUI) technologies, voice interaction allows users to perform tasks, issue commands, and access information without relying on traditional input devices such as keyboards or computer mice. The increasing popularity of voice-controlled systems reflects the growing demand for more natural, intuitive, and efficient forms of human-computer interaction (Poirier et al., 2023; Pulikottil et al., 2026).

Recent advancements in Artificial Intelligence, Natural Language Processing (NLP), and speech-recognition technologies have significantly improved the effectiveness of voice-based interfaces. Modern voice-controlled systems are capable of recognising spoken commands, interpreting user intentions, and executing corresponding actions in real time. As a result, voice interaction has been increasingly integrated into virtual assistants, smart-home devices, healthcare applications, automotive systems, and intelligent automation platforms (Fatehi et al., 2025; Kheddar et al., 2024).

One major advantage of voice-based interaction is its ability to support hands-free operation. Users can interact with computer systems while performing other activities, thereby improving convenience and productivity. In accessibility-oriented environments, voice interfaces provide

alternative interaction mechanisms for individuals who may experience difficulties using conventional input devices because of physical disabilities, mobility limitations, or repetitive strain conditions (Matre & Cameron, 2024; Volohtchuk et al., 2024).

Table 2.1 summarises several application areas where voice-controlled interfaces provide significant benefits.

Application Area	Benefit
Accessibility	Hands-free computer interaction
Healthcare	Reduced physical interaction
Education	Assistive learning support
Smart Environment	Natural interaction
Desktop Automation	Faster repetitive task execution

Table 2.1 Benefits of Voice-Controlled Interfaces

Table 2.1 illustrates the practical value of voice-controlled interfaces across various application domains. The table demonstrates that voice interaction is not limited to virtual assistants or speech transcription applications but also supports accessibility, healthcare, education, smart environments, and desktop automation. These application areas highlight the growing importance of voice-enabled technologies and justify the development of the proposed voice-controlled desktop automation system.

Despite their advantages, voice-controlled interfaces continue to face several challenges. Environmental noise, pronunciation differences, accent variation, microphone quality, and speech inconsistency may affect recognition performance and user satisfaction. Consequently, effective voice-controlled systems require robust speech-recognition technologies capable of operating accurately and reliably under different interaction conditions (Sand et al., 2025; Almgren Bäck et al., 2024).

To address these challenges, modern voice-based interaction systems rely heavily on Automatic Speech Recognition (ASR) technologies, which enable the conversion of spoken language into machine-understandable commands and actions. Therefore, understanding the underlying principles of ASR is essential for the development of effective speech-controlled desktop automation systems.

2.5 Automatic Speech Recognition Technology

Automatic Speech Recognition (ASR) is a technology that enables computer systems to convert spoken language into machine-readable text or executable commands. ASR serves as one of the most important enabling technologies for voice-controlled applications because it allows computers to understand and respond to human speech. The technology integrates concepts from Artificial Intelligence (AI), machine learning, signal processing, pattern recognition, and Natural Language Processing (NLP) to interpret spoken input and transform it into meaningful outputs (Fatehi et al., 2025; Kheddar et al., 2024).

The primary objective of ASR is to bridge the communication gap between humans and computers by enabling speech-based interaction. Instead of relying solely on traditional input devices such as keyboards and computer mice, users can communicate with computer systems through spoken commands. This capability has contributed significantly to the development of intelligent assistants, accessibility-oriented applications, smart devices, healthcare systems, robotics, and desktop automation platforms (Ferraro et al., 2023; Fadel et al., 2023).

Modern ASR systems have evolved considerably over the past decades. Early speech-recognition systems relied on rule-based and statistical approaches that required extensive manual configuration and were often limited to constrained vocabularies. Advances in machine learning and deep learning have subsequently improved recognition accuracy, contextual understanding, and robustness against environmental variability. As a result, modern ASR technologies are capable of supporting real-time speech interaction across a wide range of application domains (Ahlawat et al., 2025; Karibayeva et al., 2025).

ASR technologies can generally be applied in two primary ways. The first involves speech-to-text transcription, where spoken language is converted into textual output for documentation, communication, or information retrieval purposes. The second involves command-and-control applications, where recognised speech is translated into executable actions that allow users to interact directly with computer systems. The command-and-control approach is particularly relevant to desktop automation systems because recognised commands trigger specific operating-system functions and application-level operations (Di Leo et al., 2025; Hai et al., 2025).

In voice-controlled desktop automation environments, ASR performance is influenced by several factors, including recognition accuracy, response latency, environmental robustness, computational efficiency, and privacy protection. Poor recognition performance may result in incorrect command execution, reduced usability, and lower user confidence in the system. Therefore, modern ASR research increasingly focuses not only on transcription accuracy but also on reliability, responsiveness, deployment practicality, and real-world usability (Ferraro et al., 2023; Fadel et al., 2023).

Because ASR technologies have evolved substantially over time, understanding the historical development of speech-recognition architectures is important for evaluating the strengths and limitations of modern speech-recognition systems. Therefore, the following section discusses the evolution of Automatic Speech Recognition from classical statistical approaches to modern deep-learning and Transformer-based architectures.

2.6 Evolution of Automatic Speech Recognition

The development of Automatic Speech Recognition (ASR) technology has progressed significantly over several decades. Advances in computing power, machine learning, and artificial intelligence have transformed speech recognition from simple command-based systems into highly sophisticated technologies capable of understanding continuous and natural speech. The evolution of ASR can generally be classified into three major stages: classical statistical approaches, hybrid deep-learning approaches, and end-to-end neural architectures (Malik et al., 2021; Fatehi et al., 2025).

Early ASR systems primarily relied on Hidden Markov Models (HMMs) combined with Gaussian Mixture Models (GMMs). These systems approached speech recognition as a probabilistic sequence-modelling problem in which speech signals were represented as a sequence of acoustic states. Language models and acoustic models were used together to determine the most probable word sequence from an observed speech signal (Malik et al., 2021; Fatehi et al., 2025).

Mel Frequency Cepstral Coefficients (MFCCs) became one of the most widely adopted feature-extraction techniques because they effectively represent perceptually relevant

characteristics of human hearing. Classical HMM-GMM systems were computationally efficient and suitable for low-resource environments. However, researchers identified several limitations including weak contextual understanding, dependence on handcrafted features, and reduced robustness under noisy or spontaneous speech conditions (Fendji et al., 2022; Natarajan et al., 2025).

The probabilistic formulation of classical ASR systems can be represented as:

$$P(W|X) = \frac{P(X|W)P(W)}{P(X)}$$

where:

- $P(W|X)$ represents the probability of a word sequence given the observed speech signal,
- $P(X|W)$ represents the acoustic model probability,
- $P(W)$ represents the language model probability.

The introduction of Deep Neural Networks (DNNs), Recurrent Neural Networks (RNNs), and Long Short-Term Memory (LSTM) architectures significantly improved speech-recognition performance by enabling systems to learn complex acoustic and temporal patterns directly from training data. Hybrid DNN-HMM architectures subsequently became dominant because they combined the temporal modelling capability of HMMs with the representational power of deep learning (Ahlawat et al., 2025; Kheddar et al., 2024).

More recently, speech-recognition research has shifted towards end-to-end architectures including Connectionist Temporal Classification (CTC), Recurrent Neural Network Transducers (RNN-T), Transformer-based models, and Conformer architectures. These approaches reduce the dependence on manually separated processing stages by learning direct mappings between speech signals and textual outputs. Although Transformer-based systems currently achieve state-of-the-art recognition accuracy, they often require substantial computational resources and large-scale training infrastructure, which may limit their suitability for lightweight offline desktop automation environments (Zhang et al., 2025; Karibayeva et al., 2025).

2.7 Automatic Speech Recognition Pipeline

Although speech-recognition architectures have evolved considerably over time, most Automatic Speech Recognition (ASR) systems continue to follow a similar processing pipeline. This pipeline consists of several interconnected stages that transform raw speech signals into machine-understandable text or executable commands. Understanding the ASR pipeline is important because each processing stage directly influences recognition accuracy, computational efficiency, response latency, and overall system performance (Malik et al., 2021; Natarajan et al., 2025).

The process begins with speech acquisition, where audio signals are captured through a microphone. The captured speech then undergoes preprocessing operations such as noise reduction, silence removal, normalization, framing, and filtering. These preprocessing techniques help improve speech quality and reduce the influence of environmental noise before further analysis is performed (Fendji et al., 2022; Fatehi et al., 2025).

Following preprocessing, the speech signal is converted into numerical feature representations through feature-extraction techniques. Among the various feature-extraction methods available, Mel Frequency Cepstral Coefficients (MFCCs) remain one of the most widely used because they effectively represent the perceptually important characteristics of human speech while reducing computational complexity.

MFCC feature extraction can be mathematically represented as:

$$C_n = \sum_{k=1}^K \log(S_k) \cos \left[n \left(k - \frac{1}{2} \right) \frac{\pi}{K} \right]$$

Where:

- S_K represents the filter bank energies,
- K represents the number of Mel filters
- n represents the cepstral coefficient index.

The extracted features are subsequently processed by acoustic models that estimate phoneme probabilities from the speech signal. Language models are then applied to determine the most

probable sequence of words based on linguistic patterns and contextual information. Finally, decoding algorithms such as Weighted Finite State Transducers (WFSTs) generate the final recognition output by identifying the most likely interpretation of the spoken utterance (Fendji et al., 2022; Fatehi et al., 2025).

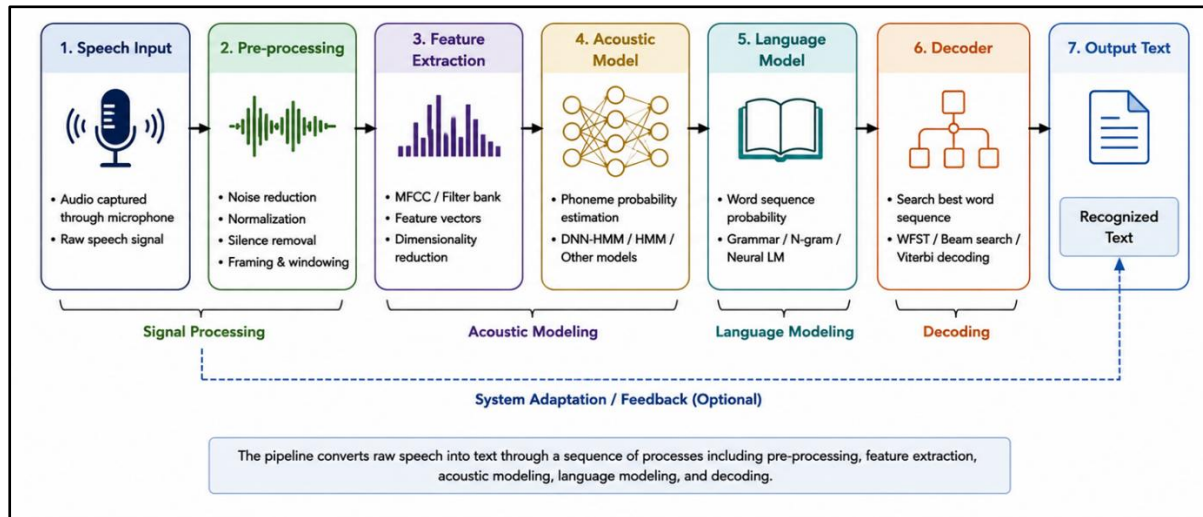


Figure 2.1 illustrates the general Automatic Speech Recognition processing pipeline used in many speech-enabled systems.

Figure 2.1 illustrates the sequence of processing stages involved in converting spoken language into machine-understandable outputs. The pipeline demonstrates how speech signals are progressively transformed from raw audio into recognised commands through feature extraction, acoustic modelling, language modelling, and decoding processes. Understanding this pipeline is important because the effectiveness of each stage directly affects recognition accuracy, response latency, and command-execution reliability. Within the proposed desktop automation system, the recognised output is subsequently mapped to predefined automation functions that enable real-time control of computer operations.

2.8 Cloud-Based and Offline Speech Recognition

One of the most important developments in modern Automatic Speech Recognition (ASR) research is the distinction between cloud-based and offline speech-recognition systems. Both approaches aim to convert spoken language into machine-understandable outputs; however,

they differ significantly in terms of processing architecture, deployment requirements, privacy considerations, and system performance (Ferraro et al., 2023; Fadel et al., 2023).

Cloud-based ASR systems process speech data using remote servers and cloud infrastructure. When a user provides a speech input, the audio data is transmitted through the internet to cloud servers where speech processing, acoustic modelling, language modelling, and recognition are performed. Cloud-based systems generally achieve high recognition accuracy because they utilise large-scale datasets, powerful computational resources, and continuously updated machine-learning models (Ferraro et al., 2023; Fatehi et al., 2025).

Google Speech-to-Text represents one of the most widely used cloud-based ASR solutions. The platform supports multilingual recognition, confidence scoring, real-time streaming recognition, and large-vocabulary speech processing. Despite these advantages, cloud-based systems introduce several challenges including internet dependency, variable response latency, privacy concerns, and reduced deployment reliability in environments with unstable network connectivity (Mulfari & Villari, 2024; Di Leo et al., 2025).

In contrast, offline speech-recognition systems perform all speech processing locally on the user's device without requiring internet connectivity. Frameworks such as PocketSphinx, CMUSphinx, and Vosk are designed to support offline deployment while maintaining acceptable recognition performance. Because speech data remains within the local device, offline ASR systems provide improved privacy protection, predictable response latency, and greater operational reliability (Gondi & Pratap, 2021; Netinant et al., 2026).

Offline speech-recognition technologies are particularly important in accessibility-oriented applications, edge-computing environments, security-sensitive systems, and desktop automation platforms where continuous internet access may not be available. However, offline systems often operate under hardware limitations and may achieve slightly lower recognition accuracy than advanced cloud-based alternatives (Hai et al., 2025; Zhang et al., 2025).

Feature	Cloud-Based ASR	Offline ASR
Internet Requirement	Required	Not Required
Privacy Protection	Lower	Higher
Latency Stability	Variable	Predictable
Resource Usage	Cloud Infrastructure	Local Device

Recognition Accuracy	Very High	Moderate to High
Deployment Cost	API or Subscription Cost	One-Time Deployment
Best Use Case	Large-Scale Transcription	Edge Computing and Desktop Automation

Table 2.3 compares the major characteristics of cloud-based and offline speech-recognition systems.

Table 2.3 highlights the fundamental differences between cloud-based and offline speech-recognition approaches. The comparison demonstrates that cloud-based systems generally provide superior recognition accuracy through access to powerful computational infrastructure, whereas offline systems offer stronger privacy protection, lower deployment dependence, and more predictable response latency. Since the proposed study focuses on offline voice-controlled desktop automation, the characteristics of offline ASR systems are particularly relevant because they support local speech processing, reduced privacy risks, and reliable operation without internet connectivity. These advantages directly address the limitations identified in the research problem statement and justify the selection of offline speech-recognition technologies for the proposed system.

2.9 Speech Recognition Frameworks

2.9.1 PocketSphinx and CMUSphinx

PocketSphinx is a lightweight offline speech-recognition engine developed as part of the broader CMUSphinx project established by Carnegie Mellon University. CMUSphinx provides a complete suite of speech-recognition tools including acoustic-modelling utilities, language-modelling frameworks, training resources, and recognition engines designed for offline speech-processing applications (Fendji et al., 2022; Fatehi et al., 2025).

PocketSphinx primarily employs Hidden Markov Models (HMMs), Gaussian Mixture Models (GMMs), and Mel Frequency Cepstral Coefficients (MFCCs) to perform speech recognition. The framework supports keyword spotting, grammar-based recognition, and constrained-vocabulary command recognition, making it particularly suitable for command-and-control applications (Ouisaadane & Safi, 2021; Zhang et al., 2023).

One major advantage of PocketSphinx is its low computational requirement. The framework can operate efficiently on devices with limited processing power and memory resources while maintaining fully offline functionality. These characteristics have contributed to its widespread use in embedded systems, educational projects, assistive technologies, and lightweight automation applications.

Despite these advantages, recent studies indicate that PocketSphinx generally demonstrates lower recognition accuracy, weaker environmental robustness, and limited contextual adaptation compared to modern deep-learning-based ASR frameworks. In addition, the framework lacks advanced confidence-scoring mechanisms and sophisticated out-of-vocabulary handling, which may affect command-recognition reliability in dynamic environments (Djeffal et al., 2025; Natarajan et al., 2025).

CMUSphinx remains valuable in research environments because it provides greater flexibility for acoustic-model customization and experimental ASR development. Consequently, PocketSphinx and CMUSphinx continue to be used in offline speech-recognition research where lightweight deployment, customization, and local processing are important requirements.

2.9.2 Vosk Speech Recognition Framework

Vosk is a modern offline speech-recognition framework built upon the Kaldi speech-recognition toolkit. Unlike traditional HMM-GMM systems, Vosk integrates hybrid Deep Neural Network-Hidden Markov Model (DNN-HMM) architectures with optimized decoding mechanisms to improve recognition accuracy while maintaining offline deployment capability (Di Leo et al., 2025; Hai et al., 2025).

The framework supports multilingual recognition, streaming speech processing, phrase adaptation, speaker adaptation, and edge-device deployment. These capabilities enable Vosk to achieve strong recognition performance while operating entirely on local hardware without requiring cloud infrastructure.

Several comparative studies demonstrate that Vosk achieves significantly lower Word Error Rates (WER) and improved environmental robustness compared to PocketSphinx. In addition, Vosk provides relatively low response latency, making it suitable for real-time speech-controlled applications and desktop automation environments (Netinant et al., 2026; Karibayeva et al., 2025).

A major advantage of Vosk is its privacy-preserving architecture. Since speech processing occurs locally, audio data is not transmitted to external servers, thereby reducing privacy and cybersecurity risks. This characteristic is particularly important for desktop automation systems where voice commands may trigger operating-system functions or access sensitive information.

Due to its balance between recognition accuracy, computational efficiency, offline deployment capability, privacy protection, and low-latency performance, Vosk is selected as the primary speech-recognition framework for the proposed desktop automation system.

2.10 Comparative Analysis of Speech Recognition Frameworks

The literature consistently demonstrates that no single speech-recognition framework is universally optimal for all deployment environments. Instead, each framework presents different trade-offs involving recognition accuracy, computational efficiency, deployment flexibility, environmental robustness, privacy protection, and response latency (Ferraro et al., 2023; Fadel et al., 2023).

Speech-recognition frameworks are typically developed to address different operational requirements. Lightweight frameworks prioritise computational efficiency and offline deployment, whereas cloud-based frameworks focus on maximizing recognition accuracy through access to large-scale computational resources. Therefore, selecting an appropriate speech-recognition framework depends heavily on the intended application environment and system requirements.

Framework	Strengths	Weaknesses	Suitable Application
PocketSphinx	Lightweight, offline, low computational usage	Weak environmental robustness and higher WER	Constrained command systems
CMUSphinx	Flexible acoustic and language modelling toolkit	Complex configuration and lower modern recognition accuracy	Research and offline ASR experimentation
Vosk	Offline, multilingual, low latency, strong privacy protection	Slightly lower accuracy than cloud-based systems	Desktop automation and edge devices
Google Speech-to-Text	High recognition accuracy and confidence scoring	Internet dependency and privacy concerns	Cloud voice assistants

Table 2.4 presents a comparative analysis of the major speech-recognition frameworks reviewed in this study.

Table 2.4 demonstrates that each framework addresses different speech-recognition requirements. PocketSphinx was designed for lightweight command-based environments where computational efficiency and offline deployment are prioritised. As a result, it is commonly used in embedded systems and constrained command-recognition applications. However, its recognition accuracy and environmental robustness are generally lower than modern deep-learning-based alternatives (Ouisaadane & Safi, 2021; Djeflal et al., 2025).

CMUSphinx provides greater flexibility for acoustic-model customization, language-model development, and speech-recognition experimentation. Consequently, the framework remains valuable for academic research and offline ASR development despite its comparatively complex configuration process and lower recognition performance (Fendji et al., 2022; Fatehi et al., 2025).

Vosk was developed to provide a balance between offline deployment capability, recognition accuracy, environmental robustness, and computational efficiency. The framework incorporates modern hybrid DNN-HMM architectures while maintaining local speech processing. These characteristics make Vosk particularly suitable for desktop automation systems, accessibility-oriented applications, and edge-computing environments where privacy protection and low-latency performance are important requirements (Hai et al., 2025; Netinant et al., 2026).

Google Speech-to-Text achieves superior recognition accuracy through large-scale cloud infrastructure and continuously updated deep-learning models. Nevertheless, its dependence

on internet connectivity and external cloud services introduces privacy concerns and deployment limitations for offline environments (Ferraro et al., 2023; Mulfari & Villari, 2024).

Based on the literature reviewed, Vosk provides the most appropriate balance between recognition accuracy, privacy protection, offline deployment capability, and real-time responsiveness. Therefore, it was selected as the primary speech-recognition framework for the proposed voice-controlled desktop automation system.

2.11 Desktop Automation Technologies

Traditional desktop automation systems rely on predefined scripts, keyboard shortcuts, GUI automation tools, and scheduling mechanisms. While these methods improve efficiency, they often require manual setup and user interaction. Recent advances in Artificial Intelligence and Human–Computer Interaction have introduced speech recognition into automation systems, allowing users to control computers using natural voice commands instead of traditional input devices.

Speech-controlled desktop automation combines Automatic Speech Recognition (ASR) with automation frameworks to convert spoken commands into executable actions. Once a command is recognised, the system interprets it and performs the corresponding task automatically. This approach enables hands-free interaction, improves accessibility, and reduces effort in repetitive computing tasks (Matre & Cameron, 2024; Volochtchuk et al., 2024).

Python is widely used for building such systems due to its powerful libraries for GUI control, system interaction, and web automation. Commonly used tools include PyAutoGUI for mouse and keyboard control, the Webbrowser module for opening websites, the Subprocess module for running external applications, and the OS module for file and system-level operations.

Together, these technologies make it possible to build efficient voice-controlled desktop automation systems capable of performing tasks such as launching applications, navigating the web, controlling the cursor, and managing system operations.

Table 2.5 summarises the primary desktop automation technologies utilised in speech-controlled desktop environments.

Technology	Primary Function	Application in Proposed System
PyAutoGUI	GUI automation and mouse control	Cursor movement, clicking, scrolling
Webbrowser	Website launching and navigation	Browser automation
Subprocess	Application execution	Launching and closing applications
OS Module	Operating-system interaction	System-level operations

Table 2.5 Desktop Automation Technologies

Table 2.5 summarises the primary desktop automation technologies employed within voice-controlled automation environments. Each technology performs a specific role within the automation architecture, ranging from graphical user-interface control to application execution and operating-system interaction. The integration of these automation technologies with speech-recognition frameworks enables the development of a complete voice-controlled desktop automation system capable of translating recognised speech commands into real-time computer actions.

2.12 Noise Robustness in Speech Recognition

Environmental noise remains one of the most significant challenges affecting the performance of Automatic Speech Recognition (ASR) systems. Since speech-recognition technologies rely on the accurate capture and processing of audio signals, background noise may interfere with speech features and reduce recognition accuracy. Common sources of environmental noise include conversations, office equipment, traffic sounds, ventilation systems, and public-place activities. Consequently, noise robustness has become an important evaluation criterion in modern speech-recognition research (Ouisaadane & Safi, 2021; Zhang et al., 2023).

The impact of environmental noise is commonly measured using the Signal-to-Noise Ratio (SNR), which represents the relationship between the desired speech signal and background noise. Higher SNR values indicate clearer speech signals, whereas lower SNR values indicate more challenging recognition conditions. Numerous studies demonstrate that recognition performance deteriorates significantly as environmental noise increases and SNR values decrease (Djeffal et al., 2025; Natarajan et al., 2025).

Controlled experiments involving PocketSphinx and similar offline speech-recognition systems have demonstrated substantial recognition degradation under noisy operating conditions. Table 2.6 presents representative findings reported in the literature.

Environment	SNR Condition	Recognition Result
Near-clean environment	>15 dB	98.25% WRR
Printing room	15 dB	68.5% WRR
Faculty hall	10 dB	49.62% WRR
Café	6 dB	37.62% WRR
Bus	2 dB	18.12% WRR

Table 2.6 Noise Robustness Evaluation

Table 2.6 demonstrates the relationship between environmental noise and speech-recognition performance. The results indicate that recognition accuracy decreases substantially as Signal-to-Noise Ratio (SNR) values become lower. While speech-recognition systems achieve excellent performance in near-clean environments, recognition reliability deteriorates rapidly under noisy conditions such as cafés, public halls, and transportation environments.

The purpose of this table is to highlight the importance of environmental robustness in speech-controlled systems. Since the proposed desktop automation system relies on accurate command recognition, environmental noise may directly affect command execution reliability and overall system usability. Incorrectly recognised commands may lead to unintended operations, reduced user confidence, and lower system effectiveness. Therefore, noise robustness evaluation is an important consideration in the development and assessment of offline voice-controlled desktop automation systems.

Recent studies involving Vosk-based speech-recognition systems report improved environmental adaptability and stronger contextual recognition capabilities compared to traditional HMM-GMM frameworks. The integration of deep-learning acoustic models enables Vosk to maintain more stable recognition performance under moderate noise conditions. Nevertheless, many existing evaluations continue to be conducted under controlled laboratory environments rather than realistic desktop automation scenarios involving microphone

variability, spontaneous speech patterns, and dynamic background noise (Djeffal et al., 2025; Natarajan et al., 2025).

Consequently, the literature still lacks sufficient cross-framework evaluations examining the effects of environmental noise on speech-controlled desktop automation systems operating in practical computing environments. This limitation further justifies the need for evaluating environmental robustness within the proposed study.

2.13 Latency and Real-Time Performance

Latency is a critical performance indicator in speech-controlled systems because it directly affects user experience, system responsiveness, and operational efficiency. In voice-controlled desktop automation environments, users expect immediate execution of actions following spoken commands. Excessive delays may reduce usability, interrupt workflow, and decrease user confidence in the system (Di Leo et al., 2025; Gondi & Pratap, 2021).

Speech-recognition latency refers to the time required for a system to process spoken input and generate a recognition output. Researchers commonly evaluate latency using metrics such as per-word processing time, utterance-level recognition delay, and end-to-end response time. End-to-end latency is particularly important in desktop automation systems because it reflects the total delay experienced by users from speech input to command execution (Di Leo et al., 2025; Netinant et al., 2026).

Previous studies indicate that speech-controlled systems should maintain low response latency to support natural interaction. Most researchers agree that delays exceeding one second may negatively affect perceived responsiveness and overall usability, particularly within interactive command-and-control environments (Ferraro et al., 2023; Hai et al., 2025).

Table 2.7 presents latency and Word Error Rate (WER) benchmarks reported in previous speech-recognition studies.

Study	Engine	Dataset	WER	Latency
Whetten et al.	Google Speech	LibriSpeech	13.2%	0.197 s/word
Whetten et al.	Vosk	LibriSpeech	6.4%–33.4%	0.104–0.167 s/word
Whetten et al.	PocketSphinx	LibriSpeech	31.8%–40.4%	0.105–0.220 s/word
Di Leo et al.	Vosk	Common Voice	2.4%–4.9%	<1.1 s total latency

Table 2.7 Literature WER and Latency Benchmarks

Table 2.7 compares Word Error Rate (WER) and latency performance reported for several speech-recognition frameworks in previous studies. Word Error Rate measures recognition accuracy, whereas latency measures system responsiveness. Together, these metrics provide an indication of how effectively a speech-recognition framework can support real-time interaction.

The purpose of this table is to evaluate the trade-off between recognition accuracy and response speed. A speech-recognition system with high accuracy but excessive latency may not be suitable for real-time desktop automation because users expect immediate command execution. Similarly, extremely fast recognition with poor accuracy may result in incorrect command activation and unreliable system behaviour.

The results indicate that Vosk generally provides a favourable balance between recognition quality and responsiveness. Although cloud-based systems such as Google Speech-to-Text often achieve strong recognition performance, offline frameworks such as Vosk offer competitive accuracy while maintaining low response latency and local processing capability. These characteristics make Vosk particularly suitable for real-time voice-controlled desktop automation environments.

Consequently, latency evaluation forms an important component of the proposed study because the effectiveness of the developed system depends not only on recognition accuracy but also on its ability to respond quickly and reliably to user commands.

Recent studies consistently demonstrate that modern offline speech-recognition frameworks have improved significantly in terms of latency performance. Advances in decoding algorithms, streaming recognition techniques, and lightweight neural-network architectures have reduced processing delays while maintaining acceptable recognition accuracy. These

developments have contributed to the growing adoption of offline ASR systems in accessibility-oriented applications, edge-computing environments, and desktop automation platforms (Di Leo et al., 2025; Netinant et al., 2026).

Despite these improvements, latency performance remains highly dependent on hardware specifications, microphone quality, command complexity, and environmental conditions. Therefore, evaluating response latency under realistic operating conditions remains essential for determining the practical usability of speech-controlled desktop automation systems.

2.14 Research Gap

Although extensive research has been conducted on Automatic Speech Recognition (ASR) technologies, several limitations remain within the existing literature. Most studies primarily focus on improving speech-to-text transcription accuracy, reducing Word Error Rate (WER), and developing advanced speech-recognition architectures. While these contributions have significantly advanced ASR performance, relatively limited attention has been given to the practical implementation of speech-recognition technologies within offline desktop automation environments (Ferraro et al., 2023; Fadel et al., 2023).

Many existing studies evaluate ASR systems under controlled laboratory conditions using benchmark datasets such as LibriSpeech and Common Voice. Although these evaluations provide valuable performance indicators, they may not accurately reflect real-world desktop automation environments involving microphone variability, spontaneous speech patterns, user pronunciation differences, and dynamic environmental noise. Consequently, the practical effectiveness of speech-recognition systems within desktop automation applications remains insufficiently investigated (Ouisaadane & Safi, 2021; Zhang et al., 2023).

Furthermore, previous research frequently concentrates on cloud-based speech-recognition platforms and virtual-assistant applications. While cloud-based systems often achieve high recognition accuracy, their dependence on internet connectivity introduces concerns relating to privacy, security, deployment reliability, and response-latency consistency. Comparatively fewer studies investigate the implementation of fully offline speech-recognition solutions

capable of performing real-time desktop automation without relying on external cloud infrastructure (Mulfari & Villari, 2024; Di Leo et al., 2025).

The literature also reveals a lack of comprehensive comparative evaluations involving offline speech-recognition frameworks such as Vosk, PocketSphinx, and CMUSphinx within desktop automation environments. Existing studies typically evaluate these frameworks independently or focus primarily on recognition performance without considering command-execution reliability, usability, environmental robustness, and practical deployment requirements (Djeffal et al., 2025; Netinant et al., 2026).

In addition, limited research investigates the integration of offline speech recognition with Python-based desktop automation technologies for controlling operating-system functions, application launching, browser navigation, multimedia interaction, and cursor operations through voice commands. This represents an important research opportunity because such systems may improve accessibility, support hands-free interaction, and provide greater privacy protection through local processing.

Therefore, this research aims to address these gaps by developing and evaluating an offline voice-controlled desktop automation system that integrates speech-recognition technologies with desktop automation frameworks. The study focuses on command-recognition accuracy, command-execution reliability, environmental robustness, usability, privacy protection, and real-time desktop automation performance within practical computing environments.

Author	Focus Area	Main Findings	Limitation
Whetten et al.	ASR latency and WER	Vosk achieved balanced performance	Limited desktop automation focus
Di Leo et al.	Edge ASR systems	Low latency with Vosk	Small deployment environment
Ouisaadane et al.	Noise robustness	Accuracy decreased significantly under noise	Focused mainly on PocketSphinx
Vieira et al.	Accessibility systems	Voice systems improved user independence	Limited technical evaluation

Table 2.8 Summary of Literature Findings

Table 2.8 summarises key findings identified throughout the literature review. Although previous studies have contributed significantly to speech-recognition research, several limitations remain regarding offline desktop automation, command-execution reliability, environmental robustness, usability evaluation, and real-world deployment conditions. These

limitations collectively establish the research gap addressed by the present study and provide the justification for developing the proposed offline voice-controlled desktop automation system.

2.15 Chapter Summary

This chapter critically reviewed the literature related to Artificial Intelligence (AI), Human–Computer Interaction (HCI), voice-based interaction, Automatic Speech Recognition (ASR) technologies, speech-recognition architectures, desktop automation technologies, environmental robustness, latency, privacy considerations, and accessibility-oriented computing.

The review demonstrated that AI has significantly influenced the development of intelligent interactive systems by enabling more natural and efficient communication between humans and computers. Within HCI, speech-based interaction has emerged as an important accessibility-oriented technology that supports hands-free operation, improves user independence, and enhances interaction efficiency in modern computing environments.

The literature further revealed that Automatic Speech Recognition has evolved substantially from classical Hidden Markov Model (HMM) and Gaussian Mixture Model (GMM) approaches to modern deep-learning and Transformer-based architectures. These advancements have improved recognition accuracy, contextual understanding, environmental adaptability, and real-time performance. Nevertheless, trade-offs involving computational requirements, deployment complexity, latency, and privacy remain important considerations in practical speech-recognition applications.

The comparison of cloud-based and offline speech-recognition systems demonstrated that offline ASR technologies provide advantages in privacy protection, deployment reliability, and predictable latency. Among the reviewed speech-recognition frameworks, Vosk emerged as the most suitable solution for offline desktop automation because it offers a balanced combination of recognition accuracy, low response latency, multilingual support, environmental robustness, and local speech processing capability. PocketSphinx and CMUSphinx remain valuable for lightweight and experimental deployments, whereas Google Speech-to-Text provides strong

recognition performance at the cost of internet dependency and cloud-based processing requirements.

The review also highlighted the importance of desktop automation technologies in enabling practical voice-controlled interaction. Python-based automation frameworks such as PyAutoGUI, Webbrowser, Subprocess, and OS modules provide the necessary functionality for translating recognised speech commands into executable desktop operations.

Finally, the literature identified several research gaps involving command-execution reliability, real-world desktop automation evaluation, environmental robustness, usability assessment, and the practical deployment of offline speech-recognition systems. These gaps provide the justification for the present study and support the development of the proposed offline voice-controlled desktop automation system.

The findings presented in this chapter establish the theoretical and technological foundation for the research methodology discussed in Chapter 3. The following chapter describes the research design, system architecture, implementation procedures, testing strategies, and evaluation methods used to develop and assess the proposed system.

CHAPTER 3

METHODOLOGY

3.1 Introduction

This chapter presents the methodology adopted for the design, development, and implementation of the proposed Offline Voice-Controlled Desktop Automation System using Vosk, PocketSphinx, and CMUSphinx speech-recognition frameworks. The methodology provides a structured approach for transforming the research problem identified in Chapter 1 and the literature findings discussed in Chapter 2 into a functional software solution capable of supporting offline voice-based desktop interaction.

The primary objective of this chapter is to explain the systematic procedures followed throughout the development process. The chapter describes how system requirements were identified, how the overall architecture was designed, how the speech-recognition frameworks were integrated, and how desktop automation functionalities were implemented using Python-based automation technologies. The methodology also explains the operational workflow and functional modules developed to support voice-controlled interaction within a desktop computing environment.

A prototype-based development approach was adopted for this study because the project required the design, implementation, testing, and refinement of a working software system. The development process was conducted iteratively, allowing system components to be gradually integrated and improved throughout the implementation phase. This approach was suitable because the proposed system combines multiple technologies, including offline speech recognition, command processing, desktop automation, and runtime monitoring.

The proposed system was developed using three offline Automatic Speech Recognition (ASR) frameworks, namely Vosk, PocketSphinx, and CMUSphinx. These frameworks were integrated with Python automation libraries to enable users to perform desktop operations through voice commands without relying on internet connectivity or cloud-based speech-processing services. The implemented functionalities include mouse control, scrolling operations, browser

automation, multimedia navigation, application launching, audio control, and runtime monitoring.

This chapter is organised into several sections. It begins with the research design and development methodology, followed by requirement analysis, system architecture, operational workflow, and the implementation of the major functional modules developed for the proposed system. Together, these sections provide a comprehensive description of the methodology and implementation steps followed throughout the development of the Offline Voice-Controlled Desktop Automation System.

3.2 Research Design and Development Methodology

This study adopted a prototype-based system development methodology for the design and implementation of the proposed Offline Voice-Controlled Desktop Automation System. The methodology was selected because the objective of the research was not only to investigate speech-recognition technologies but also to develop a functional software system capable of supporting real-time desktop automation through offline voice interaction. Prototype-based methodologies are commonly used in software engineering and Human–Computer Interaction (HCI) projects because they support iterative development, continuous testing, and progressive system improvement throughout the development lifecycle.

The prototype-based approach was considered appropriate for this study because the proposed system integrates multiple technologies, including offline Automatic Speech Recognition (ASR), command-processing mechanisms, desktop automation libraries, and operating-system interaction modules. Developing the system through incremental stages allowed individual components to be implemented, tested, refined, and integrated gradually before constructing the final working system.

To ensure systematic development, the research process was divided into several stages, beginning with requirement gathering and ending with system refinement. Each stage contributed to the successful implementation of the proposed framework and ensured that the final system satisfied the objectives established in Chapter 1.

The relationship between the research questions, research objectives, and methodological activities conducted throughout the study is presented in Table 3.1.

	Research Question (RQ)	Research Objective (RO)	Research Activity / Methodology	Evaluation Method
1	How can Vosk, PocketSphinx, and CMUSphinx offline speech-recognition technologies be integrated with Python-based automation tools to support real-time desktop automation?	To develop an offline voice-controlled desktop automation system using Vosk, PocketSphinx, and CMUSphinx speech-recognition engines integrated with Python-based automation technologies to support accessible hands-free human-computer interaction.	Requirement analysis, system architecture design, ASR integration, command-processing development, desktop automation implementation.	Functional testing and prototype validation.
2	How effectively can the proposed voice-controlled system recognize speech commands and execute desktop automation tasks such as mouse control, scrolling, browser automation, application launching, and multimedia control?	To implement voice-command functionalities for mouse control, scrolling operations, browser automation, multimedia interaction, application launching, and operating-system control using PyAutoGUI, Webbrowser, and Subprocess modules.	Command mapping, automation module development, application integration, execution testing.	Command-execution validation and functional testing.
3	What is the recognition accuracy, command-execution success rate, and usability level achieved by the proposed offline voice-controlled desktop automation system?	To evaluate the accuracy, reliability, and usability of the proposed system using command-recognition accuracy testing, Word Error Rate (WER) analysis, command-execution validation, and functional testing.	Experimental testing, WER calculation, command-execution validation, and usability evaluation.	Accuracy analysis, WER analysis, command-execution analysis, and usability assessment.
4	How does environmental noise influence speech-recognition accuracy and command-execution performance in the proposed system?	To analyse the effects of environmental noise on speech-recognition accuracy and command-execution performance through noise-robustness testing, ambient-noise calibration evaluation, and recognition-stability analysis under different operating conditions.	Controlled noise testing under different environments and noise levels.	Noise robustness analysis and comparative performance evaluation.

Table 3.1 Research Question–Research Objective–Methodology Mapping

The overall prototype development process adopted in this study is illustrated in Figure 3.1.

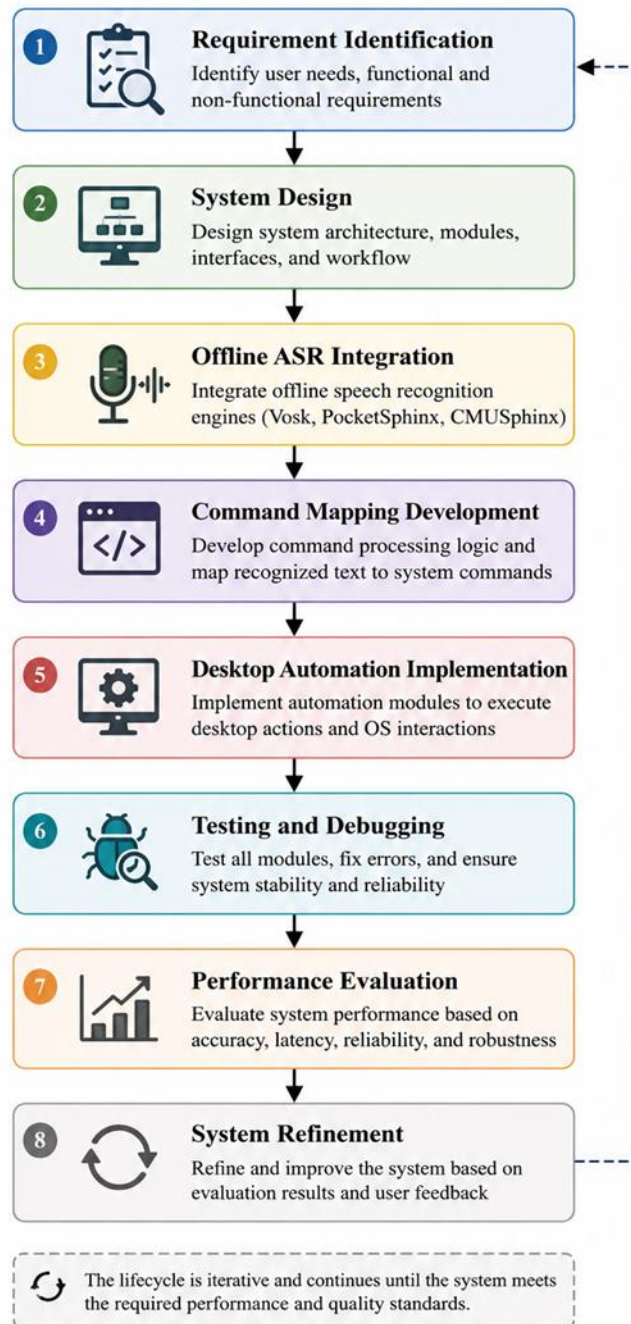


Figure 3.1 Prototype Development Lifecycle

As illustrated in Figure 3.1, the development process consisted of six major stages: requirement gathering, system design, implementation, integration, testing, and refinement.

The first stage involved requirement gathering and analysis. During this stage, the functional and non-functional requirements of the proposed system were identified based on the research

objectives and literature review findings. The analysis focused on determining the speech-recognition capabilities, automation functionalities, accessibility requirements, and operational constraints required for the successful implementation of the system.

Following requirement analysis, the system design stage was conducted. During this phase, the overall architecture of the proposed framework was developed. The architecture defined the interaction between the microphone input layer, audio streaming subsystem, speech-recognition engines, command-processing module, desktop automation layer, and operating-system environment. Workflow diagrams, use case diagrams, and module structures were also designed during this stage.

The implementation stage involved the development of the system using the Python programming language. Three offline speech-recognition frameworks, namely Vosk, PocketSphinx, and CMUSphinx, were integrated into the development environment. Command-processing functions were implemented to validate recognised speech commands and map them to predefined desktop automation operations. Desktop automation functionalities were developed using Python libraries such as PyAutoGUI, Webbrowser, and Subprocess.

After implementation, the individual system components were integrated into a unified framework. During this stage, speech-recognition modules, command-processing mechanisms, and desktop automation functions were combined to support complete speech-to-action interaction. Integration testing was performed continuously to ensure that all components communicated correctly during runtime operation.

The testing stage was conducted to verify that the implemented modules operated according to the defined requirements. Functional testing was performed throughout development to ensure that voice commands were recognised correctly and that corresponding desktop actions were executed successfully. Testing activities also helped identify implementation issues and improve system stability.

The final stage involved refinement and optimisation. Based on observations made during implementation and testing, improvements were introduced to enhance command reliability, improve speech-recognition consistency, strengthen error handling, and optimise overall

system performance. This iterative refinement process ensured that the final prototype achieved the intended functionality and provided a stable platform for subsequent evaluation.

The prototype-based development methodology therefore provided a structured framework for designing, implementing, integrating, and refining the proposed Offline Voice-Controlled Desktop Automation System. By following a systematic development lifecycle, the study successfully transformed the identified research requirements into a functional software solution capable of supporting offline voice-controlled desktop interaction.

3.3 Requirement Analysis

Requirement analysis is one of the most important phases in software system development because it establishes the foundation upon which the entire system is designed, implemented, and evaluated. The purpose of this phase is to identify the functionalities, operational characteristics, technical resources, and user expectations required for the successful development of the proposed Offline Voice-Controlled Desktop Automation System.

For this research, requirement analysis was conducted before system implementation to ensure that the proposed solution satisfied the research objectives presented in Chapter 1. The analysis focused on determining the voice-recognition capabilities, desktop automation functionalities, accessibility requirements, operational constraints, and technical resources required to support hands-free desktop interaction through offline speech recognition.

The proposed system integrates multiple technologies, including offline Automatic Speech Recognition (ASR), command-processing mechanisms, desktop automation libraries, and operating-system interaction modules. Therefore, a comprehensive requirement analysis was necessary to ensure that all system components could operate together effectively within a fully offline environment.

To provide a structured analysis, the requirements were classified into five categories: functional requirements, non-functional requirements, user requirements, hardware requirements, and software requirements. These categories collectively define what the system

must do, how it should perform, who will use it, and the resources required to support development and deployment.

3.3.1 Functional Requirements

Functional requirements define the specific operations, services, and behaviours that the proposed system must perform to achieve the objectives of this research. These requirements describe how the system responds to user input, processes speech commands, validates recognised text, and executes desktop automation functions. Functional requirements are particularly important because they establish the capabilities that must be implemented during system development and provide a basis for evaluating whether the completed system satisfies its intended purpose.

For the proposed Offline Voice-Controlled Desktop Automation System, functional requirements were identified during the requirement-gathering phase based on the research objectives and the practical functionalities required for hands-free desktop interaction. Since the system was designed to allow users to control desktop operations through voice commands, the identified requirements include speech recognition, command validation, browser automation, application launching, mouse control, scrolling operations, multimedia interaction, volume management, and runtime monitoring.

The identified functional requirements of the proposed system are presented in Table 3.2.

ID	Functional Requirement	Description
FR1	Speech Recognition Processing	The system shall capture microphone input, perform offline speech recognition, and convert spoken commands into machine-readable text.
FR2	Multi-ASR Integration	The system shall support speech recognition using Vosk, PocketSphinx, and CMUSphinx frameworks.
FR3	Command Interpretation and Validation	The system shall validate recognised commands against predefined command sets before execution.
FR4	Desktop Control Operations	The system shall support mouse movement, clicking, scrolling, and other desktop-control functions through voice commands.
FR5	Application and Web Automation	The system shall launch and control desktop applications, web browsers, and websites using voice commands.
FR6	Multimedia and System Control	The system shall support multimedia interaction and system functions such as volume adjustment and platform navigation.
FR7	Real-Time Command Execution and Feedback	The system shall execute recognised commands with minimal delay and provide execution feedback to the user.
FR8	Error Detection and Handling	The system shall detect unrecognised commands and provide appropriate error messages or feedback.

Table 3.2 Functional Requirements of the Proposed System

Table 3.2 summarises the primary operational functions required for the successful implementation of the proposed system. Requirements FR1, FR2, and FR3 focus on offline speech acquisition and speech-to-text conversion. Requirements FR4 and FR5 define desktop-control and application-automation capabilities. The remaining requirements define the desktop automation functionalities implemented within the system, including browser control, application launching, cursor movement, scrolling operations, multimedia interaction, volume management, runtime feedback, and error handling. Collectively, these requirements establish the functional scope of the proposed framework and guide the implementation activities described in subsequent sections of this chapter.

3.3.2 Non-Functional Requirements

Non-functional requirements describe the quality attributes, operational characteristics, and performance expectations of the proposed system. Unlike functional requirements, which define what the system does, non-functional requirements define how effectively and reliably the system performs its functions during execution.

For this project, several non-functional characteristics were prioritised throughout development. Since the proposed framework was designed as a real-time offline speech-

controlled automation system, emphasis was placed on responsiveness, privacy preservation, reliability, maintainability, scalability, accessibility, and usability. These characteristics influence user experience and determine the practical effectiveness of the system during deployment.

A key design objective of the proposed framework was to eliminate dependence on cloud-based speech-processing services. Consequently, privacy and availability became important non-functional requirements. By processing speech locally on the user's device, the system avoids transmitting audio data to external servers, thereby improving data privacy and reducing internet dependency.

The identified non-functional requirements are presented in Table 3.3.

Requirement Type	Relevance to the Proposed System
Performance	The system must process and execute recognised voice commands with low response latency to support real-time desktop automation.
Privacy	The system must process speech locally using offline ASR engines to prevent transmission of voice data to external cloud services.
Reliability	The system must consistently recognise predefined voice commands and execute the intended desktop operations correctly.
Usability	The system must provide intuitive voice-based interaction that allows users to perform desktop tasks with minimal effort.
Accessibility	The system must support hands-free computer interaction for users who experience difficulties using conventional input devices.
Maintainability	The system must utilise modular Python-based components to simplify debugging, modification, and future system enhancements.
Scalability	The system must support the addition of new commands, automation functions, and speech-recognition modules in future versions.
Portability	The system must operate on standard Windows desktop environments without requiring specialised hardware.
Security	The system must minimise risks associated with external speech-data transmission through local offline processing.
Robustness	The system must maintain stable operation under moderate environmental noise conditions encountered during normal usage.
Availability	The system must remain fully operational without requiring internet connectivity.
Flexibility	The system must support integration and comparison of multiple offline speech-recognition frameworks including Vosk, PocketSphinx, and CMUSphinx.

Table 3.3 Non-Functional Requirements of the Proposed System

Table 3.3 summarises the quality attributes that guided system design and implementation. Performance requirements ensure that recognised commands can be processed with minimal

delay, while reliability requirements ensure consistent command recognition and execution. Privacy and security requirements support offline speech processing and reduce exposure to external networks. Maintainability and scalability requirements influenced the adoption of a modular architecture, enabling future enhancements and easier debugging. Together, these requirements contribute to the development of a robust, user-friendly, and sustainable desktop automation framework.

3.3.3 User Requirements

User requirements describe the needs, expectations, and interaction preferences of individuals who will operate the proposed system. These requirements were identified to ensure that the developed solution provides a practical and accessible user experience while supporting the objectives of voice-controlled desktop interaction.

The proposed system was designed primarily for users who require an alternative to traditional keyboard-and-mouse interaction. Potential users include individuals seeking hands-free computer control, users performing repetitive desktop tasks, and users who may experience mobility-related limitations that make conventional input devices less convenient.

The system enables users to perform common desktop operations such as browser navigation, application launching, multimedia interaction, scrolling, cursor movement, and volume control using predefined voice commands. To ensure practical usability, the command structure was intentionally designed to remain simple, consistent, and easy to remember.

The identified user requirements are presented in Table 3.4.

User Requirement ID	User Requirement	Description
UR1	Hands-Free Interaction	Users should be able to interact with the desktop system using voice commands.
UR2	Simple Command Structure	The system should support simple and easily memorable voice commands.
UR3	Fast Response	Users should receive rapid command execution feedback.
UR4	Offline Accessibility	Users should be able to use the system without internet access.
UR5	Easy System Interaction	The interface should minimize user complexity during operation.
UR6	Reliable Recognition	The system should accurately recognize commonly used commands.
UR7	Accessibility Support	The system should support users with limited physical interaction capability.

Table 3.4 User Requirements of the Proposed System

Table 3.4 summarises the key expectations of potential users. These requirements emphasise ease of use, accessibility, reliable speech recognition, fast response times, and offline availability. The identified requirements influenced several implementation decisions, including the use of predefined command structures, real-time command processing, and lightweight desktop automation mechanisms. As a result, the proposed system provides a user-friendly interaction environment capable of supporting practical voice-controlled desktop operation.

3.3.4 Hardware Requirements

Hardware requirements define the physical computing resources necessary to develop, test, and operate the proposed system. Since the framework performs continuous microphone listening, offline speech recognition, command validation, and desktop automation in real time, appropriate hardware resources are required to ensure stable operation and acceptable performance.

The hardware requirements were determined based on the operational needs of the speech-recognition frameworks, audio-processing libraries, and desktop automation modules integrated within the system.

The identified hardware requirements are presented in Table 3.5.

Hardware Component	Specification
Processor	Intel/AMD Multi-Core Processor
RAM	Minimum 8 GB
Storage	Minimum 5 GB Free Space
Microphone	Integrated or External USB Microphone
Operating System	Windows 10/11
Audio Device	Real-Time Audio Input Support

Table 3.5 Hardware Requirements

Table 3.5 summarises the hardware resources used during development and testing. The microphone serves as the primary input device for speech acquisition, while the processor and memory support speech processing, command validation, and automation execution. The computer system provides the operating environment required for integrating the offline speech-recognition engines and desktop automation components. Collectively, these resources provide sufficient computational capability to support the proposed framework throughout development and deployment.

3.3.5 Software Requirements

Software requirements describe the development tools, operating-system environment, programming language, frameworks, and libraries required to implement the proposed system. These software components provide the functionality necessary for speech recognition, command processing, automation execution, and system monitoring.

The proposed Offline Voice-Controlled Desktop Automation System was developed using the Python programming language because of its extensive support for speech-processing libraries, automation frameworks, and rapid application development. Multiple third-party libraries were integrated to support offline speech recognition, audio streaming, operating-system interaction, and desktop automation.

The identified software requirements are presented in Table 3.6.

Software/Library	Purpose
Python 3.11	Core programming language
PyCharm IDE	Development environment
Vosk API	Offline speech recognition engine
PocketSphinx	Lightweight offline ASR framework
CMUSphinx	Offline speech recognition framework
SpeechRecognition	Speech-processing integration
PyAutoGUI	Desktop automation and GUI interaction
SoundDevice	Real-time microphone streaming
Webbrowsers Module	Browser automation
Subprocess Module	Operating system process execution
JSON Module	Speech-text extraction and parsing
Queue Module	Audio buffering and processing
NumPy	Word Error Rate computation and matrix operations

Table 3.6 Software Requirements

Table 3.6 summarises the software tools and libraries utilised throughout the development process. Python serves as the core development platform, while Vosk, PocketSphinx, and CMUSphinx provide offline speech-recognition capabilities. Additional libraries such as PyAutoGUI, SoundDevice, Webbrowsers, and Subprocess support desktop automation and system interaction. Together, these software components form the technological foundation of the proposed framework and enable the implementation of the functionalities described throughout this chapter.

3.3.6 Selection of Vosk Speech Recognition Model

Before implementing the speech-recognition subsystem, several English-language models available within the Vosk framework were reviewed and compared. The available models differ in terms of recognition accuracy, memory consumption, processing requirements, and storage size. Therefore, selecting an appropriate model was important to ensure efficient offline speech recognition while maintaining acceptable command-recognition performance for desktop automation tasks.

The comparison of available Vosk English-language models considered during this study is presented in Table 3.7.

Model Name	Description	WER Performance	Size
vosk-model-small-en-us-0.15	Lightweight English-language model suitable for desktop and embedded applications	9.85 (LibriSpeech), 10.38 (TED-LIUM)	40 MB
vosk-model-en-us-0.22	General-purpose US English model with improved recognition accuracy	5.69 (LibriSpeech), 6.05 (TED-LIUM), 29.78 (Call Center)	1.8 GB
vosk-model-en-us-0.22-lgraph	Dynamic graph English model balancing recognition accuracy and memory usage	7.82 (LibriSpeech), 8.20 (TED-LIUM)	128 MB
vosk-model-en-us-0.42-gigaspeech	Advanced English model trained using the GigaSpeech dataset	5.64 (LibriSpeech), 6.24 (TED-LIUM), 30.17 (Call Center)	2.3 GB

Table 3.7 Comparison of Available Vosk English-Language Models

As shown in Table 3.7, larger Vosk models generally provide lower Word Error Rates (WER) and higher recognition accuracy. However, these models require significantly greater storage capacity and computational resources. Since the objective of this research was to develop a lightweight offline voice-controlled desktop automation system capable of operating efficiently on standard personal computers, the `vosk-model-small-en-us-0.15` model was selected for implementation.

Although the larger models provide higher recognition accuracy, they require substantially greater storage capacity and computational resources. The selected lightweight model offers lower memory consumption, faster loading time, and efficient real-time execution while maintaining satisfactory command-recognition performance for the predefined command vocabulary used in this study. Furthermore, the predefined command vocabulary used in this study did not require the additional computational complexity associated with larger speech-recognition models.

Therefore, the `vosk-model-small-en-us-0.15` model was considered the most suitable choice for supporting real-time offline speech recognition within the proposed desktop automation framework.

3.3.7 Use Case Analysis

Use case analysis was conducted to identify the interactions between the user and the proposed Offline Voice-Controlled Desktop Automation System. The use case diagram provides a visual representation of the functionalities supported by the system and illustrates how users interact with the various desktop automation features through voice commands.

In the proposed framework, the primary actor is the user. The user communicates with the system through spoken commands, which are captured by the microphone and processed by the offline speech-recognition subsystem. Following recognition and validation, the system executes the requested desktop operation.

The use case diagram of the proposed system is illustrated in Figure 3.2.

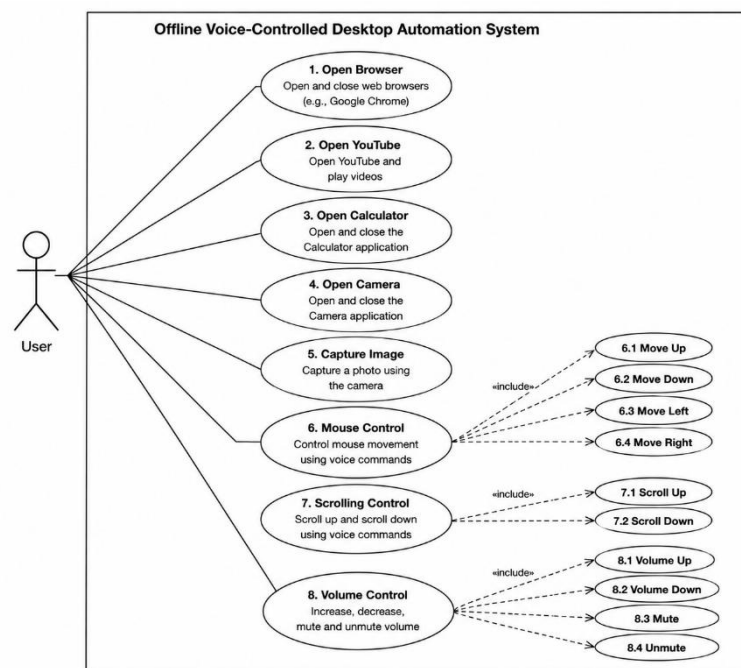


Figure 3.2 Use Case Diagram of the Proposed Offline Voice-Controlled Desktop Automation System

Figure 3.2 illustrates the relationship between the user and the major functionalities supported by the proposed framework. The diagram shows that the user can initiate various desktop operations, including mouse control, scrolling, browser automation, multimedia navigation, application launching, volume management, and runtime monitoring. Each use case represents a specific service provided by the system in response to recognised voice commands.

The use case diagram provides a high-level overview of system functionality and serves as a foundation for the architecture, workflow, and module designs presented in the subsequent

sections of this chapter. Through this analysis, the interactions between the user and the proposed system can be clearly visualised before implementation details are examined.

3.4 System Architecture

System architecture defines the structural organisation of the proposed system and illustrates how individual components interact to achieve the desired functionality. It provides a high-level representation of the major subsystems involved in speech acquisition, speech recognition, command processing, desktop automation, and runtime interaction.

For the proposed Offline Voice-Controlled Desktop Automation System, the architecture was designed to support fully offline speech recognition and real-time desktop automation without relying on cloud-based services. The architecture integrates three offline Automatic Speech Recognition (ASR) frameworks, namely Vosk, PocketSphinx, and CMUSphinx, together with Python-based automation libraries to enable voice-controlled desktop interaction.

The architecture was intentionally developed using a modular approach. A modular architecture allows individual components to operate independently while remaining connected within a unified workflow. This design improves maintainability, scalability, debugging efficiency, and future system enhancement because modifications can be made to one component without affecting the entire system.

The proposed architecture consists of several interconnected layers including audio acquisition, speech recognition, command processing, automation execution, and desktop interaction. Each layer performs a specific function and contributes to the overall speech-to-action workflow implemented within the system.

The overall architecture of the proposed system is presented in Figure 3.3.

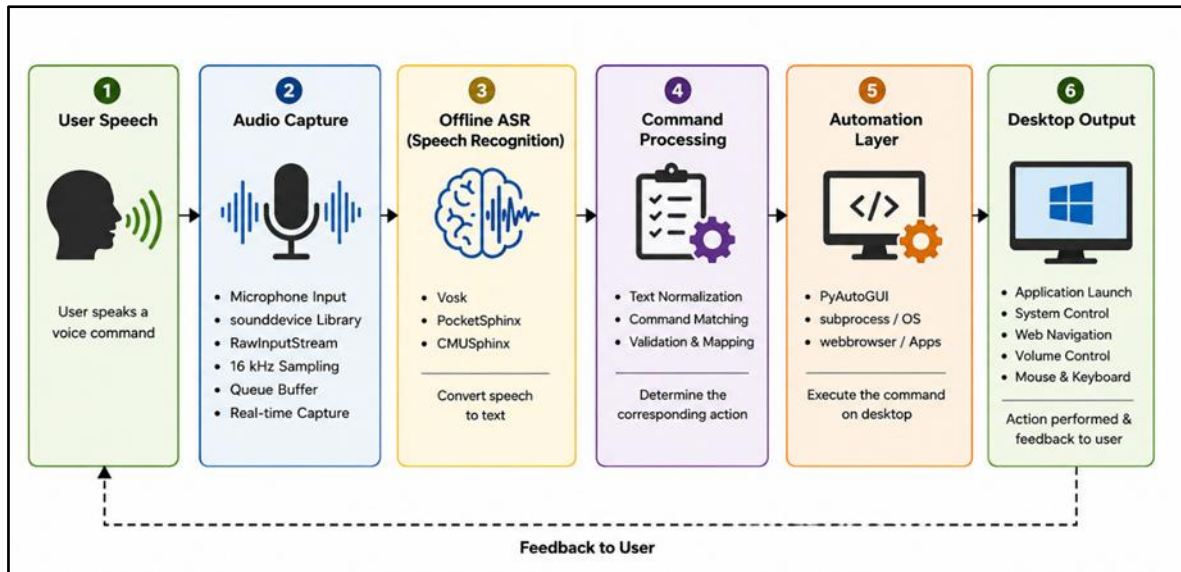


Figure 3.3 Overall System Architecture of the Proposed Offline Voice-Controlled Desktop Automation System

Figure 3.3 illustrates the complete architecture of the proposed framework. The workflow begins with speech input provided by the user through a microphone device. The captured audio signal is forwarded to the audio acquisition layer where speech data is continuously collected and buffered for processing.

The buffered audio is then transferred to one of the integrated offline speech-recognition engines. Depending on the implementation being executed, the speech signal is processed by Vosk, PocketSphinx, or CMUSphinx. Each recognition engine converts the incoming speech signal into textual output representing the recognised command.

After speech recognition is completed, the recognised text is forwarded to the command-processing layer. This layer performs text normalisation, keyword matching, command validation, and automation-function selection. Only commands that successfully match predefined command patterns are permitted to proceed to the execution stage.

Validated commands are subsequently passed to the desktop automation layer. This layer utilises Python libraries such as PyAutoGUI, Webbrowser, and Subprocess to execute the requested operation. Depending on the recognised command, the system may perform actions

such as cursor movement, mouse clicking, scrolling, browser launching, application execution, multimedia navigation, or volume control.

Finally, the execution result is reflected within the desktop environment through visible operating-system actions. The completed workflow therefore transforms human speech into executable desktop operations while maintaining fully offline functionality.

The modular architecture implemented in this study provides a clear separation between speech processing, command interpretation, and automation execution. This separation improves system reliability and allows individual components to be tested, modified, and extended independently during future development.

3.4.1 Audio Acquisition and Streaming Architecture

The Audio Acquisition and Streaming Architecture is responsible for capturing real-time speech input from the user and transferring audio data to the offline speech-recognition subsystem. This layer acts as the entry point of the proposed framework because all voice commands must first be acquired before speech recognition and automation processing can occur.

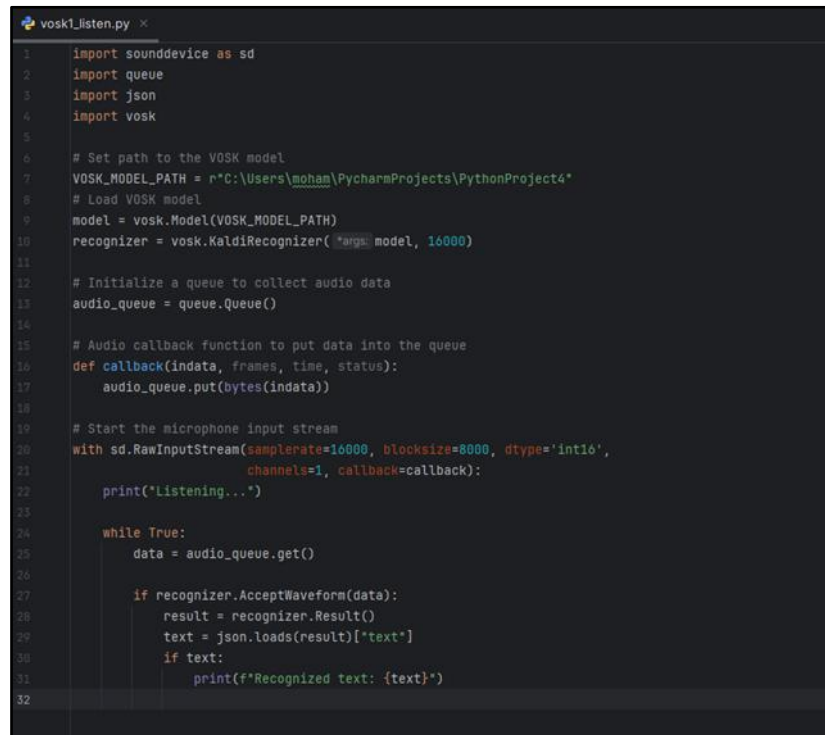
For the proposed system, audio acquisition was implemented using the SoundDevice library. SoundDevice was selected because it provides efficient access to microphone devices and supports continuous real-time audio streaming. Continuous audio streaming is particularly important for voice-controlled systems because users may issue commands at any time during system operation.

The architecture begins with microphone input. When the system is activated, the microphone continuously captures spoken audio and converts the incoming sound waves into digital audio signals suitable for processing. These audio signals are then forwarded into the streaming subsystem.

To support continuous listening, the Vosk implementation utilises the `RawInputStream()` function provided by the SoundDevice library. This function establishes a real-time audio

stream between the microphone and the speech-processing subsystem. The audio stream was configured using a sampling rate of 16,000 Hz and mono-channel audio input to ensure compatibility with the offline speech-recognition engines used throughout the project.

The complete audio acquisition and streaming mechanism implemented in the proposed system is illustrated in Figure 3.4.



```
1 import sounddevice as sd
2 import queue
3 import json
4 import vosk
5
6 # Set path to the VOSK model
7 VOSK_MODEL_PATH = r"C:\Users\mohan\PycharmProjects\PythonProject4"
8 # Load VOSK model
9 model = vosk.Model(VOSK_MODEL_PATH)
10 recognizer = vosk.KaldiRecognizer(model, 16000)
11
12 # Initialize a queue to collect audio data
13 audio_queue = queue.Queue()
14
15 # Audio callback function to put data into the queue
16 def callback(indata, frames, time, status):
17     audio_queue.put(bytes(indata))
18
19 # Start the microphone input stream
20 with sd.RawInputStream(samplerate=16000, blocksize=8000, dtype='int16',
21                       channels=1, callback=callback):
22     print("Listening...")
23
24     while True:
25         data = audio_queue.get()
26
27         if recognizer.AcceptWaveform(data):
28             result = recognizer.Result()
29             text = json.loads(result)["text"]
30             if text:
31                 print(f"Recognized text: {text}")
32
```

Figure 3.4 Real-Time Audio Streaming and Queue Callback Mechanism

Figure 3.4 illustrates the implementation of the real-time audio streaming subsystem used within the Vosk recognition framework. The screenshot shows the initialisation of the audio stream using the `RawInputStream()` function together with the callback mechanism responsible for handling incoming audio data.

When speech is captured by the microphone, the callback function is automatically triggered. The callback function receives incoming audio frames and converts them into byte-format data suitable for speech-processing operations. Instead of sending audio directly to the recognition engine, the system temporarily stores incoming audio frames within a queue structure.

The queue buffering mechanism is implemented using Python's Queue module. The operation `queue.put(bytes(indata))` continuously inserts captured audio data into the queue buffer. This architecture enables asynchronous communication between the audio acquisition layer and the speech-recognition layer.

The use of queue buffering provides several advantages. First, it prevents audio loss during continuous streaming by ensuring that incoming audio frames are stored temporarily before processing. Second, it allows audio acquisition and speech recognition to operate simultaneously without blocking each other. Third, it improves system responsiveness because the recognition engine can process audio chunks incrementally while new audio data continues to arrive from the microphone.

The streaming architecture therefore enables uninterrupted speech acquisition and forms the foundation for real-time voice interaction within the proposed Offline Voice-Controlled Desktop Automation System. By combining microphone input, SoundDevice streaming, callback-based processing, and queue buffering, the system is capable of continuously capturing speech and delivering audio data efficiently to the offline recognition subsystem for further processing.

3.4.2 Offline Speech Recognition Architecture

The Offline Speech Recognition Architecture forms the core intelligence layer of the proposed Offline Voice-Controlled Desktop Automation System. The primary responsibility of this layer is to convert spoken language into machine-readable text that can subsequently be interpreted and processed by the command-processing subsystem.

Unlike cloud-based speech-recognition systems that rely on remote servers and internet connectivity, the proposed framework performs all speech-recognition activities locally on the user's device. This design decision was made to improve privacy, reduce network dependency, minimise communication latency, and ensure continuous operation even when internet access is unavailable.

To investigate the effectiveness of different offline speech-recognition technologies, three independent Automatic Speech Recognition (ASR) frameworks were integrated into the proposed system, namely Vosk, PocketSphinx, and CMUSphinx. Each framework utilises a different speech-recognition approach and provides unique advantages in terms of recognition performance, computational complexity, and deployment requirements.

The recognised text generated by these frameworks is subsequently forwarded to the command-processing subsystem, where command validation and desktop automation execution occur. The speech-recognition architecture therefore acts as the bridge between human speech input and machine-executable desktop actions.

3.4.3 Speech Recognition Algorithms Used

The proposed system integrates three offline speech-recognition frameworks and a rule-based command-processing mechanism. Each component utilises a different algorithmic approach to transform spoken commands into executable desktop actions. Understanding these algorithms is important because they determine how speech signals are processed, interpreted, and converted into textual commands before automation execution occurs.

3.4.3.1 Vosk Speech Recognition Algorithm

Vosk was implemented as the primary speech-recognition framework within the proposed system because it supports real-time streaming recognition, low-latency processing, and fully offline deployment. The Vosk framework is built upon the Kaldi speech-recognition toolkit and utilises Deep Neural Network (DNN)-based acoustic modelling to recognise speech patterns.

The Kaldi-based architecture analyses incoming speech signals by extracting acoustic features and comparing them against trained neural-network models. These models learn complex relationships between speech sounds and language patterns, enabling the framework to recognise spoken words with higher accuracy than many traditional speech-recognition approaches.

A major advantage of DNN-based recognition is its ability to model variations in pronunciation, speaking style, and acoustic conditions more effectively than conventional statistical techniques. This contributes to improved recognition accuracy and reduced Word Error Rate (WER) during speech-processing tasks.

Within the proposed system, Vosk receives streamed microphone audio, processes the incoming speech through Kaldi-based acoustic models, and generates recognised text that is subsequently forwarded to the command-processing subsystem for validation and automation execution.

3.4.3.2 PocketSphinx Speech Recognition Algorithm

PocketSphinx was implemented as a lightweight offline speech-recognition framework designed for low-resource computing environments. Unlike Vosk, which utilises deep neural networks, PocketSphinx is based primarily on the Hidden Markov Model (HMM) speech-recognition approach.

The Hidden Markov Model is a statistical model that represents speech as a sequence of observable acoustic signals generated by hidden speech states. During recognition, the system attempts to determine the most probable sequence of words corresponding to the observed speech input.

The HMM-based approach has historically been one of the most widely used techniques in automatic speech recognition because of its relatively low computational requirements and straightforward implementation. Although modern deep-learning approaches generally achieve higher recognition accuracy, HMM-based systems remain useful for constrained command-recognition environments where vocabulary size is limited.

Within the proposed framework, PocketSphinx continuously processes microphone speech input and generates textual output that can be interpreted by the desktop automation subsystem.

3.4.3.3 CMUSphinx Speech Recognition Algorithm

CMUSphinx was implemented through the SpeechRecognition Python framework using the `recognize_sphinx()` function. Similar to PocketSphinx, CMUSphinx is primarily based on the Hidden Markov Model (HMM) recognition methodology.

The recognition process begins by capturing audio from the microphone and extracting acoustic features from the speech signal. The HMM-based decoder then evaluates possible word sequences and selects the most probable interpretation of the spoken input. The recognised command is subsequently converted into text and passed to the command-processing subsystem.

An additional feature implemented within the CMUSphinx subsystem is ambient-noise calibration using the `adjust_for_ambient_noise()` function. This process improves microphone sensitivity adjustment and helps reduce the impact of background noise on recognition performance during runtime operation.

The CMUSphinx implementation therefore provides an alternative offline recognition framework that can be compared against both Vosk and PocketSphinx during system evaluation.

3.4.3.4 Rule-Based Command Processing Algorithm

After speech recognition is completed, the recognised text enters the command-processing subsystem. Unlike conversational Artificial Intelligence systems that rely on Natural Language Processing (NLP) and machine-learning-based intent detection, the proposed framework utilises a rule-based keyword-matching algorithm.

The rule-based approach was selected because the proposed system operates within a predefined command environment where supported commands are known in advance. The algorithm compares recognised text against a list of predefined command keywords using sequential conditional structures implemented through Python `if-elif` statements.

For example, when the recognised command contains the phrase "open browser", the system maps the command to the browser-launching function. Similarly, commands such as "move up", "left click", "scroll down", and "mute" are mapped to their corresponding automation operations.

The rule-based command-processing approach offers several advantages for desktop automation systems. It improves execution predictability, reduces command ambiguity, simplifies implementation, and prevents unsupported speech inputs from triggering unintended desktop actions.

The combination of Kaldi-based Deep Neural Network recognition, Hidden Markov Model recognition, and rule-based command processing therefore provides the algorithmic foundation of the proposed Offline Voice-Controlled Desktop Automation System. Together, these algorithms enable the system to transform human speech into validated desktop automation actions while maintaining fully offline operation.

3.4.4 Vosk Recognition Pipeline

Vosk was implemented as the primary offline speech-recognition framework within the proposed system because it supports real-time streaming recognition, low-latency speech processing, and fully offline deployment. The framework was integrated using the Vosk Python API together with the Kaldi speech-recognition toolkit. The objective of the Vosk subsystem was to continuously receive audio input from the microphone, convert speech into text, and forward recognised commands to the command-processing layer for automation execution.

Before recognition could begin, the offline acoustic model was loaded into memory using the `Model()` function. The recogniser was then initialised using the `KaldiRecognizer()` class configured with a sampling frequency of 16,000 Hz. This configuration ensured compatibility with the audio acquisition subsystem and allowed incoming speech to be processed in real time.

The implementation of the Vosk recognition pipeline is illustrated in Figure 3.5.

```

1 import sounddevice as sd
2 import queue
3 import json
4 import vosk
5
6 # Set path to the VOSK model
7 VOSK_MODEL_PATH = r"C:\Users\mohan\PycharmProjects\PythonProject4"
8 # Load VOSK model
9 model = vosk.Model(VOSK_MODEL_PATH)
10 recognizer = vosk.KaldiRecognizer(model, 16000)
11
12 # Initialize a queue to collect audio data
13 audio_queue = queue.Queue()
14

```

Figure 3.5 Vosk Model Initialization and Recognition Pipeline

Figure 3.5 illustrates the implementation of the Vosk recognition subsystem used within the proposed framework. The screenshot shows the initialisation of the offline acoustic model through the `Model()` function together with the creation of the `KaldiRecognizer()` object responsible for speech decoding.

The figure also demonstrates how incoming audio frames are continuously retrieved from the queue structure and forwarded to the recogniser using the `AcceptWaveform()` function. This function evaluates incoming audio segments and determines whether a complete speech segment has been detected. Once a valid speech segment is identified, the recogniser generates recognition output in JSON format.

The recognised text is subsequently extracted through JSON parsing operations using the `json.loads()` function. The extracted command is then forwarded into the `process_command()` function where command validation and automation execution occur.

The Vosk implementation therefore provides a complete speech-recognition pipeline capable of transforming continuous microphone audio into executable desktop commands. By combining real-time audio streaming, Kaldi-based acoustic modelling, and incremental speech processing, the framework supports responsive offline voice interaction within the proposed system.

3.4.5 PocketSphinx Recognition Pipeline

PocketSphinx was implemented as a lightweight offline speech-recognition framework capable of operating with relatively low computational requirements. The framework was selected to

provide an alternative recognition approach and to enable comparative analysis between different offline speech-recognition technologies.

Unlike the Vosk implementation, which processes audio through a queue-based streaming architecture, PocketSphinx performs speech recognition using the LiveSpeech() function. This function continuously listens for speech input from the microphone and generates recognised text directly during runtime operation.

The implementation of the PocketSphinx recognition pipeline is illustrated in Figure 3.6.

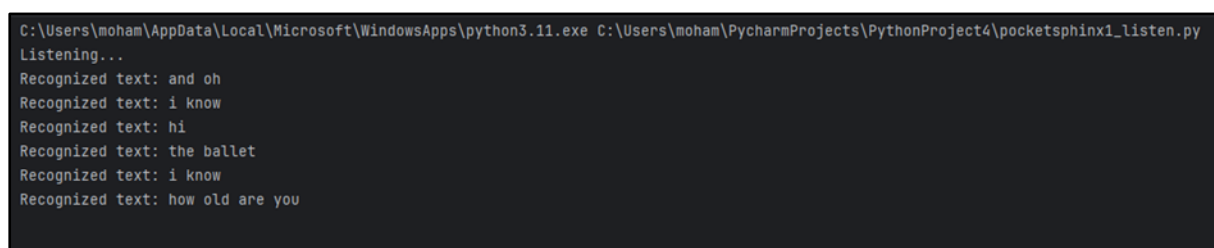
A screenshot of a terminal window with a dark background and light-colored text. The text shows the execution of a Python script. The first line is the command prompt: `C:\Users\moham\AppData\Local\Microsoft\WindowsApps\python3.11.exe C:\Users\moham\PycharmProjects\PythonProject4\pocketsphinx1_listen.py`. The output starts with `Listening...`, followed by several lines of recognized text: `Recognized text: and oh`, `Recognized text: i know`, `Recognized text: hi`, `Recognized text: the ballet`, `Recognized text: i know`, and `Recognized text: how old are you`.

Figure 3.6 PocketSphinx Continuous Listening Architecture

Figure 3.6 illustrates the continuous listening architecture implemented using the PocketSphinx framework. The screenshot shows the initialisation of the LiveSpeech() object together with the runtime loop responsible for continuously monitoring microphone input.

As speech is captured by the microphone, PocketSphinx processes the incoming audio using its Hidden Markov Model (HMM)-based recognition engine. The recognised speech is automatically converted into textual output and returned by the LiveSpeech() iterator. Each recognised phrase is then displayed through the runtime console and forwarded to the command-processing subsystem.

The figure demonstrates how the framework continuously listens for voice input without requiring explicit recording termination before recognition can occur. This continuous listening mechanism improves user interaction by enabling uninterrupted speech-command monitoring throughout runtime operation.

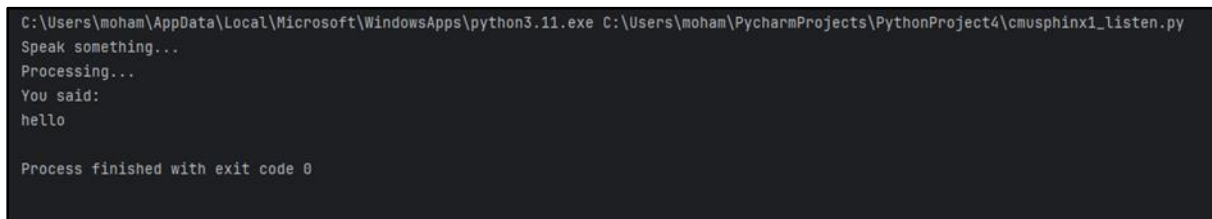
The PocketSphinx implementation therefore provides a lightweight speech-recognition pipeline capable of supporting offline voice-command interaction while maintaining relatively low computational overhead.

3.4.6 CMUSphinx Recognition Pipeline

CMUSphinx was implemented through the SpeechRecognition Python framework using the `recognize_sphinx()` function. The framework was selected to provide an additional offline speech-recognition alternative and to facilitate comparative evaluation against the Vosk and PocketSphinx implementations.

Unlike Vosk and PocketSphinx, the CMUSphinx subsystem relies on explicit microphone listening operations combined with ambient-noise calibration. Before speech acquisition begins, the recogniser adjusts microphone sensitivity to account for environmental noise conditions. This calibration process improves recognition consistency and reduces the influence of background noise during speech processing.

The implementation of the CMUSphinx recognition pipeline is illustrated in Figure 3.7.



```
C:\Users\moham\AppData\Local\Microsoft\WindowsApps\python3.11.exe C:\Users\moham\PycharmProjects\PythonProject4\cmusphinx1_listen.py
Speak something...
Processing...
You said:
hello

Process finished with exit code 0
```

Figure 3.7 CMUSphinx Offline Recognition Workflow

Figure 3.7 illustrates the complete offline recognition workflow implemented using the CMUSphinx framework. The screenshot shows the initialisation of the `Recognizer()` and `Microphone()` objects together with the ambient-noise calibration procedure performed through the `adjust_for_ambient_noise()` function.

After calibration is completed, the recogniser continuously listens for speech input using the `listen()` function. The captured audio data is subsequently processed by the `recognize_sphinx()`

function, which performs offline speech decoding and generates textual output representing the recognised command.

The figure also demonstrates the implementation of exception-handling mechanisms used to manage recognition failures and microphone-processing errors. Try-except structures were incorporated into the implementation to prevent unexpected system termination during runtime operation and to improve overall system stability.

Following recognition, the recognised text is forwarded to the command-processing subsystem where keyword matching, command validation, and desktop automation execution occur.

The CMUSphinx implementation therefore provides a complete offline recognition workflow that combines microphone calibration, speech acquisition, offline decoding, and error handling to support reliable voice-command interaction within the proposed desktop automation environment.

3.4.7 Command Processing and Validation Architecture

The Command Processing and Validation Architecture serves as the decision-making component of the proposed Offline Voice-Controlled Desktop Automation System. While the speech-recognition subsystem is responsible for converting spoken language into text, the command-processing subsystem determines whether the recognised text represents a valid desktop command and identifies the appropriate automation action that should be executed.

The primary objective of this architecture is to ensure that only supported commands are allowed to trigger desktop automation operations. This validation mechanism is important because incorrect recognition output may otherwise result in unintended actions being executed within the operating system. By validating commands before execution, the proposed system improves operational reliability, execution consistency, and user safety.

The command-processing architecture was implemented using a dedicated function named `process_command()`. This function receives recognised text generated by the speech-

recognition subsystem and performs a sequence of processing operations including text normalisation, keyword matching, command validation, and automation-function selection.

The complete command-processing workflow implemented within the proposed system is illustrated in Figure 3.8.

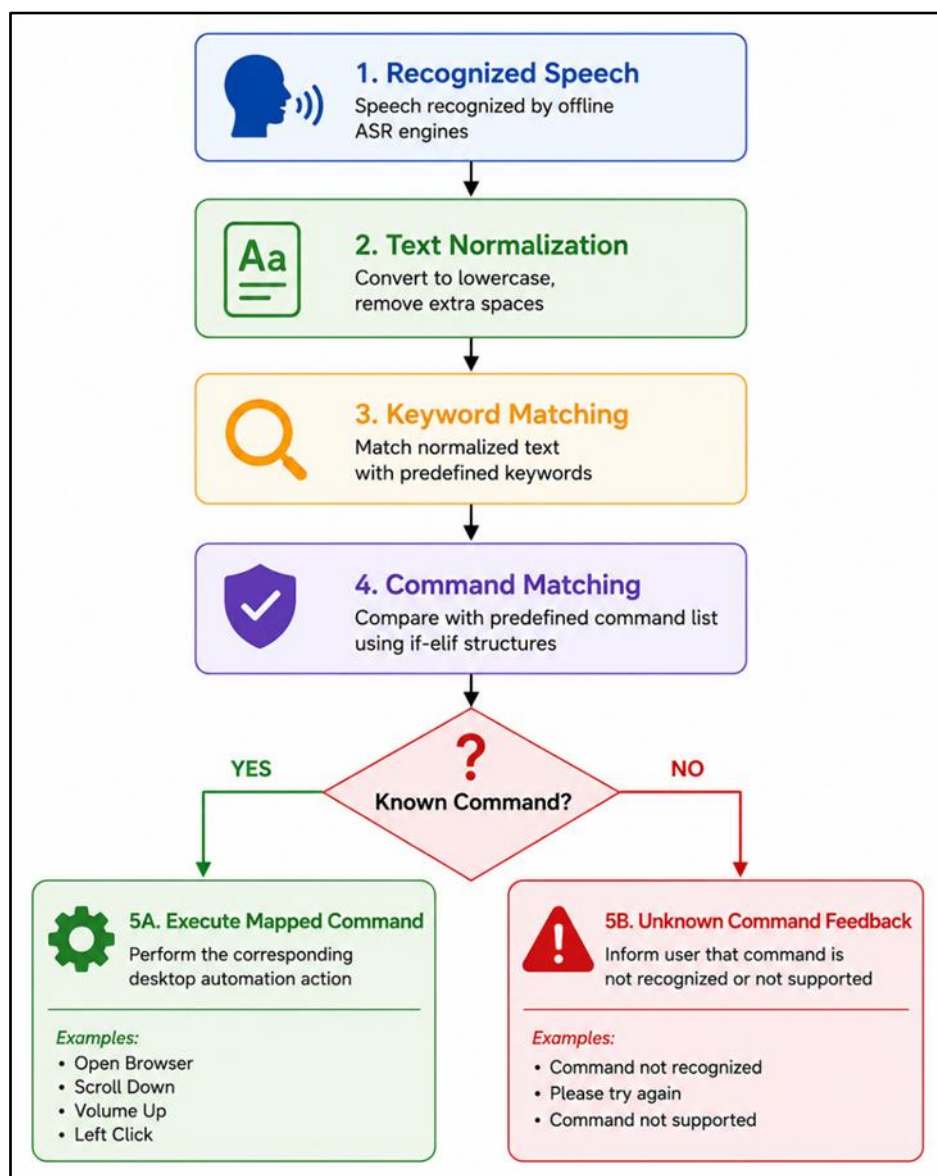


Figure 3.8 Command Processing and Validation Workflow

Figure 3.8 illustrates the complete workflow used to transform recognised speech into executable desktop actions. The workflow begins when recognised text generated by the speech-recognition engine is forwarded to the `process_command()` function for processing.

The first stage of the workflow is text normalisation. During this stage, the recognised command is converted into lowercase format using the `lower()` function. This operation ensures that variations in capitalisation do not affect command validation. For example, commands such as "Open Browser", "OPEN BROWSER", and "open browser" are treated as identical inputs during subsequent processing stages. Text normalisation therefore improves consistency and reduces command-validation errors.

Following text normalisation, the system performs command matching. During this stage, the recognised text is compared against a predefined set of supported command keywords. The proposed system operates within a restricted command environment where all supported commands are known in advance. Consequently, command matching is performed using a rule-based keyword-validation approach rather than unrestricted natural-language understanding.

The validation process was implemented using sequential if-elif conditional structures within the `process_command()` function. Each conditional block represents a specific desktop automation command supported by the system. When a matching keyword is detected, the corresponding automation function is selected for execution.

For example, recognised phrases such as "left click", "move up", "scroll down", "open browser", "open calculator", and "mute" are mapped directly to their corresponding desktop automation functions. This mapping process establishes a direct relationship between recognised speech and executable operating-system actions.

After command matching is completed, the system evaluates whether a valid command has been identified. If the recognised command successfully matches one of the predefined command patterns, the workflow follows the validation-success path and proceeds to automation execution. The corresponding desktop automation function is then executed using Python libraries such as PyAutoGUI, Webbrowser, or Subprocess depending on the requested operation.

If no matching keyword is detected, the workflow follows the validation-failure path. In this situation, the system generates an "Unknown Command" message and terminates processing for the current speech input. No automation action is executed. This mechanism prevents

unsupported speech inputs from triggering unintended desktop operations and improves system safety during runtime execution.

The command-processing architecture implemented in the proposed system offers several advantages. First, it improves execution predictability because every supported command is mapped explicitly to a predefined automation action. Second, it reduces command ambiguity by limiting recognition outcomes to known command structures. Third, it simplifies implementation and debugging because command-validation behaviour can be traced directly through the conditional decision structure.

The architecture also supports future scalability. Additional voice commands can be incorporated into the system by extending the `process_command()` function and introducing new command-validation conditions. This modular design enables future enhancements without requiring modifications to the speech-recognition subsystem.

The Command Processing and Validation Architecture therefore acts as the central control mechanism of the proposed framework. By combining text normalisation, rule-based keyword matching, conditional validation, and automation-function selection, the architecture ensures that recognised speech is transformed into safe, reliable, and executable desktop automation actions within a fully offline environment.

3.5 Operational Workflow

The operational workflow describes the sequence of activities performed by the proposed Offline Voice-Controlled Desktop Automation System during runtime operation. The workflow illustrates how spoken commands are transformed into executable desktop actions through the integration of speech acquisition, offline speech recognition, command processing, and desktop automation components.

The operational workflow is important because it provides a clear understanding of how individual system modules interact with one another during execution. It also demonstrates the complete speech-to-action process implemented within the proposed framework, beginning with user speech input and ending with desktop automation execution.

To facilitate understanding, the workflow is presented at two levels. First, the overall operational workflow provides a high-level overview of the entire system. Second, the command validation and execution workflow presents a detailed view of how recognised commands are processed before automation actions are performed.

3.5.1 Overall Operational Workflow

The overall operational workflow illustrates the complete sequence of processes performed by the proposed system during runtime operation. The workflow begins with user speech input and continues through audio acquisition, speech recognition, command validation, automation execution, and desktop interaction.

The overall workflow of the proposed Offline Voice-Controlled Desktop Automation System is illustrated in Figure 3.9.

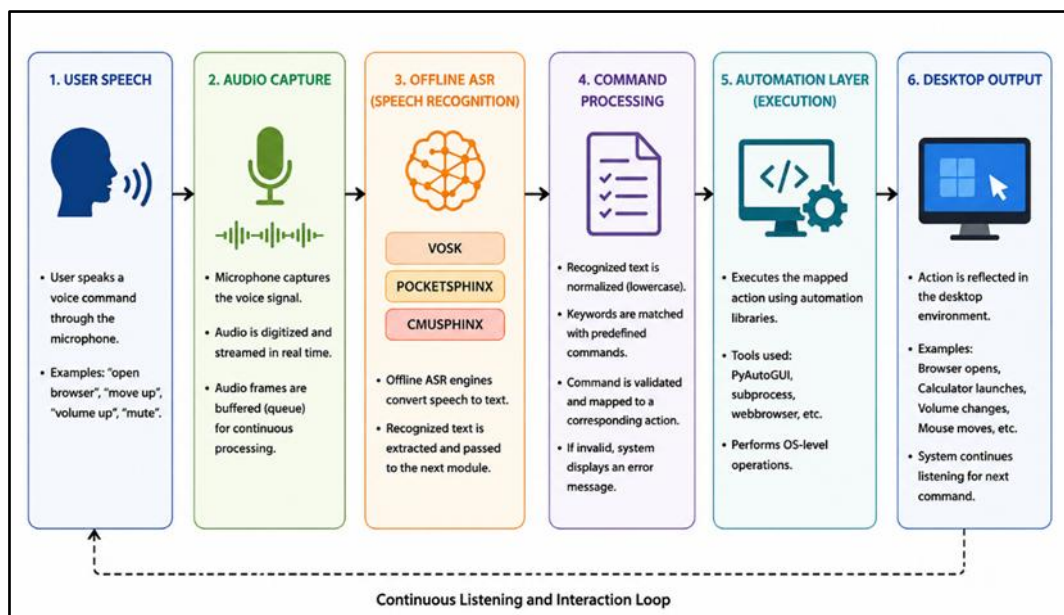


Figure 3.9 Overall Operational Workflow of the Proposed System

Figure 3.9 illustrates the complete operational flow of the proposed framework. The workflow begins when the user provides a spoken command through a microphone connected to the

computer system. The microphone acts as the primary input interface between the user and the automation framework.

Once speech is captured, the audio signal is forwarded to the audio acquisition subsystem where continuous audio streaming is performed. The streaming architecture ensures that incoming speech can be processed in real time without requiring manual recording initiation or termination.

The captured audio is subsequently transferred to the speech-recognition subsystem. Depending on the implementation being executed, the speech signal is processed by one of the integrated offline speech-recognition frameworks, namely Vosk, PocketSphinx, or CMUSphinx. These frameworks analyse the incoming speech signal and convert it into textual output representing the recognised command.

After speech recognition is completed, the recognised text is forwarded to the command-processing subsystem. During this stage, the recognised command undergoes normalisation, keyword matching, and validation to determine whether it corresponds to a supported desktop operation.

If the command is successfully validated, the system selects the corresponding automation function and forwards the instruction to the desktop automation layer. The automation layer then executes the requested operation using Python libraries such as PyAutoGUI, Webbrowser, or Subprocess.

The final stage of the workflow occurs within the desktop environment where the requested action becomes visible to the user. Depending on the command provided, the system may perform actions such as cursor movement, scrolling, browser launching, website navigation, application execution, volume adjustment, or multimedia control.

The overall operational workflow therefore demonstrates how the proposed system integrates speech acquisition, offline speech recognition, command processing, and desktop automation into a unified framework capable of supporting fully offline voice-controlled desktop interaction.

3.5.2 Command Validation and Execution Workflow

While the overall operational workflow describes the complete system operation, the command validation and execution workflow focuses specifically on the decision-making process that occurs after speech recognition has been completed. This workflow explains how recognised text is analysed, validated, and transformed into executable automation actions.

The command validation mechanism is a critical component of the proposed framework because it prevents unsupported speech inputs from triggering unintended desktop operations. By validating commands before execution, the system improves operational reliability, execution consistency, and user safety.

The command validation and execution workflow implemented within the proposed system is illustrated in Figure 3.10.

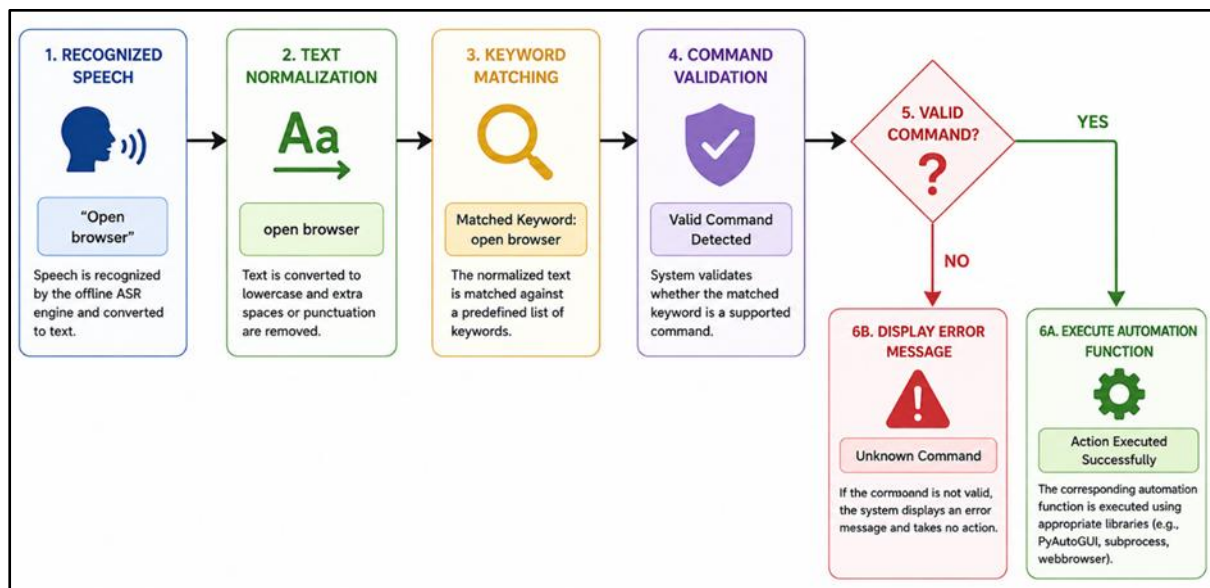


Figure 3.10 Command Validation and Execution Workflow

Figure 3.10 illustrates the detailed workflow used to process recognised speech before automation execution occurs. The workflow begins when recognised text generated by the speech-recognition subsystem is forwarded to the `process_command()` function.

The first stage of processing involves text normalisation. During this stage, the recognised command is converted into lowercase format using the `lower()` function. This operation ensures that variations in capitalisation do not affect command validation and improves consistency during keyword comparison.

Following normalisation, the recognised text enters the keyword-matching stage. During this process, the system compares the recognised command against a predefined set of supported command keywords. The proposed framework utilises a rule-based command-processing approach in which all supported commands are explicitly defined within the system.

The command-validation stage is subsequently performed using sequential `if-elif` conditional structures. Each conditional block represents a valid desktop automation command supported by the framework. When a matching keyword is detected, the system identifies the corresponding automation function associated with that command.

After validation is completed, the system evaluates whether a valid command has been identified. If the recognised command matches one of the predefined command structures, the workflow follows the successful validation path and proceeds to automation execution.

During the execution stage, the selected desktop automation function is invoked using the appropriate automation library. Depending on the recognised command, the system may execute mouse-control operations, scrolling actions, browser navigation, application launching, multimedia interaction, or volume management functions.

If no matching keyword is detected, the workflow follows the error-handling path. In this situation, the system generates an "Unknown Command" message and terminates processing for the current input. No automation action is executed. This mechanism prevents invalid or unsupported speech inputs from producing unintended operating-system actions.

The command validation and execution workflow therefore serves as the central decision-making mechanism within the proposed framework. By combining text normalisation, keyword matching, conditional validation, automation selection, and error handling, the workflow ensures that recognised speech is transformed into reliable and executable desktop automation actions within a fully offline environment.

3.6 System Modules and Functional Components

The proposed Offline Voice-Controlled Desktop Automation System was developed using a modular architecture in which individual functionalities were organised into separate system modules. Each module was responsible for performing a specific desktop automation task after receiving validated commands from the command-processing subsystem. This modular approach improves maintainability, scalability, debugging efficiency, and future system enhancement because individual components can be modified independently without affecting the overall system architecture.

The major functional modules implemented within the proposed framework include mouse control, scrolling control, browser automation, multimedia navigation, application launching, audio control, and runtime monitoring. Each module receives validated commands from the command-processing subsystem and executes the corresponding desktop action through Python-based automation libraries.

3.6.1 Mouse Control Module

The Mouse Control Module was developed to enable users to perform common mouse-related operations through voice commands without requiring physical interaction with a mouse device. This module is one of the core accessibility components of the proposed framework because it allows users to control cursor movement and mouse-click operations using speech-based interaction.

The implementation of this module was integrated with all three speech-recognition frameworks used in the study, namely Vosk, PocketSphinx, and CMUSphinx. After a voice command is recognised and validated, the command-processing subsystem maps the recognised phrase to the appropriate mouse-control function implemented through the PyAutoGUI automation library.

The mouse-control subsystem supports four primary categories of operations: left-click execution, right-click execution, double-click execution, and cursor movement. These

functions were selected because they represent the most commonly used mouse interactions within desktop environments and provide users with the ability to navigate and interact with graphical user interfaces through speech commands.

Within the implementation, the command "left click" is mapped to the `pyautogui.click()` function. When this command is recognised, the system simulates a standard left-mouse-button click at the current cursor position. Similarly, the command "right click" is mapped to `pyautogui.click(button='right')`, enabling users to access context menus and perform secondary mouse interactions through voice control. The command "double click" is mapped to `pyautogui.doubleClick()`, allowing rapid selection and activation of desktop items.

In addition to click operations, the module supports directional cursor movement. Cursor navigation was implemented using the `pyautogui.moveRel()` function, which performs relative cursor movement based on predefined coordinate values. Commands such as "move up", "move down", "move left", and "move right" cause the cursor to move incrementally in the corresponding direction. This approach allows users to reposition the cursor through voice commands and interact with different areas of the desktop environment.

The implementation of mouse-control functionality within the Vosk framework is illustrated in Figure 3.11.

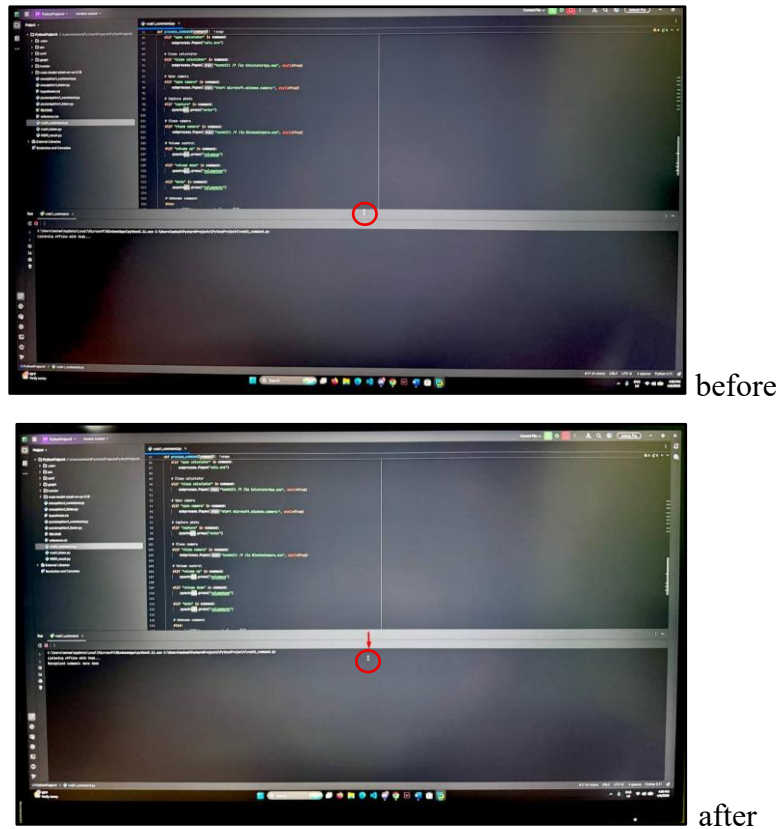


Figure 3.11 Vosk Mouse Control Command Mapping

Figure 3.11 illustrates the implementation of mouse-control commands within the Vosk command-processing subsystem. The screenshot shows the conditional validation logic used to detect mouse-related voice commands and map them to corresponding PyAutoGUI automation functions. The figure demonstrates the implementation of left-click, right-click, double-click, and directional cursor-movement operations. Once a recognised command matches one of the predefined mouse-control keywords, the corresponding automation function is executed immediately to perform the requested action.

The implementation of mouse-control functionality within the PocketSphinx framework is illustrated in Figure 3.12.

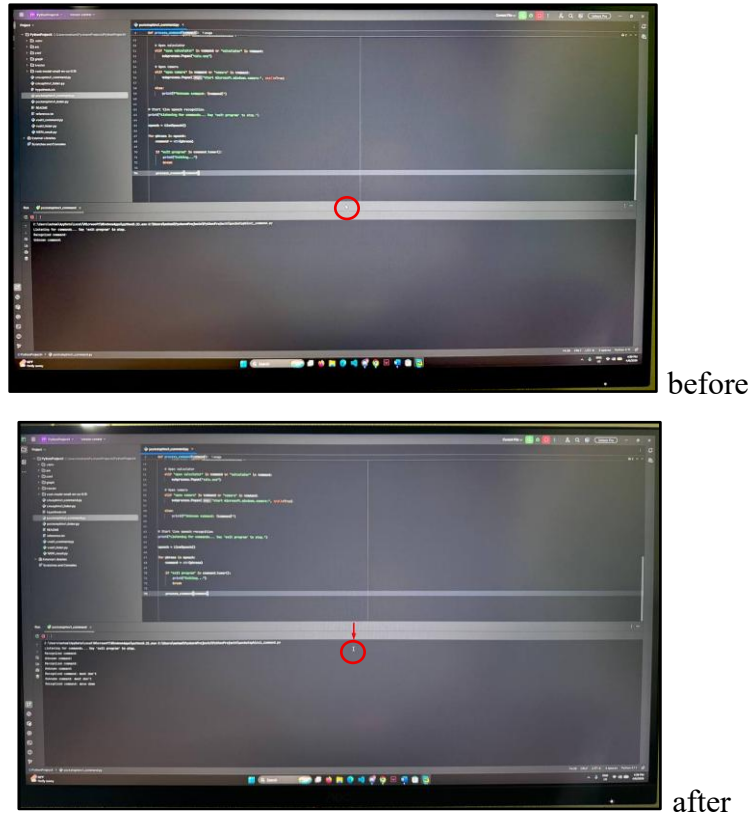


Figure 3.12 PocketSphinx Mouse Control Command Mapping

Figure 3.12 illustrates the command-processing logic used to support mouse-control operations within the PocketSphinx implementation. Similar to the Vosk framework, recognised commands are validated through conditional keyword matching before being forwarded to the PyAutoGUI automation layer. The figure demonstrates how recognised speech commands are translated into executable mouse-control operations during runtime execution.

The implementation of mouse-control functionality within the CMUSphinx framework is illustrated in Figure 3.13.

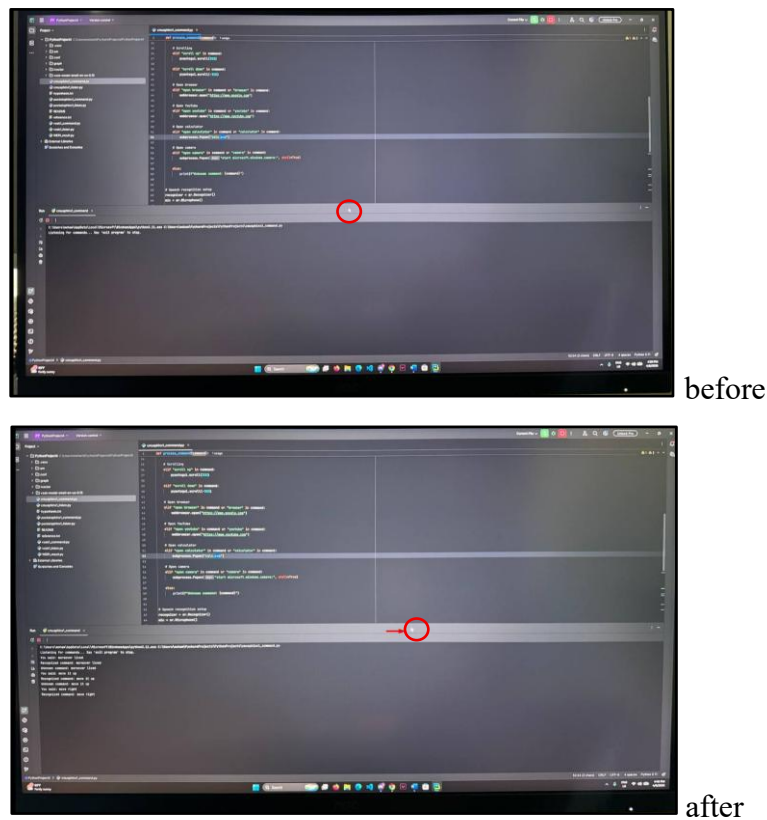


Figure 3.13 CMUSphinx Mouse Control Command Mapping

Figure 3.13 illustrates the implementation of mouse-control commands within the CMUSphinx recognition environment. The screenshot demonstrates the integration of recognised speech, command validation, and desktop automation execution. After a valid mouse-control command is identified, the corresponding PyAutoGUI function is executed to perform cursor movement or mouse-click actions. The implementation also benefits from the error-handling mechanisms integrated within the CMUSphinx subsystem, ensuring stable operation during runtime execution.

The Mouse Control Module therefore provides a practical mechanism for enabling hands-free interaction with desktop graphical user interfaces. By integrating speech recognition with PyAutoGUI-based automation functions, the module allows users to perform essential mouse operations through offline voice commands. This functionality contributes significantly to the

accessibility and usability objectives of the proposed Offline Voice-Controlled Desktop Automation System.

3.6.2 Scrolling Control Module

The Scrolling Control Module was developed to enable users to navigate webpages, documents, and application interfaces through voice commands without requiring interaction with a mouse wheel or keyboard. This functionality improves accessibility and user convenience by allowing vertical navigation to be performed entirely through speech-based interaction.

Within desktop computing environments, scrolling is one of the most frequently performed navigation activities. Therefore, the proposed system was designed to support two primary scrolling operations, namely upward scrolling and downward scrolling. These operations enable users to move through content displayed on the screen using predefined voice commands.

The implementation of the scrolling subsystem was integrated with the command-processing architecture and utilised the PyAutoGUI automation library for execution. After a spoken command is recognised by the speech-recognition subsystem and validated by the command-processing layer, the corresponding scrolling function is executed automatically.

The command "scroll up" was mapped to the `pyautogui.scroll(300)` function. When the command is recognised and validated, the system simulates an upward mouse-wheel scrolling action. Similarly, the command "scroll down" was mapped to `pyautogui.scroll(-300)`, which performs downward scrolling within the currently active window. Positive values generate upward scrolling movement, whereas negative values generate downward scrolling movement.

The implementation of the scrolling subsystem is illustrated in Figure 3.14.

```

58     # Scrolling
59     elif "scroll up" in command:
60         pyautogui.scroll(300)
61
62     elif "scroll down" in command:
63         pyautogui.scroll(-300)

```

Figure 3.14 Scrolling Control Command Mapping

The figure demonstrates how recognised commands are translated into operating-system-level scrolling operations through desktop automation mechanisms. The figure demonstrates how recognised commands are translated into operating-system-level scrolling operations through desktop automation mechanisms.

The scrolling module improves usability by enabling users to navigate digital content through voice interaction while maintaining fully offline operation. By integrating speech recognition with automated scrolling execution, the module contributes to the overall objective of providing accessible and hands-free desktop control.

3.6.3 Browser Automation Module

The Browser Automation Module was developed to enable users to control web-browser operations through voice commands. Browser access is one of the most common desktop activities performed by users, and therefore browser automation was included as an important functionality within the proposed framework.

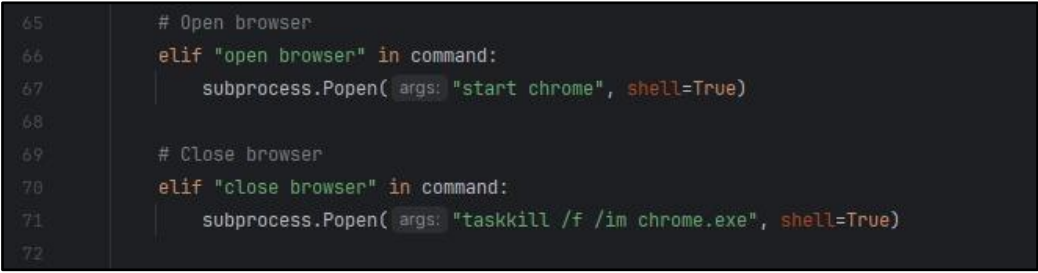
The module allows users to launch and terminate web-browser processes using predefined speech commands. This functionality reduces the need for manual interaction and demonstrates the ability of the proposed system to perform operating-system-level application control through offline voice recognition.

The implementation was integrated with the command-processing subsystem and utilised the Python Webbrowser and Subprocess modules for browser management. Once a recognised command is validated, the corresponding browser-control function is executed automatically.

The command "open browser" was implemented to launch Google Chrome using the subprocess.Popen() function. When the command is recognised and validated, the system executes the "start chrome" instruction, causing the Chrome browser to open automatically. This functionality enables users to access web resources through voice commands without requiring manual interaction with desktop shortcuts or browser icons.

The command "close browser" was implemented to terminate active Google Chrome processes using the taskkill command executed through the Subprocess library. When the command is recognised, the system executes "taskkill /f /im chrome.exe", which forcefully closes all active Chrome browser instances.

The implementation of the browser automation subsystem is illustrated in Figure 3.15.



```
65 # Open browser
66 elif "open browser" in command:
67     subprocess.Popen( args: "start chrome", shell=True)
68
69 # Close browser
70 elif "close browser" in command:
71     subprocess.Popen( args: "taskkill /f /im chrome.exe", shell=True)
72
```

Figure 3.15 Browser Automation Command Mapping

Figure 3.15 illustrates the implementation of browser automation functionality within the proposed system. The screenshot shows the conditional validation logic used to detect the commands "open browser" and "close browser". When the open-browser command is recognised, the system launches Google Chrome through the subprocess.Popen() function. Similarly, when the close-browser command is recognised, the system executes a process-termination instruction to close active Chrome browser instances. The figure demonstrates how recognised speech commands are translated into browser-control operations through operating-system process management.

The browser automation module demonstrates the integration of offline speech recognition with real-time application control and provides users with a convenient mechanism for voice-based browser management.

3.6.4 Multimedia Navigation Module

The Multimedia Navigation Module was developed to support voice-controlled access to online multimedia platforms. Multimedia content represents a significant component of modern computer usage, and therefore voice-based multimedia navigation was incorporated as part of the proposed desktop automation framework.

The module was designed to provide users with rapid access to YouTube through predefined voice commands. By integrating speech recognition with browser-based navigation, users are able to access multimedia resources without requiring manual interaction with a browser interface.

The implementation utilises the Webbrowser module to perform automatic webpage navigation after successful command recognition and validation. Once a multimedia-related command is identified, the system automatically redirects the user to the corresponding online platform.

The command "open YouTube" was implemented to launch the YouTube website automatically within the user's default browser. When recognised, the command triggers a predefined URL navigation process that redirects the user directly to the platform homepage.

The command "close YouTube" was implemented using the Subprocess library to terminate active Google Chrome processes. When the command is recognised and validated, the system executes the "taskkill /f /im chrome.exe" instruction, which closes the browser instance being used to access YouTube. This functionality allows users to terminate multimedia sessions through voice commands without requiring manual browser interaction.

The implementation of the multimedia navigation subsystem is illustrated in Figure 3.16.

```

73     # Open YouTube
74     elif "open" in command and (
75         "youtube" in command or "you tube" in command or "you too" in command
76     ):
77         webbrowser.open("https://www.youtube.com")
78
79     # Close YouTube
80     elif "close" in command and (
81         "youtube" in command or "you tube" in command or "you too" in command
82     ):
83         subprocess.Popen( args: "taskkill /f /im chrome.exe", shell=True)
84

```

Figure 3.16 YouTube Command Mapping Implementation

Figure 3.16 illustrates the implementation of YouTube navigation functionality within the proposed system. The screenshot shows the conditional validation logic used to detect YouTube-related voice commands. Multiple keyword variations including "youtube", "you tube", and "you too" are supported to improve recognition flexibility. When an open command is detected, the system launches the YouTube website using the `webbrowser.open()` function. When a close command is detected, the system executes a Chrome process-termination instruction through the Subprocess library. The figure demonstrates how recognised speech commands are converted into multimedia-navigation operations through desktop automation mechanisms.

The Multimedia Navigation Module extends the capabilities of the proposed system beyond basic desktop control by enabling users to access online multimedia resources through voice commands. This functionality further demonstrates the practical application of offline speech recognition within real-world desktop interaction environments.

Alternative keyword variations were incorporated to compensate for speech-recognition inconsistencies and pronunciation differences during runtime operation.

3.6.5 Application Launching Module

The Application Launching Module was developed to enable users to start desktop applications through voice commands without requiring manual interaction with operating-system menus, icons, or keyboard shortcuts. This functionality demonstrates the ability of the proposed Offline

Voice-Controlled Desktop Automation System to perform direct application control through speech-based interaction.

The module was designed to support commonly used Windows applications and was integrated with the command-processing subsystem through predefined command mappings. After a speech command is recognised by the offline speech-recognition engine and successfully validated by the command-processing architecture, the corresponding application-launching function is executed automatically.

Within the proposed framework, application launching was implemented using the Python subprocess module. The Subprocess library allows external applications to be executed directly from Python scripts by creating new operating-system processes. This capability enables the system to launch software applications in response to recognised voice commands.

Two applications were selected for implementation within this module, namely the Windows Calculator application and the Windows Camera application. These applications were chosen because they represent commonly used desktop utilities and provide a practical demonstration of voice-controlled application management.

3.6.5.1 Calculator Launching Functionality

The Calculator application was integrated into the proposed system to demonstrate voice-controlled execution of desktop utility software. When the command "open calculator" is recognised and validated, the command-processing subsystem forwards the instruction to the application-launching module.

The module subsequently executes the Calculator application using the Subprocess library. The operating system then creates a new process and launches the Calculator interface automatically without requiring any manual user interaction.

The implementation of the Calculator launching functionality is illustrated in Figure 3.17.

```

85     # Open calculator
86     elif "open calculator" in command:
87         subprocess.Popen("calc.exe")
88
89     # Close calculator
90     elif "close calculator" in command:
91         subprocess.Popen(args="taskkill /f /im CalculatorApp.exe", shell=True)
92

```

Figure 3.17 Calculator Application Control Functionality

Figure 3.17 illustrates the implementation of Calculator application management within the proposed framework. The screenshot shows the conditional validation logic used to detect both "open calculator" and "close calculator" commands. When the open command is recognised, the system launches the Calculator application using the `subprocess.Popen("calc.exe")` function. When the close command is recognised, the system executes a process-termination instruction using the Windows `taskkill` command. The figure demonstrates how recognised speech commands are translated into application-launching and application-termination operations through the desktop automation subsystem.

In addition to application launching, the Calculator module also supports application termination. The command "close calculator" was implemented using the Subprocess library together with the Windows `taskkill` command. When recognised and validated, the system executes `"taskkill /f /im CalculatorApp.exe"`, which terminates active Calculator processes. This functionality enables users to both launch and close the Calculator application through voice commands, providing greater control over desktop applications during runtime operation.

3.6.5.2 Camera Launching Functionality

The Camera application was integrated into the proposed system to further demonstrate application-control capabilities within the desktop environment. Similar to the Calculator implementation, the Camera application can be launched directly through voice commands after successful command recognition and validation.

When the command "open camera" is recognised, the validated instruction is forwarded to the application-launching module. The Subprocess library is then used to initiate the Camera application by executing the corresponding operating-system command.

The implementation of the Camera launching functionality is illustrated in Figure 3.18.

```
93     # Open camera
94     elif "open camera" in command:
95         subprocess.Popen(args: "start microsoft.windows.camera:", shell=True)
96
97     # Capture photo
98     elif "capture" in command:
99         pyautogui.press("enter")
100
101     # Close camera
102     elif "close camera" in command:
103         subprocess.Popen(args: "taskkill /f /im WindowsCamera.exe", shell=True)
104
```

Figure 3.18 Camera Application Control Functionality

Figure 3.18 illustrates the implementation of Camera application control functionality within the proposed framework. The screenshot shows the conditional validation logic used to detect camera-related voice commands including "open camera", "capture", and "close camera". When the open-camera command is recognised, the system launches the Windows Camera application through the Subprocess library. The capture command triggers a simulated Enter-key press using the PyAutoGUI library to activate the image-capture function. When the close-camera command is recognised, the system executes a Windows process-termination instruction to close the active Camera application. The figure demonstrates how recognised speech commands are translated into camera-management operations through desktop automation mechanisms.

In addition to application launching, the Camera module also supports image-capture and application-termination functionalities. The "capture" command was implemented using the `pyautogui.press("enter")` function, which simulates the Enter key to trigger the camera shutter button. Furthermore, the "close camera" command was implemented using the Subprocess library together with the Windows taskkill command. When recognised and validated, the system executes `"taskkill /f /im WindowsCamera.exe"` to terminate the active Camera process. These additional functionalities provide users with greater control over the Camera application through voice interaction.

The Application Launching Module therefore extends the capabilities of the proposed framework beyond basic cursor control and browser interaction by enabling direct execution of desktop software. Through the integration of speech recognition, command validation, and subprocess-based application management, users are able to launch, control, and terminate applications efficiently using hands-free voice commands

3.6.6 Audio and Volume Control Module

The Audio and Volume Control Module was developed to enable users to manage operating-system audio settings through voice commands. Audio control is an important desktop interaction feature because users frequently adjust volume levels while using multimedia applications, communication platforms, and web-based content. By integrating audio-control functionality into the proposed Offline Voice-Controlled Desktop Automation System, users can manipulate system sound settings without requiring physical interaction with keyboards, mice, or operating-system controls.

The implementation of this module was integrated directly into the command-processing subsystem. After a voice command is recognised by the speech-recognition framework and validated through the command-processing architecture, the corresponding audio-control function is executed automatically.

The module supports three primary audio-management operations, namely volume increase, volume decrease, and mute functionality. These operations provide users with basic but essential control over multimedia settings within the desktop environment.

The implementation was achieved using the PyAutoGUI automation library. Rather than interacting directly with audio-management APIs, the system simulates multimedia keyboard events recognised by the Windows operating system. This approach simplifies implementation while maintaining compatibility with standard desktop environments.

The command "volume up" was mapped to the `pyautogui.press("volumeup")` function. When this command is recognised and validated, the system simulates a multimedia key press that increases the operating-system audio level. Similarly, the command "volume down" was

mapped to the `pyautogui.press("volumedown")` function, allowing users to reduce the current audio level through voice interaction.

The mute functionality was implemented using the `pyautogui.press("volumemute")` command. When the user issues the "mute" command, the operating-system audio output is immediately muted through simulated multimedia-key interaction.

The implementation of the audio-control subsystem is illustrated in Figure 3.19.

```
104
105     # Volume control
106     elif "volume up" in command:
107         pyautogui.press("volumeup")
108
109     elif "volume down" in command:
110         pyautogui.press("volumedown")
111
112     elif "mute" in command:
113         pyautogui.press("volumemute")
114
```

Figure 3.19 Volume Control Command Mapping

Figure 3.19 illustrates the implementation of volume-control functionality within the proposed framework. The screenshot demonstrates the conditional validation structures used to detect volume-related voice commands and map them to the corresponding PyAutoGUI multimedia-control functions. The figure shows how recognised commands such as "volume up", "volume down", and "mute" are converted into operating-system audio-control actions through keyboard-event simulation.

The Audio and Volume Control Module demonstrate the ability of the proposed framework to interact with operating-system multimedia functions through offline voice commands. By integrating speech recognition, command validation, and multimedia-key automation, the module provides users with convenient and hands-free control over desktop audio settings.

3.7 Chapter Summary

This chapter presented the methodology adopted for the design, development, and implementation of the proposed Offline Voice-Controlled Desktop Automation System. A prototype-based development methodology was selected because it provided a structured framework for designing, implementing, testing, and refining the system through iterative development stages. The chapter described the development process followed throughout the study, beginning with requirement analysis and continuing through system design, implementation, integration, and module development.

The chapter also discussed the requirement analysis conducted prior to implementation. Functional requirements, non-functional requirements, user requirements, hardware requirements, and software requirements were identified to establish the operational scope and technical foundation of the proposed system. These requirements guided the development process and ensured that the implemented framework satisfied the objectives established in Chapter 1.

In addition, the chapter presented the overall system architecture developed for the proposed framework. The architecture illustrated the interaction between the audio acquisition subsystem, offline speech-recognition engines, command-processing mechanisms, desktop automation layer, and runtime monitoring components. The implementation of three offline speech-recognition frameworks, namely Vosk, PocketSphinx, and CMUSphinx, was also described together with the speech-recognition algorithms utilised by each framework.

The operational workflows implemented within the system were further discussed, including the overall speech-to-action process and the command validation and execution mechanisms used to transform recognised speech into executable desktop operations. The chapter additionally described the implementation of the major functional modules developed for the system, including mouse control, scrolling control, browser automation, multimedia navigation, application launching, audio control, and runtime monitoring.

Through the integration of offline speech-recognition technologies and Python-based automation libraries, a complete voice-controlled desktop automation environment was

successfully designed and implemented. The methodology, architecture, workflows, and implementation procedures presented in this chapter provide the technical foundation for evaluating the performance and effectiveness of the proposed system.

The next chapter presents the evaluation and analysis of the developed system. The performance of the proposed framework is assessed through functional testing, command-recognition accuracy analysis, Word Error Rate (WER) evaluation, command-execution validation, environmental robustness testing, and comparative evaluation of the integrated speech-recognition frameworks.

CHAPTER 4

IMPLEMENTATION

4.1 Introduction

This chapter presents the evaluation and performance analysis of the proposed Offline Voice-Controlled Desktop Automation System developed using Vosk, PocketSphinx, and CMUSphinx speech-recognition frameworks. The purpose of this evaluation is to determine the effectiveness of the proposed system in performing real-time desktop automation through offline speech recognition without relying on cloud-based services or internet connectivity.

The evaluation process was conducted by testing the developed system under controlled operating conditions using predefined voice commands associated with desktop automation tasks such as mouse control, scrolling, browser automation, multimedia navigation, application launching, and volume control. Multiple testing procedures were performed to assess recognition accuracy, command detection capability, Word Error Rate (WER), runtime execution behaviour, and environmental robustness under different noise conditions.

To ensure fair comparison, identical command sets, hardware configurations, microphone settings, and testing procedures were used for all three speech-recognition frameworks. Multiple recognition attempts were conducted for each framework in order to evaluate consistency, reliability, and overall recognition performance.

The evaluation presented in this chapter focuses on several key performance metrics including functional testing success rate, speech-recognition accuracy, command detection rate, Word Error Rate (WER), runtime execution reliability, and environmental noise tolerance. These metrics provide a comprehensive assessment of the proposed system and enable comparative analysis of the strengths and limitations of Vosk, PocketSphinx, and CMUSphinx within offline desktop automation environments.

The findings obtained from the evaluation process are subsequently analysed and compared to determine the most suitable speech-recognition framework for offline voice-controlled desktop

automation. The results presented in this chapter provide the foundation for the discussion, conclusions, limitations, and future improvement recommendations presented in Chapter 5.

4.2 Experimental Setup and Testing Environment

Before conducting system evaluation, a controlled testing environment was established to ensure consistency and fairness during performance assessment. The same hardware platform, software environment, microphone device, command vocabulary, and testing procedures were used throughout all experiments involving Vosk, PocketSphinx, and CMUSphinx. Maintaining identical testing conditions ensured that performance differences observed during evaluation were primarily attributable to the speech-recognition frameworks rather than external environmental factors.

The proposed Offline Voice-Controlled Desktop Automation System was implemented and evaluated using a Windows-based computing environment. Python was used as the primary programming language for system development, while PyCharm served as the Integrated Development Environment (IDE) for coding, debugging, and testing activities. The evaluation involved three offline Automatic Speech Recognition (ASR) frameworks, namely Vosk, PocketSphinx, and CMUSphinx, operating under identical runtime conditions.

Audio acquisition was performed using a microphone connected to the testing computer. Speech input was captured using a sampling rate of 16,000 Hz, which is the recommended configuration for the offline speech-recognition models utilised in this study. The testing environment was maintained indoors to minimise uncontrolled environmental interference and to provide stable conditions during recognition experiments.

The hardware and software configuration used during evaluation is summarised in Table 4.1.

Component	Specification
Operating System	Windows 11
Programming Language	Python
IDE	PyCharm
ASR Frameworks	Vosk, PocketSphinx, CMUSphinx
Sampling Rate	16000 Hz
Microphone Device	Hyperx cloud3
Testing Environment	Indoor

Table 4.1 Experimental Environment Configuration

Table 4.1 presents the hardware and software environment used throughout the evaluation process. The configuration remained unchanged during all testing procedures to ensure consistency across experiments. By maintaining identical experimental conditions, the evaluation results obtained for Vosk, PocketSphinx, and CMUSphinx could be compared fairly and objectively.

The testing procedures involved multiple command-recognition trials, command-execution verification, response-time observation, Word Error Rate (WER) evaluation, and environmental noise testing. All frameworks were evaluated using the same predefined voice-command vocabulary developed during system implementation. The use of identical command sets ensured that each recognition framework was assessed under equivalent operating conditions.

The experimental setup therefore provided a controlled and repeatable evaluation environment suitable for analysing the effectiveness, reliability, and performance of the proposed Offline Voice-Controlled Desktop Automation System.

4.3 Functional Testing of Desktop Automation Commands

Functional testing was conducted to verify whether the proposed Offline Voice-Controlled Desktop Automation System could successfully perform the desktop automation tasks implemented during system development. The objective of this evaluation was to determine whether recognised voice commands were correctly interpreted, validated, and executed as intended.

The testing process involved issuing predefined voice commands associated with mouse control, scrolling operations, browser automation, multimedia navigation, application launching, and audio control. Each command was spoken multiple times and the resulting desktop action was observed to determine whether the expected functionality was successfully executed.

For every test case, the expected output was first defined based on the intended system behaviour. The actual output generated by the system was then recorded and compared against the expected outcome. A command was classified as Pass when the automation action was executed correctly and classified as Fail when the expected desktop action was not performed.

The functional testing results obtained during system evaluation are presented in Table 4.2.

Command	Expected Output	Actual Output	Status
Open Browser	Google Chrome launches	Google Chrome launched successfully	Pass
Close Browser	Google Chrome closes	Google Chrome closed successfully	Pass
Open YouTube	YouTube website opens	YouTube website opened successfully	Pass
Close YouTube	Browser process closes	Browser process closed successfully	Pass
Open Calculator	Calculator launches	Calculator launched successfully	Pass
Close Calculator	Calculator closes	Calculator closed successfully	Pass
Open Camera	Camera application launches	Camera launched successfully	Pass
Capture	Camera captures image	Capture function executed successfully	Pass
Close Camera	Camera closes	Camera closed successfully	Pass
Left Click	Mouse performs left click	Left click executed successfully	Pass
Right Click	Mouse performs right click	Right click executed successfully	Pass
Double Click	Mouse performs double click	Double click executed successfully	Pass
Move Up	Cursor moves upward	Cursor moved upward successfully	Pass
Move Down	Cursor moves downward	Cursor moved downward successfully	Pass
Move Left	Cursor moves left	Cursor moved left successfully	Pass
Move Right	Cursor moves right	Cursor moved right successfully	Pass
Scroll Up	Screen scrolls upward	Upward scrolling executed successfully	Pass
Scroll Down	Screen scrolls downward	Downward scrolling executed successfully	Pass
Volume Up	System volume increases	Volume increased successfully	Pass
Volume Down	System volume decreases	Volume decreased successfully	Pass
Mute	System audio muted	Audio muted successfully	Pass

Table 4.2 Functional Testing Results

The functional testing results demonstrate that the proposed system successfully executed the implemented desktop automation commands through offline voice interaction. All predefined commands were correctly recognised, validated, and mapped to their corresponding automation functions. The successful completion of these tests confirms that the integration between the speech-recognition subsystem, command-processing subsystem, and desktop automation layer operated as intended during runtime execution.

The following figures provide visual evidence of selected automation tasks executed during functional testing.

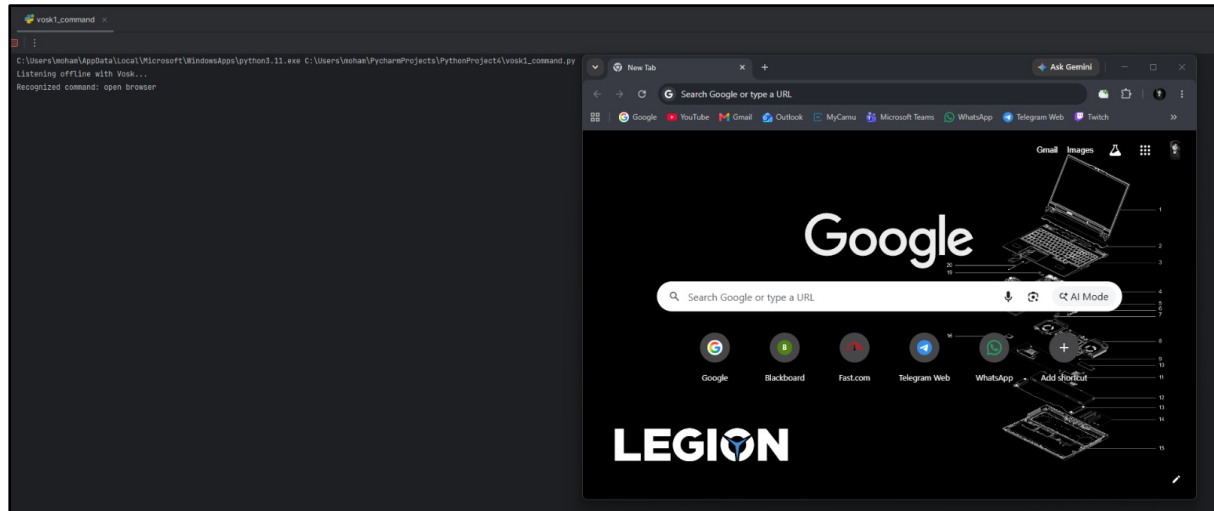


Figure 4.1 Browser Automation Output

Figure 4.1 presents the successful execution of the "open browser" voice command during functional testing. The screenshot shows that the Vosk speech-recognition engine successfully recognised the spoken command and displayed the recognised text within the runtime console. Following command validation, the automation subsystem executed the corresponding browser-launching function and successfully opened Google Chrome. The appearance of the browser window confirms that the command-processing subsystem correctly mapped the recognised speech input to the intended desktop automation action. The successful execution of this operation demonstrates the ability of the proposed system to perform voice-controlled browser automation in a fully offline environment.

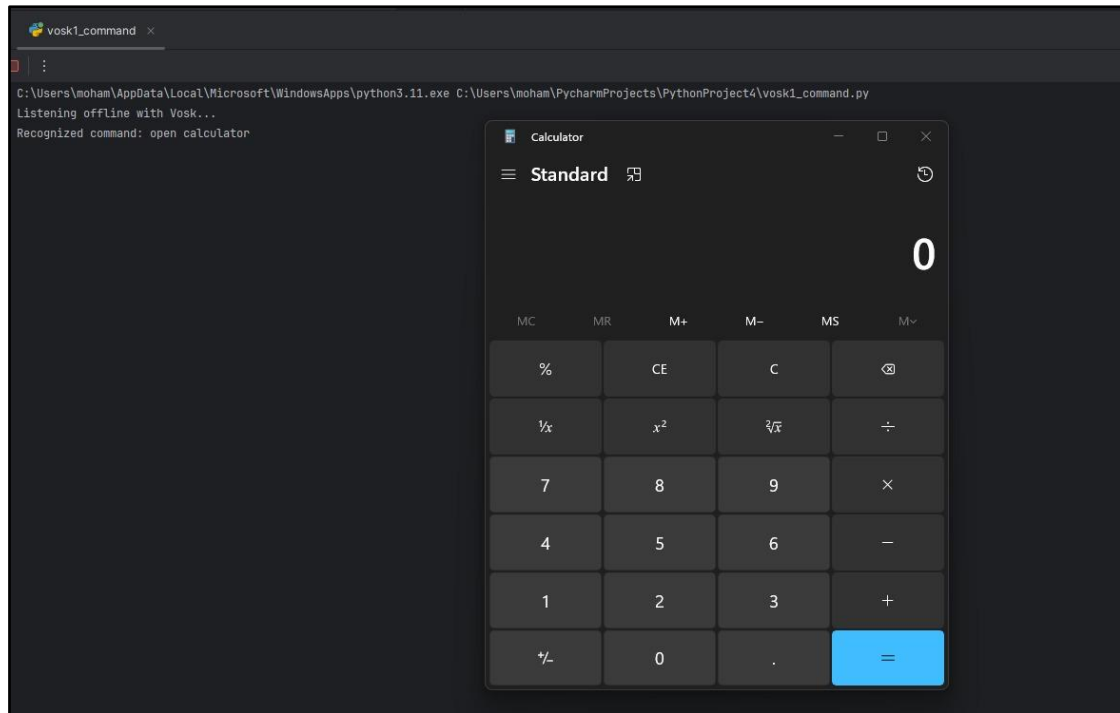


Figure 4.2 Calculator Automation Output

Figure 4.2 illustrates the successful execution of the "open calculator" command during system evaluation. The recognised command is displayed within the runtime console, indicating that the speech-recognition subsystem successfully converted the spoken instruction into text. After command validation was completed, the automation subsystem launched the Windows Calculator application using the predefined application-execution mechanism. The appearance of the Calculator window confirms that the proposed system successfully integrated offline speech recognition with application-launching functionality. This result verifies that voice-controlled execution of desktop applications can be achieved without requiring manual user interaction.

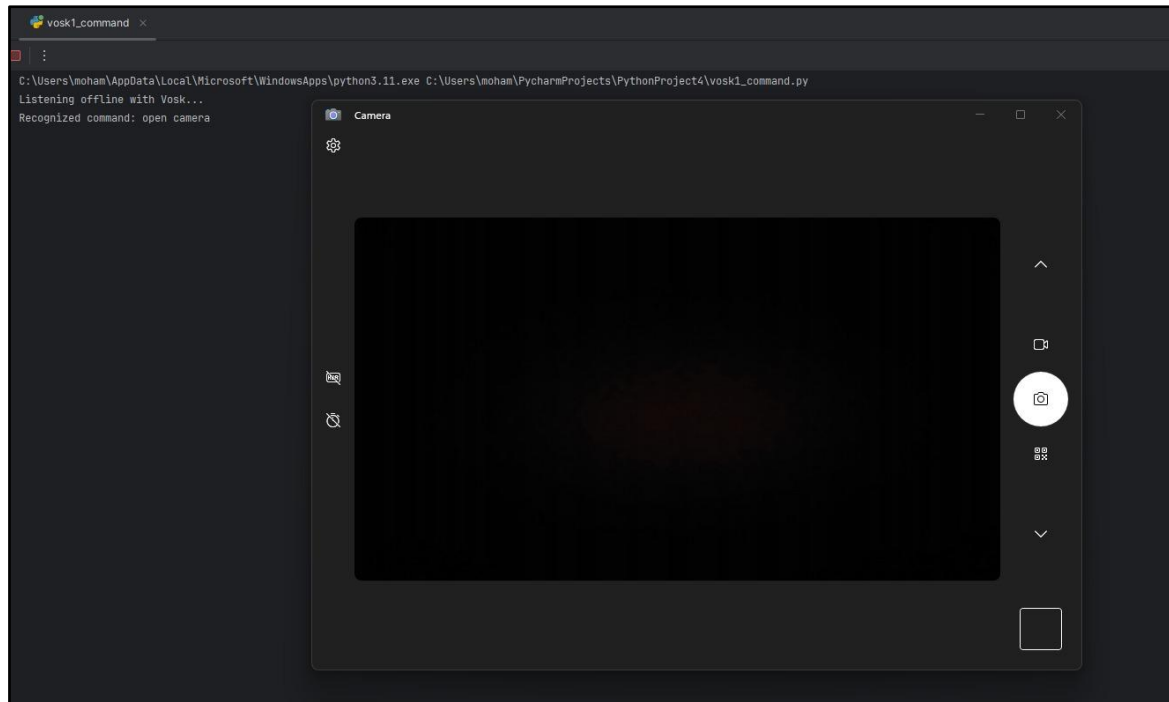


Figure 4.3 Camera Automation Output

Figure 4.3 demonstrates the successful execution of the "open camera" voice command. The screenshot shows that the command was correctly recognised and displayed within the runtime console before being forwarded to the automation layer for execution. Following command validation, the Windows Camera application was successfully launched. The displayed camera interface confirms that the proposed system was capable of interacting with operating-system resources and initiating external applications through offline voice commands. This result further validates the effectiveness of the application-control functionality implemented within the proposed framework.

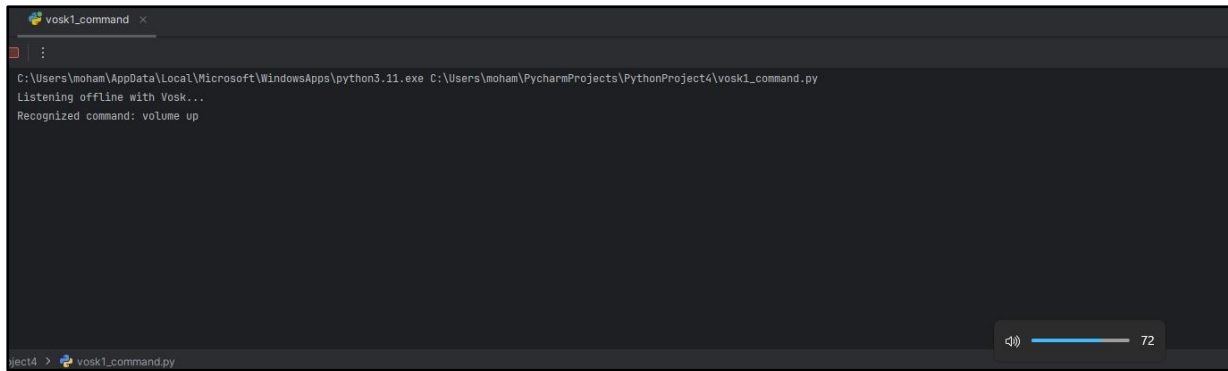


Figure 4.4 Volume Increase Output

Figure 4.4 presents the successful execution of the "volume up" voice command during functional testing. The recognised command is displayed within the runtime console and the operating-system volume indicator confirms that the system volume level increased after command execution. This result demonstrates that the command-processing subsystem correctly identified the recognised command and mapped it to the corresponding multimedia-control function. The successful adjustment of the system volume verifies the effectiveness of the audio-control subsystem implemented within the proposed desktop automation framework.

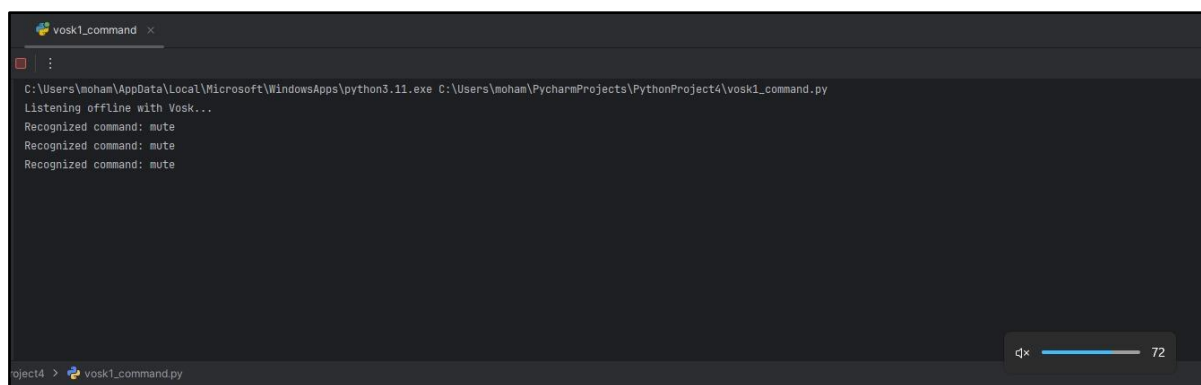


Figure 4.5 Volume Mute Output

Figure 4.5 illustrates the successful execution of the "mute" voice command. The screenshot shows multiple recognised instances of the mute command displayed within the runtime console, indicating that the speech-recognition subsystem consistently detected the spoken instruction. After validation, the automation subsystem triggered the multimedia mute function and successfully modified the system audio state. The operating-system volume indicator

confirms that the mute operation was executed correctly. This result demonstrates the reliability of the voice-controlled multimedia subsystem and confirms that the proposed system can successfully perform operating-system-level audio management through offline speech interaction.

Functional Testing Analysis

The functional testing results presented in Figures 4.1 to 4.5 demonstrate that the proposed Offline Voice-Controlled Desktop Automation System successfully executed the implemented automation functions through offline speech interaction. Browser automation, application launching, camera activation, volume adjustment, and mute functionality were all performed successfully following speech recognition and command validation. The successful execution of these operations confirms the effective integration of the speech-recognition subsystem, command-processing subsystem, and desktop automation layer. Based on the observed results, all evaluated commands achieved their intended functionality and therefore satisfied the predefined functional testing requirements established for the proposed system.

4.3.1 User Voice Testing Sessions

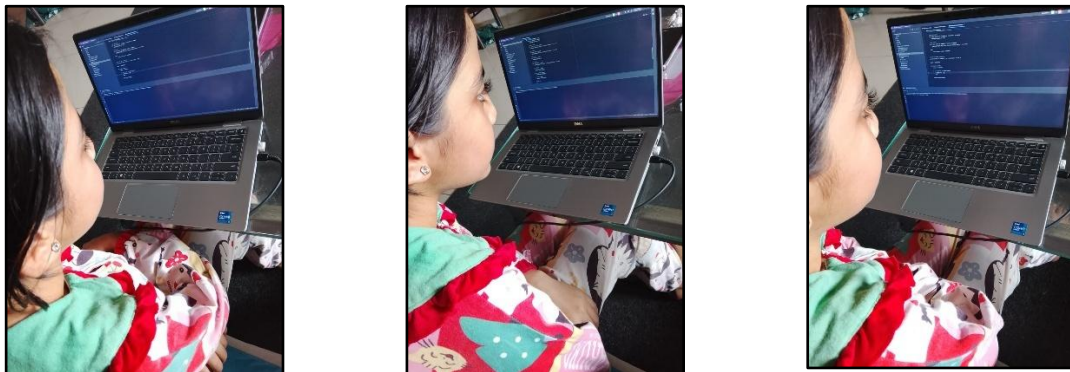


Figure 4.6 User 1 Voice Command Testing Session

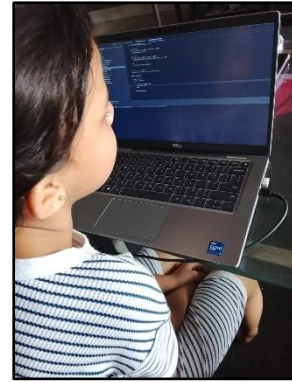
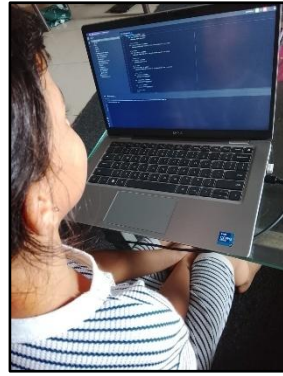
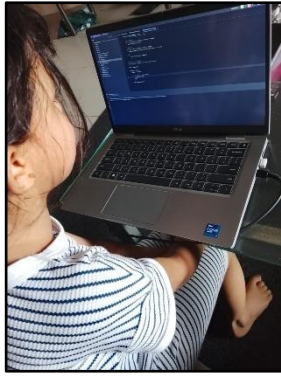


Figure 4.7 User 2 Voice Command Testing Session

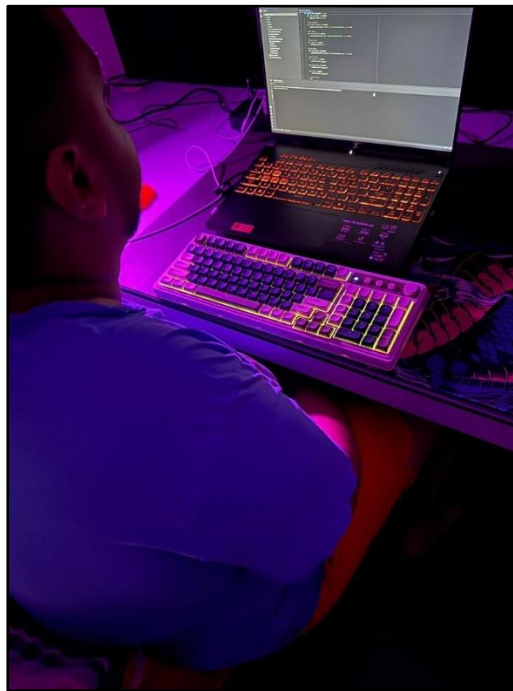


Figure 4.8 User 3 Voice Command Testing Session



Figure 4.9 User 4 Voice Command Testing Session

To evaluate the practical performance of the proposed offline voice-controlled desktop automation system, voice-command testing was conducted with 17 participants from different age groups and genders. The participant pool included both children and adults with varying speech patterns, pronunciations, accents, and speaking speeds. This diversity helped assess the robustness and usability of the system under realistic operating conditions.

During the testing sessions, participants were instructed to issue predefined voice commands related to browser automation, application launching, cursor movement, scrolling operations, volume control, and other supported desktop automation functions. The speech-recognition engine processed each spoken command and translated it into the corresponding desktop action in real time.

The objective of the testing was to evaluate the system's command-recognition accuracy, responsiveness, and overall usability across different users. Variations in pronunciation, speech speed, and speaking style were considered during the evaluation to determine the robustness of the proposed system under real-world conditions.

The results demonstrated that the system was capable of recognising and executing the majority of commands successfully, indicating that the offline speech-recognition approach can provide effective and reliable desktop automation.

4.4 Vosk Recognition Performance Evaluation

The Vosk speech-recognition framework was evaluated to determine its effectiveness in recognising predefined voice commands and executing the corresponding desktop automation functions. The evaluation focused on command-recognition accuracy, recognition consistency, command-execution reliability, and overall operational performance during runtime interaction.

During testing, multiple voice commands representing different automation categories were issued to the system. The recognised commands were displayed through the runtime console and subsequently forwarded to the command-processing subsystem for validation and execution. Successful command execution indicated that both the speech-recognition subsystem and automation subsystem were operating correctly.

The evaluation included browser automation commands, application-launching commands, cursor-control commands, and scrolling commands. The following figures present representative examples of successful command recognition and execution using the Vosk framework.

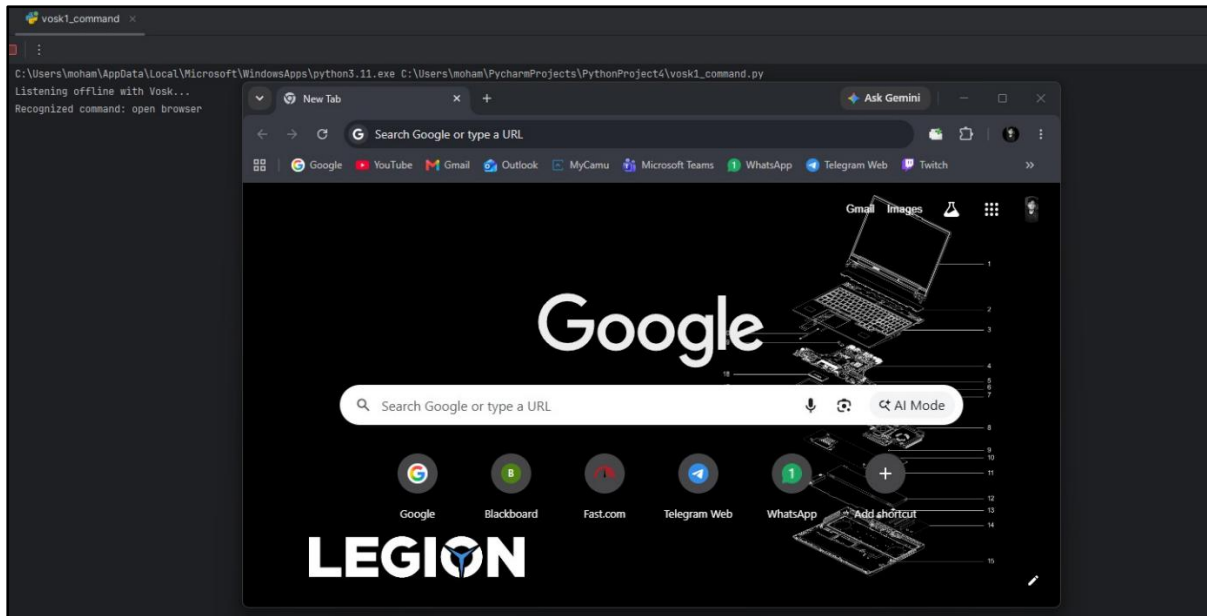


Figure 4.10 Vosk Recognition Test – Attempt 1

Figure 4.10 illustrates the successful recognition and execution of the "open browser" voice command using the Vosk speech-recognition framework. The recognised command is displayed within the runtime console, confirming that the spoken instruction was successfully converted into text. Following command validation, the browser automation subsystem launched Google Chrome successfully. The result demonstrates the ability of Vosk to recognise browser-control commands accurately and initiate the corresponding desktop automation action.

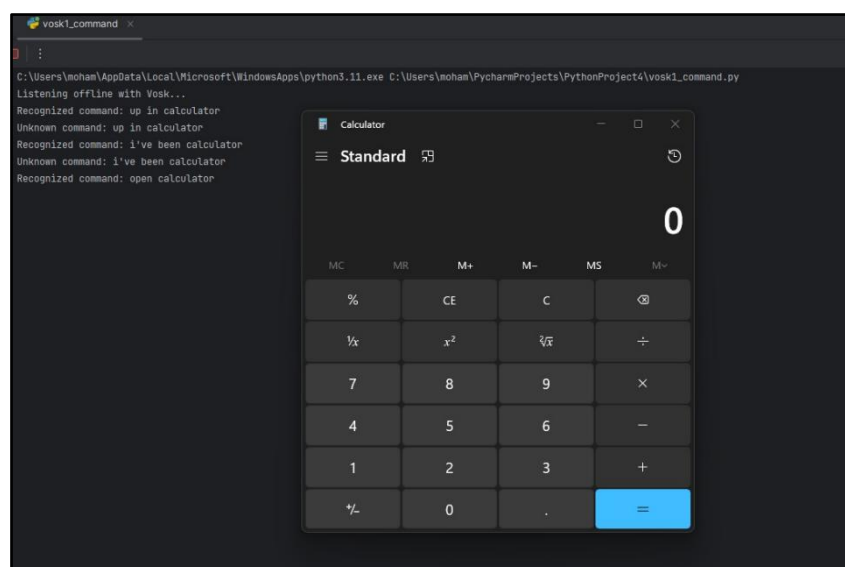
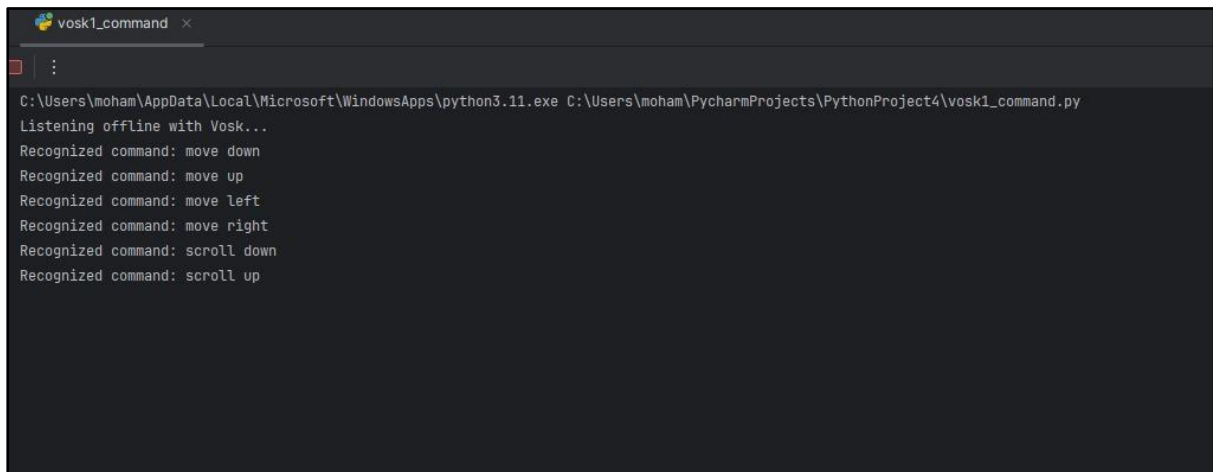


Figure 4.11 Vosk Recognition Test – Attempt 2

Figure 4.11 presents the successful recognition and execution of the "open calculator" command. The screenshot shows that the recognised command was correctly detected by the Vosk recognition engine and displayed within the runtime console. Following validation, the automation subsystem executed the application-launching function and successfully opened the Windows Calculator application. The result confirms the effectiveness of Vosk in supporting voice-controlled application management.



```
vosk1_command x
:
C:\Users\moham\AppData\Local\Microsoft\WindowsApps\python3.11.exe C:\Users\moham\PycharmProjects\PythonProject4\vosk1_command.py
Listening offline with Vosk...
Recognized command: move down
Recognized command: move up
Recognized command: move left
Recognized command: move right
Recognized command: scroll down
Recognized command: scroll up
```

Figure 4.12 Vosk Recognition Test – Attempt 3

Figure 4.12 demonstrates the successful recognition of multiple cursor-control and scrolling commands using the Vosk framework. Commands including "move up," "move down," "move left," "move right," "scroll up," and "scroll down" were recognised correctly and displayed within the runtime console. The successful detection of these commands confirms the ability of the Vosk framework to support continuous command recognition during runtime operation while maintaining reliable command-processing performance.

Test Command	Recognition Result	Automation Executed	Status
Open Browser	Correct	Yes	Pass
Open Calculator	Correct	Yes	Pass
Move Up	Correct	Yes	Pass
Move Down	Correct	Yes	Pass
Move Left	Correct	Yes	Pass
Move Right	Correct	Yes	Pass
Scroll Up	Correct	Yes	Pass
Scroll Down	Correct	Yes	Pass

Table 4.3 Vosk Recognition Results

Table 4.3 presents the recognition results obtained during Vosk command testing. The evaluation involved commands from multiple automation categories including browser control, application launching, cursor movement, and scrolling operations. The results demonstrate that every tested command was successfully recognised and executed by the proposed system.

The successful recognition of browser-related commands confirms the effectiveness of Vosk in handling application-control instructions. Similarly, the correct execution of calculator-launching commands demonstrates the ability of the framework to support voice-controlled application management. The cursor-movement and scrolling commands were also recognised correctly, indicating that Vosk can reliably process short navigation-related instructions during continuous system operation.

Although Table 4.3 presents representative command examples, additional commands were also evaluated during testing. Therefore, the overall accuracy calculation was based on eleven tested commands.

Total Commands Tested	Correct Recognition	Accuracy (%)
11	9	81.8%

Table 4.4 Vosk Recognition Accuracy Summary

Table 4.4 summarises the overall recognition performance achieved by the Vosk speech-recognition framework during experimental evaluation. A total of eleven predefined voice commands representing different desktop automation functions were tested. These commands

included browser automation, application launching, cursor movement, scrolling operations, and other desktop control functions.

The results show that nine out of eleven commands were recognised correctly and executed successfully, resulting in an overall recognition accuracy of 81.8%. Although a small number of recognition errors were observed during testing, the framework demonstrated strong recognition consistency and reliable command-execution performance throughout the evaluation process.

The recognition errors may have been caused by pronunciation variations, microphone sensitivity, background noise, or limitations in speech segmentation during real-time operation. Nevertheless, the majority of commands were successfully recognised and executed, indicating that the framework was capable of providing dependable offline speech recognition for desktop automation tasks.

The high recognition accuracy achieved by Vosk can be attributed to its Kaldi-based Deep Neural Network architecture, which provides robust acoustic modelling and improved recognition capability compared with traditional speech-recognition approaches. The results therefore suggest that Vosk is highly suitable for offline voice-controlled desktop automation applications and provides reliable command recognition during real-time system operation.

4.5 PocketSphinx Recognition Performance Evaluation

The PocketSphinx speech-recognition framework was evaluated to assess its ability to recognise predefined voice commands and support desktop automation operations within a fully offline environment. The evaluation focused on command-recognition capability, recognition consistency, recognition accuracy, and overall reliability during runtime operation.

PocketSphinx continuously listened for spoken commands through the microphone and converted the captured speech into textual output. The recognised text was subsequently forwarded to the command-processing subsystem where validation and automation execution were performed. During testing, PocketSphinx demonstrated the ability to recognise valid commands and trigger the corresponding desktop automation functions. However, compared

with Vosk, the framework occasionally produced incorrectly recognised phrases that did not correspond to valid commands stored within the system vocabulary.

The following figures present representative examples of command recognition and execution using PocketSphinx.

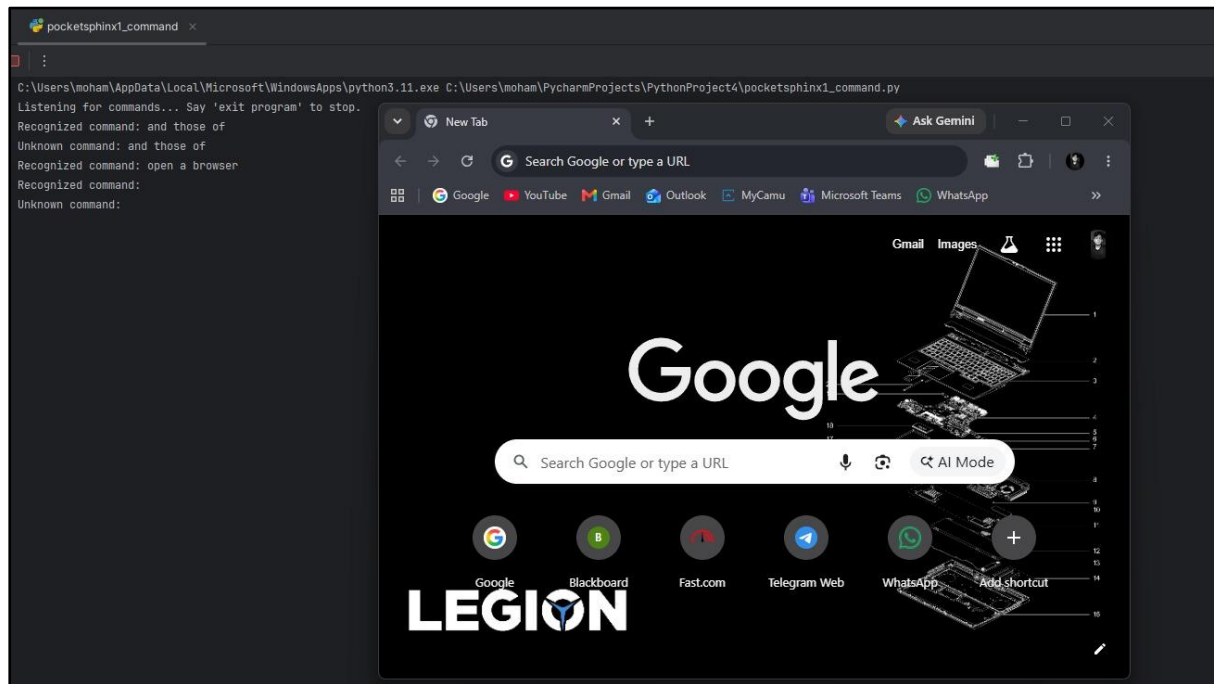


Figure 4.13 PocketSphinx Recognition Test – Attempt 1

Figure 4.13 illustrates the recognition and execution of the browser-launching command using PocketSphinx. The screenshot shows that the framework recognised the spoken command as "open a browser", which was subsequently matched to the predefined browser command within the command-processing subsystem. Following validation, Google Chrome was successfully launched.

The figure additionally demonstrates that PocketSphinx occasionally generated incorrect recognition outputs such as "and those of", which did not correspond to any valid automation command. These phrases were correctly identified by the validation subsystem and classified as unknown commands. The result demonstrates that the proposed framework successfully prevented unintended automation execution while maintaining operational safety.

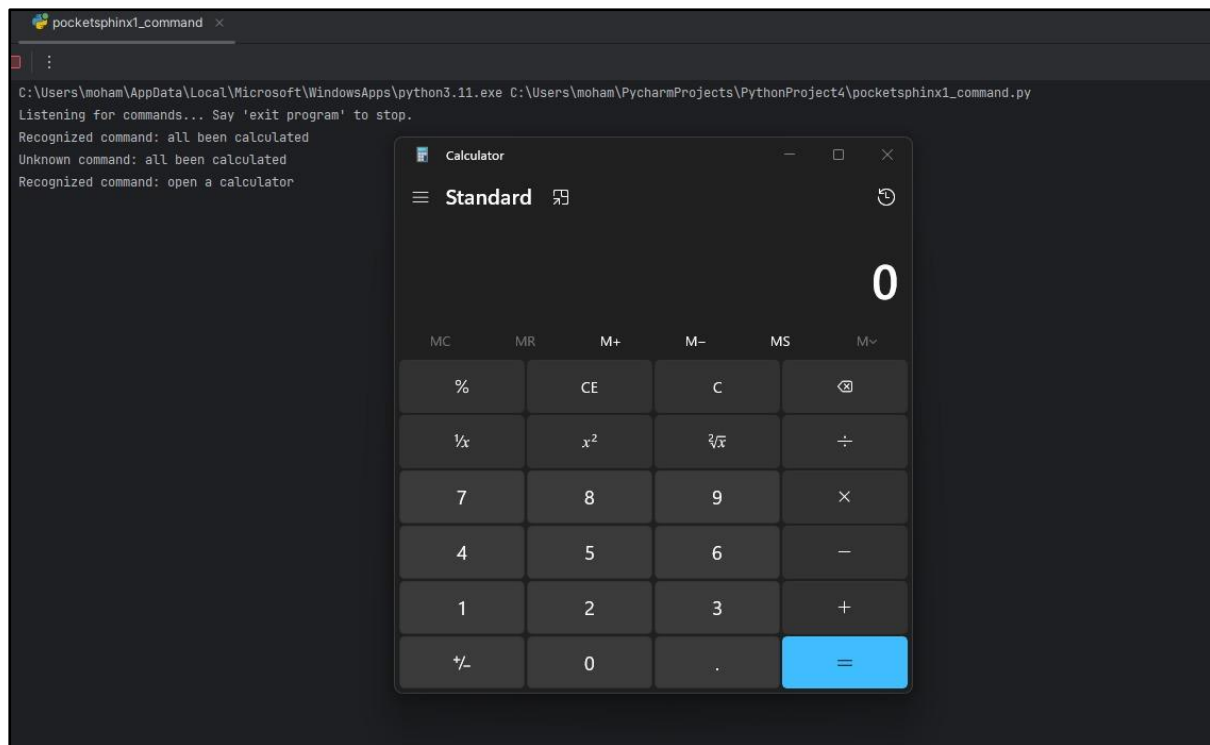


Figure 4.14 PocketSphinx Recognition Test – Attempt 2

Figure 4.14 presents the recognition and execution of the calculator-launching command using PocketSphinx. The screenshot shows that the framework initially recognised an incorrect phrase, namely "all been calculated", which did not correspond to a valid automation command and was therefore rejected by the validation subsystem.

Subsequently, PocketSphinx correctly recognised the command "open a calculator", resulting in successful execution of the Calculator application. The appearance of the Calculator window confirms that valid commands were correctly processed and executed despite the occurrence of earlier recognition inaccuracies. This result demonstrates the ability of PocketSphinx to perform application-launching tasks while highlighting its susceptibility to occasional recognition errors.


```

C:\Users\moham\AppData\Local\Microsoft\WindowsApps\python3.11.exe C:\Users\moham\PycharmProjects\PythonProject4\pocketsphinx1_command.py
Listening for commands... Say 'exit program' to stop.
Recognized command: morpheus
Unknown command: morpheus
Recognized command: overnight
Unknown command: overnight
Recognized command: oh right
Unknown command: oh right
Recognized command: all down
Unknown command: all down
Recognized command: hold up
Recognized command: walcott
Unknown command: walcott
Recognized command: move up

```

Figure 4.15 PocketSphinx Recognition Test – Attempt 3

Figure 4.15 illustrates recognition behaviour observed during continuous runtime testing. Several incorrectly recognised phrases including "morpheus," "overnight," "oh right," "all down," "hold up," and "walcott" were detected and subsequently rejected by the command-processing subsystem because they did not match any predefined command vocabulary.

Following these recognition attempts, PocketSphinx successfully recognised the valid command "move up". This result demonstrates that although the framework occasionally generated recognition errors, the implemented validation mechanism successfully prevented invalid command execution. The figure also highlights the importance of command validation in maintaining system reliability during continuous voice interaction.

Test Command	Recognition Result	Automation Executed	Status
Open Browser	Recognised as "open a browser"	Yes	Pass
Open Calculator	Recognised as "open a calculator"	Yes	Pass
Move Up	Recognised correctly	Yes	Pass
Invalid Phrase ("and those of")	Not Executed	No	Pass
Invalid Phrase ("all been calculated")	Not Executed	No	Pass
Invalid Phrase ("morpheus")	Not Executed	No	Pass
Invalid Phrase ("overnight")	Not Executed	No	Pass
Invalid Phrase ("oh right")	Not Executed	No	Pass

Table 4.5 PocketSphinx Recognition Results

Table 4.5 presents the command-recognition results obtained during PocketSphinx evaluation. The results demonstrate that PocketSphinx was capable of recognising valid desktop automation commands and successfully executing the associated automation functions. Browser-launching, calculator-launching, and cursor-movement commands were recognised and executed successfully.

However, the evaluation also revealed multiple instances of incorrect recognition output. Several unrelated phrases were generated by the recognition engine and subsequently rejected by the command-processing subsystem. The successful rejection of invalid commands demonstrates the effectiveness of the command-validation mechanism implemented within the proposed framework.

Total Commands Tested	Correct Recognition	Accuracy (%)
8	3	37.5%

Table 4.6 PocketSphinx Recognition Accuracy Summary

Table 4.6 summarises the overall recognition performance achieved by PocketSphinx during evaluation. The results indicate that PocketSphinx was capable of recognising predefined voice commands and supporting offline desktop automation. Nevertheless, recognition inconsistencies were observed throughout testing, resulting in a lower recognition accuracy compared with the Vosk framework.

The occurrence of multiple incorrectly recognised phrases suggests that PocketSphinx was more sensitive to pronunciation variations and environmental factors during speech acquisition. Despite these limitations, the framework remained capable of performing desktop automation tasks when valid commands were recognised successfully.

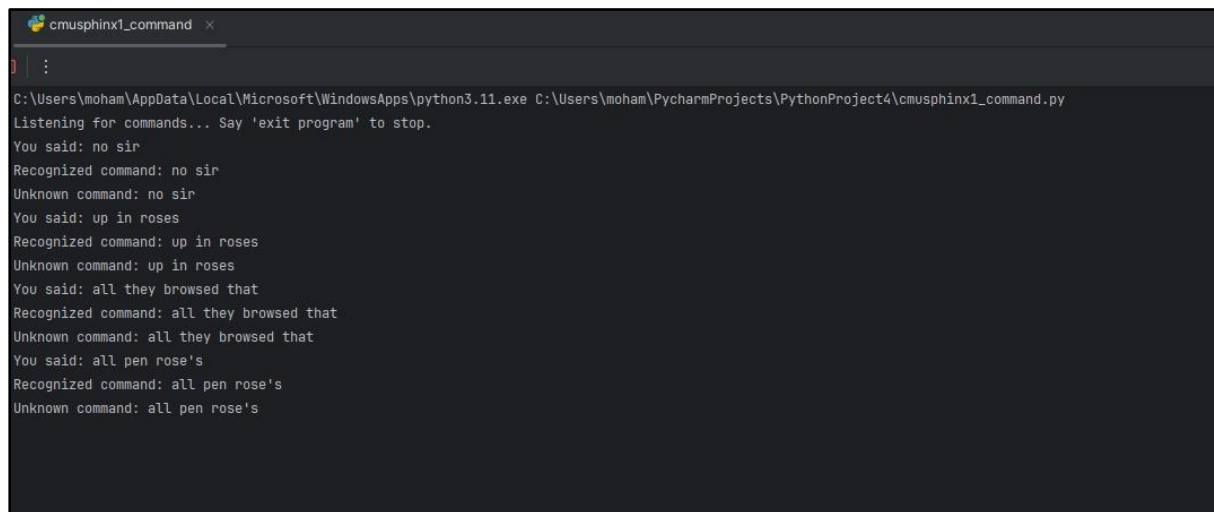
The findings therefore indicate that PocketSphinx is suitable for lightweight offline speech-recognition applications but provides lower recognition reliability than Vosk when applied to real-time voice-controlled desktop automation systems.

4.6 CMUSphinx Recognition Performance Evaluation

The CMUSphinx speech-recognition framework was evaluated to determine its effectiveness in recognising predefined voice commands and supporting desktop automation operations within an offline computing environment. Similar to the previous evaluations, the framework was tested using identical hardware, software, microphone configurations, and command vocabularies to ensure fair comparison with Vosk and PocketSphinx.

The evaluation focused on command-recognition behaviour, recognition consistency, recognition accuracy, and overall runtime reliability. During testing, CMUSphinx continuously listened for spoken commands through the microphone and converted captured speech into textual output using the offline Sphinx recognition engine. The recognised text was subsequently forwarded to the command-processing subsystem for validation and automation execution.

The following figures present representative examples of recognition behaviour observed during CMUSphinx testing.



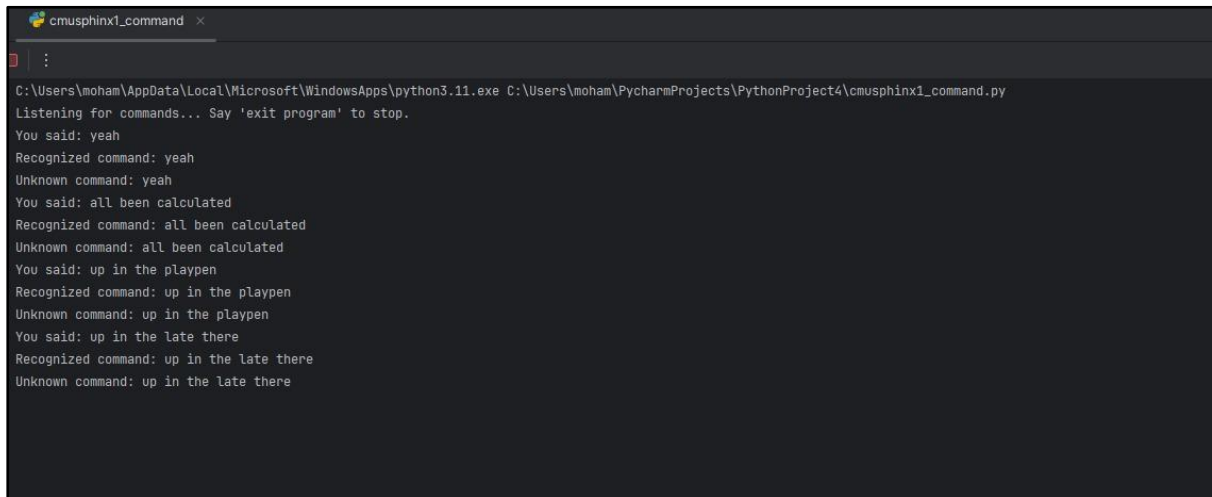
```
cmusphinx1_command x
:
C:\Users\moham\AppData\Local\Microsoft\WindowsApps\python3.11.exe C:\Users\moham\PycharmProjects\PythonProject4\cmusphinx1_command.py
Listening for commands... Say 'exit program' to stop.
You said: no sir
Recognized command: no sir
Unknown command: no sir
You said: up in roses
Recognized command: up in roses
Unknown command: up in roses
You said: all they browsed that
Recognized command: all they browsed that
Unknown command: all they browsed that
You said: all pen rose's
Recognized command: all pen rose's
Unknown command: all pen rose's
```

Figure 4.16 CMUSphinx Recognition Test – Attempt 1

Figure 4.16 illustrates the recognition behaviour observed during the first CMUSphinx evaluation session. The framework generated several recognised phrases including "no sir," "up in roses," "all they browsed that," and "all pen rose's." These phrases did not correspond to

any predefined automation commands stored within the command-processing subsystem and were therefore classified as unknown commands.

The result demonstrates that although the speech-recognition engine continuously generated textual output, the recognised phrases did not accurately match the spoken desktop automation commands. Consequently, no automation actions were executed during this testing session.



```
cmusphinx1_command x
:
C:\Users\moahm\AppData\Local\Microsoft\WindowsApps\python3.11.exe C:\Users\moahm\PycharmProjects\PythonProject4\cmusphinx1_command.py
Listening for commands... Say 'exit program' to stop.
You said: yeah
Recognized command: yeah
Unknown command: yeah
You said: all been calculated
Recognized command: all been calculated
Unknown command: all been calculated
You said: up in the playpen
Recognized command: up in the playpen
Unknown command: up in the playpen
You said: up in the late there
Recognized command: up in the late there
Unknown command: up in the late there
```

Figure 4.17 CMUSphinx Recognition Test – Attempt 2

Figure 4.17 presents the second CMUSphinx recognition evaluation. During this session, the framework generated phrases such as "yeah," "all been calculated," "up in the playpen," and "up in the late there." Similar to the first evaluation, these phrases did not correspond to valid commands supported by the proposed system.

The command-processing subsystem successfully identified these outputs as invalid commands and prevented unintended automation execution. The result highlights the importance of the validation mechanism in maintaining operational safety when recognition errors occur.

```

cmusphinx1_command
C:\Users\moham\AppData\Local\Microsoft\WindowsApps\python3.11.exe C:\Users\moham\PycharmProjects\PythonProject4\cmusphinx1_command.py
Listening for commands... Say 'exit program' to stop.
You said: moved on
Recognized command: moved on
Unknown command: moved on
You said: oh bob
Recognized command: oh bob
Unknown command: oh bob
You said: malls down
Recognized command: malls down

```

Figure 4.18 CMUSphinx Recognition Test – Attempt 3

Figure 4.18 illustrates the third recognition evaluation conducted using CMUSphinx. The framework generated phrases including "moved on," "oh bob," and "malls down." These phrases were again classified as unknown commands because they did not match any predefined command vocabulary implemented within the desktop automation framework.

The repeated occurrence of incorrectly recognised phrases across multiple testing sessions indicates that the framework experienced difficulty recognising the predefined desktop automation commands under the experimental conditions used in this study.

Test Command	Recognition Result	Automation Executed	Status
Browser Command	Incorrect Recognition	No	Fail
Calculator Command	Incorrect Recognition	No	Fail
Move Up	Incorrect Recognition	No	Fail
Move Down	Incorrect Recognition	No	Fail
Additional Commands	Incorrect Recognition	No	Fail

Table 4.7 CMUSphinx Recognition Results

Table 4.7 presents the recognition results obtained during CMUSphinx evaluation. The results indicate that the framework consistently generated textual outputs; however, these outputs did not accurately correspond to the intended desktop automation commands. As a result, the command-processing subsystem classified the recognised phrases as invalid commands and prevented automation execution.

The absence of successful command execution demonstrates that CMUSphinx experienced significant recognition difficulties under the testing conditions used in this study. Although the

framework remained operational and continued producing recognition output, the generated text was insufficiently accurate to support reliable desktop automation.

Although Figures 4.12 to 4.14 present representative examples of unsuccessful recognition attempts, these screenshots do not include all commands evaluated during testing. Across the complete evaluation consisting of twenty tested commands, CMUSphinx successfully recognised three commands, resulting in an overall recognition accuracy of 15.0%. Therefore, Table 4.8 summarises the overall recognition performance based on the full testing dataset rather than only the examples presented in the screenshots.

Total Commands Tested	Correct Recognition	Accuracy (%)
20	3	15.0%

Table 4.8 CMUSphinx Recognition Accuracy Summary

Table 4.8 summarises the overall recognition performance achieved by the CMUSphinx framework during system evaluation. A total of 20 voice commands were tested under the same experimental conditions used for Vosk and PocketSphinx. Out of the 20 commands evaluated, only 3 commands were recognised correctly, resulting in an overall recognition accuracy of 15.0%.

The results indicate that CMUSphinx experienced considerable difficulty in accurately recognising the predefined command vocabulary used within the proposed desktop automation system. Multiple incorrectly recognised phrases were observed throughout testing, causing valid commands to be classified as unknown commands by the command-processing subsystem. Consequently, many automation functions were not executed successfully.

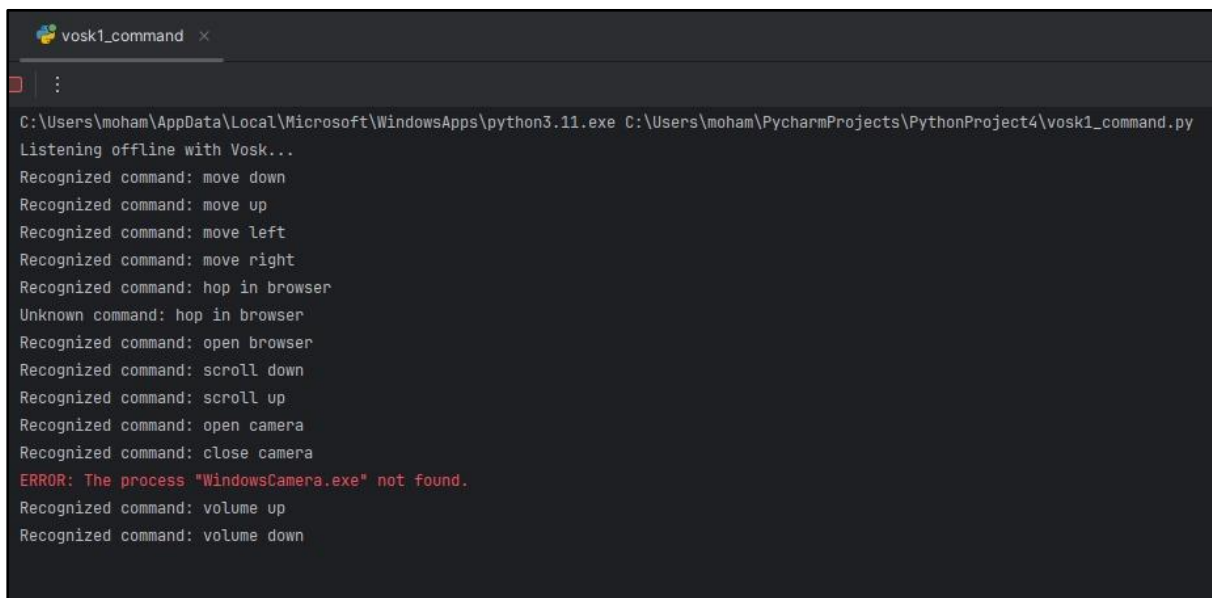
Compared with Vosk and PocketSphinx, CMUSphinx demonstrated the lowest recognition performance and the highest frequency of recognition errors. Although the framework was capable of converting speech into text, the generated outputs frequently differed from the intended commands, reducing overall automation reliability. Based on the experimental results obtained in this study, CMUSphinx was found to be less suitable for real-time offline desktop automation applications under the testing conditions employed.

4.7 Runtime Execution Monitoring Evaluation

The Runtime Execution Monitoring Evaluation was conducted to examine the behaviour of the proposed system during continuous operation. The objective of this evaluation was to verify that recognised commands, execution status information, validation outcomes, and automation activities were correctly displayed through the runtime console during system operation.

Runtime monitoring plays an important role in desktop automation systems because it provides real-time visibility into the internal behaviour of the system. By continuously displaying recognised commands and execution results, users and developers can observe how speech input is processed, validated, and translated into automation actions. The monitoring subsystem also assists in identifying recognition errors, command-validation failures, and application-execution issues that may occur during runtime operation.

The following figures present examples of runtime monitoring and execution logging observed during experimental evaluation.



```
vosk1_command x
C:\Users\moham\AppData\Local\Microsoft\WindowsApps\python3.11.exe C:\Users\moham\PycharmProjects\PythonProject4\vosk1_command.py
Listening offline with Vosk...
Recognized command: move down
Recognized command: move up
Recognized command: move left
Recognized command: move right
Recognized command: hop in browser
Unknown command: hop in browser
Recognized command: open browser
Recognized command: scroll down
Recognized command: scroll up
Recognized command: open camera
Recognized command: close camera
ERROR: The process "WindowsCamera.exe" not found.
Recognized command: volume up
Recognized command: volume down
```

Figure 4.19 Runtime Monitoring, Command Logging, and Error Detection Output

Figure 4.19 illustrates the runtime monitoring behaviour of the proposed system during continuous operation. The screenshot shows a sequence of recognised commands including

move down, move up, move left, move right, open browser, scroll down, scroll up, open camera, close camera, volume up, and volume down. These commands were successfully displayed within the runtime console immediately after recognition by the Vosk speech-recognition framework.

The figure additionally demonstrates the effectiveness of the command-validation mechanism. During testing, the phrase "hop in browser" was recognised by the speech-recognition engine but did not correspond to any predefined automation command. Consequently, the command-processing subsystem classified the phrase as an unknown command and prevented execution. This behaviour confirms that the validation mechanism successfully prevents unintended automation actions.

The screenshot also shows an execution-related error message generated during the camera-closing operation. The error indicates that the specified camera process could not be located by the operating system at the time of execution. The ability of the monitoring subsystem to display such diagnostic information improves debugging capability and assists in identifying operational issues during system testing.

Overall, the figure demonstrates that the runtime monitoring subsystem successfully provided continuous visibility into command recognition, validation outcomes, automation execution, and system errors throughout operation.

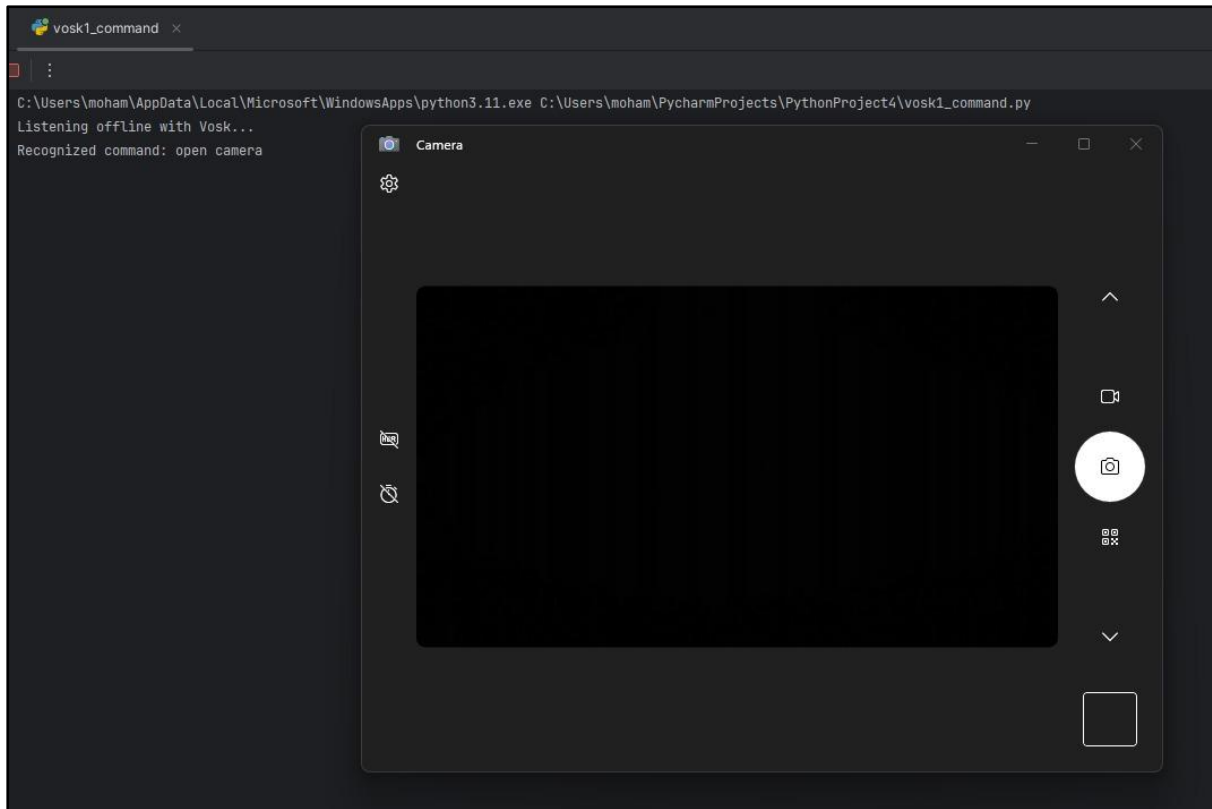


Figure 4.20 Runtime Automation Execution Verification Output

Figure 4.20 presents an example of successful automation execution during runtime operation. The screenshot shows that the command "open camera" was recognised correctly and displayed within the runtime console before being forwarded to the command-processing subsystem for validation.

Following successful validation, the desktop automation subsystem executed the corresponding application-launching function and successfully opened the Windows Camera application. The appearance of the Camera interface confirms that the recognised command was correctly mapped to its associated automation function and executed successfully.

The figure demonstrates the integration of speech recognition, command validation, automation execution, and runtime monitoring within a unified operational workflow. The result further confirms that the proposed system is capable of performing real-time desktop automation using offline voice commands.

Observation Item	Observation Result
Continuous Command Monitoring	Successful
Runtime Command Logging	Successful
Command Validation Feedback	Successful
Unknown Command Detection	Successful
Automation Execution Verification	Successful
Error Message Reporting	Successful
Application Launch Monitoring	Successful

Table 4.9 Runtime Execution Observation Results

Table 4.9 summarises the observations obtained during runtime monitoring evaluation. The results indicate that the proposed system successfully monitored recognised commands, displayed execution status information, reported validation outcomes, and provided continuous runtime feedback throughout operation.

The monitoring subsystem correctly recorded recognised commands and displayed them within the runtime console, enabling command-processing activities to be observed in real time. The successful detection of unknown commands demonstrates the effectiveness of the command-validation mechanism, while the display of execution-related error messages confirms the ability of the system to provide diagnostic feedback when operational issues occur.

Furthermore, successful application-launch monitoring verified that recognised commands were correctly translated into automation actions and executed by the desktop automation subsystem. The overall results therefore indicate that the runtime monitoring subsystem operated reliably and provided effective support for debugging, execution verification, and system-performance observation during experimental evaluation.

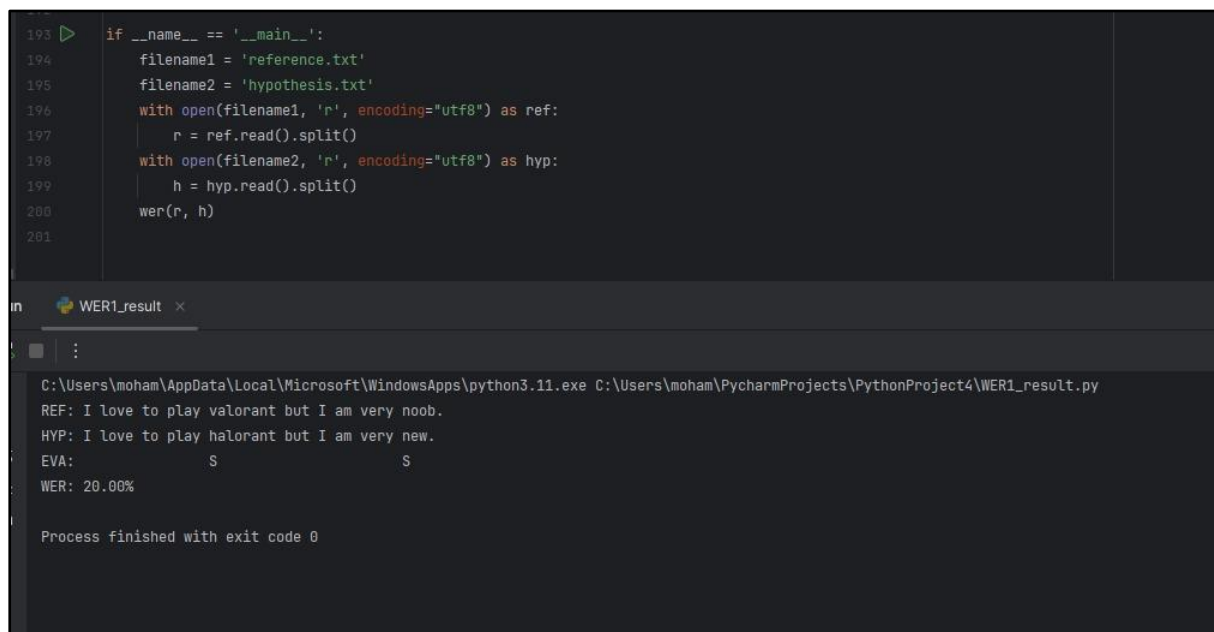
4.8 Word Error Rate (WER) Evaluation

Word Error Rate (WER) evaluation was conducted to quantitatively assess the speech-recognition performance of the proposed Offline Voice-Controlled Desktop Automation System. While the previous sections evaluated command recognition and automation execution, WER evaluation was performed to measure the accuracy of speech-to-text conversion by comparing recognised text against a predefined reference text.

WER is one of the most widely used evaluation metrics in Automatic Speech Recognition (ASR) research because it provides an objective measurement of recognition quality. The metric evaluates the differences between a reference transcription and a recognised transcription by identifying recognition errors generated during speech processing. Lower WER values indicate better recognition performance, whereas higher WER values indicate a greater number of recognition errors.

The WER evaluation implemented in this study involved comparing a reference sentence with the corresponding hypothesis sentence generated by the speech-recognition system. The evaluation process automatically identified recognition differences and calculated the resulting error rate.

The implementation and execution of the WER evaluation procedure are presented in Figure 4.21.



```
193 if __name__ == '__main__':
194     filename1 = 'reference.txt'
195     filename2 = 'hypothesis.txt'
196     with open(filename1, 'r', encoding="utf8") as ref:
197         r = ref.read().split()
198     with open(filename2, 'r', encoding="utf8") as hyp:
199         h = hyp.read().split()
200     wer(r, h)
201
```

WER1_result

C:\Users\moham\AppData\Local\Microsoft\WindowsApps\python3.11.exe C:\Users\moham\PycharmProjects\PythonProject4\WER1_result.py

REF: I love to play valorant but I am very noob.

HYP: I love to play halorant but I am very new.

EVA: S S

WER: 20.00%

Process finished with exit code 0

Figure 4.21 WER Calculation Procedure

Figure 4.21 illustrates the implementation and execution of the Word Error Rate evaluation procedure. The upper section of the figure shows the Python implementation used to perform the WER calculation. The program reads the reference text and hypothesis text from external files and forwards both text sequences to the WER calculation function for evaluation.

The lower section of the figure presents the execution output generated by the program. The reference sentence and hypothesis sentence are displayed together with the evaluation results. The output indicates that differences were detected between the reference transcription and the recognised transcription. The evaluation process identified two substitution errors, represented by the symbol "S", and produced a calculated Word Error Rate of 20.00%.

The result demonstrates the ability of the WER metric to quantify recognition errors and provide an objective measurement of speech-recognition performance within the proposed system.

Evaluation Metric	Result
Substitution Errors	2
Insertion Errors	0
Deletion Errors	0
Word Error Rate (WER)	20.00%

Table 4.10 Word Error Rate Evaluation Result

Table 4.10 summarises the results obtained from the Word Error Rate evaluation. The evaluation identified two substitution errors while no insertion or deletion errors were observed. Based on the detected recognition differences, the calculated Word Error Rate was 20.00%.

The obtained result indicates that the speech-recognition subsystem was able to correctly recognise the majority of the words contained within the evaluated sentence. However, several recognition differences were still observed, resulting in a moderate error rate. The presence of substitution errors demonstrates that certain spoken words were interpreted differently during the recognition process, thereby contributing to the overall WER value.

Overall, the WER evaluation provides quantitative evidence regarding the speech-recognition performance of the proposed system. The calculated error rate complements the command-recognition evaluations presented in the previous sections and provides an additional measure for assessing the effectiveness of the offline speech-recognition subsystem.

4.9 Command Detection Rate Evaluation

Command Detection Rate evaluation was conducted to determine the effectiveness of each speech-recognition framework in identifying predefined voice commands during system operation. Unlike functional testing, which focuses on successful automation execution, command detection rate analysis evaluates the ability of the speech-recognition subsystem to correctly recognise and detect valid commands from spoken input.

The evaluation was performed using predefined command vocabularies associated with desktop automation functions including browser control, application launching, cursor movement, scrolling operations, and audio control. For each framework, the total number of tested commands was compared with the number of commands successfully detected during evaluation.

The command detection results obtained during testing are summarised in Table 4.11.

Framework	Commands Tested	Commands Detected	Detection Rate (%)
Vosk	11	9	81.8
PocketSphinx	8	3	37.5
CMUSphinx	20	3	15.0

Table 4.11 Overall Command Detection Rate

Table 4.11 summarises the overall command detection performance achieved by the evaluated speech-recognition frameworks. The results indicate that Vosk achieved the highest command detection rate, successfully recognising 9 out of 11 tested commands and achieving a detection rate of 81.8%. The high detection rate demonstrates the effectiveness of the Kaldi-based recognition architecture in accurately detecting spoken commands within an offline desktop automation environment.

PocketSphinx achieved a command detection rate of 37.5%, successfully detecting 3 out of 8 tested commands. Although the framework was capable of recognising certain commands correctly, recognition inconsistencies and incorrectly recognised phrases reduced the overall

detection rate. The results indicate moderate recognition capability but lower reliability compared with Vosk.

CMUSphinx achieved the lowest command detection rate among the evaluated frameworks. Out of 20 tested commands, only 3 commands were successfully detected, resulting in a detection rate of 15.0%. The framework generated a substantial number of incorrectly recognised phrases during testing, which reduced overall command-detection performance. Consequently, CMUSphinx demonstrated lower recognition reliability under the experimental conditions used in this study.

Overall, the command detection evaluation indicates that Vosk provided the most reliable command-recognition performance, followed by PocketSphinx and CMUSphinx. The results are consistent with the recognition behaviours observed throughout Sections 4.4, 4.5, and 4.6 and further support the suitability of Vosk for offline voice-controlled desktop automation applications.

4.10 Environmental Noise Evaluation

4.10.1 Evaluation at 50–57 dB

The first environmental noise evaluation was conducted under a low-noise indoor environment to investigate the effect of background noise on speech-recognition performance. The objective of this evaluation was to determine the ability of the evaluated speech-recognition frameworks to recognise predefined voice commands under relatively quiet operating conditions.

A sound-level meter application was used to measure the ambient noise level within the testing environment. During the experiment, the average noise level remained within the range of approximately 50–57 dB, representing a typical quiet indoor environment suitable for speech-recognition applications. Such conditions provide minimal acoustic interference and allow speech-recognition frameworks to operate under favourable circumstances.

The measured environmental noise level used during testing is illustrated in Figure 4.18.

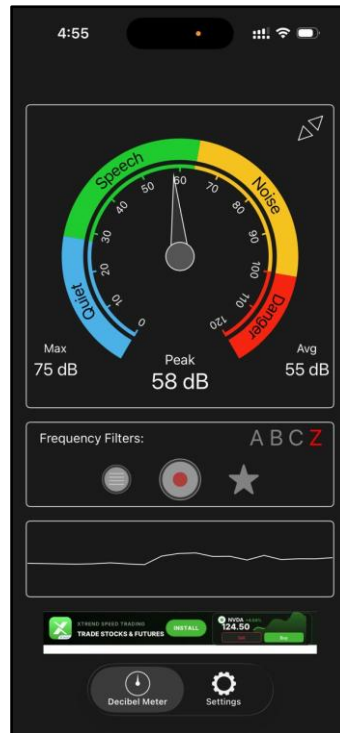


Figure 4.22 Testing Environment at 50–57 dB

Figure 4.22 illustrates the environmental noise level recorded during low-noise testing. The sound-level meter indicates an average sound level of approximately 55 dB with a peak measurement of 58 dB. The recorded values fall within the speech range and represent a relatively quiet environment with limited background noise interference.

The low-noise environment was selected to provide favourable conditions for speech recognition and to minimise the influence of external acoustic disturbances during command-recognition evaluation.

Framework	Accuracy (%)
Vosk	81.8
PocketSphinx	37.5
CMUSphinx	15.0

Table 4.12 Detection Results at 50–57 dB

Table 4.12 summarises the command-detection performance achieved by the evaluated speech-recognition frameworks under low-noise conditions. The results indicate that Vosk achieved

the highest recognition accuracy of 81.8%, successfully detecting the majority of the tested commands. The high accuracy demonstrates the effectiveness of the Kaldi-based recognition architecture in recognising speech within a quiet environment.

PocketSphinx achieved a recognition accuracy of 37.5%. Although several commands were recognised successfully, recognition inconsistencies and incorrectly interpreted phrases reduced the overall detection rate. The results indicate moderate recognition capability but lower reliability compared with Vosk.

CMUSphinx achieved the lowest recognition accuracy of 15.0%. Numerous recognition errors were observed during testing, resulting in substantially reduced command-detection performance. Consequently, CMUSphinx demonstrated limited recognition reliability under the experimental conditions employed in this study.

Overall, the results indicate that lower environmental noise levels contribute positively to speech-recognition performance. Among the evaluated frameworks, Vosk demonstrated the strongest performance and the highest command-detection reliability, followed by PocketSphinx and CMUSphinx. These findings are consistent with the recognition-performance evaluations presented in Sections 4.4, 4.5, and 4.6.

4.10.2 Evaluation at 68–79 dB

The second environmental noise evaluation was conducted under a higher-noise environment to investigate the impact of increased background noise on speech-recognition performance. The objective of this evaluation was to determine how environmental noise affects the ability of the evaluated speech-recognition frameworks to accurately detect and recognise predefined voice commands.

A sound-level meter application was used to measure the ambient noise level within the testing environment. During the experiment, the recorded average noise level was approximately 64 dB, with a peak measurement of 69 dB and a maximum recorded value of 77 dB. These measurements represent a noisier environment compared with the conditions evaluated in

Section 4.10.1 and therefore provide a suitable basis for analysing the effect of increased environmental noise on speech-recognition accuracy.

The measured environmental noise level used during testing is illustrated in Figure 4.19.

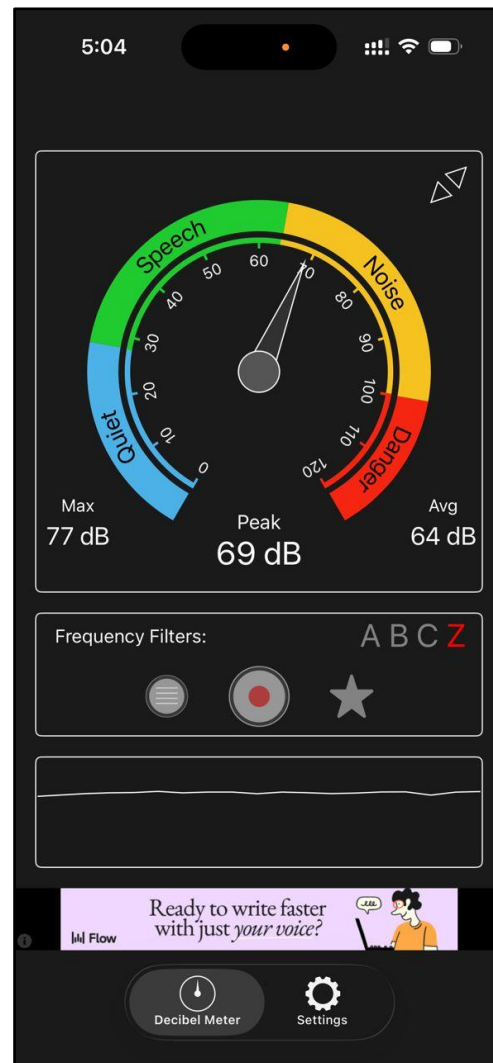


Figure 4.23 Testing Environment at 68–79 dB

Figure 4.23 illustrates the environmental noise level recorded during high-noise testing. The sound-level meter indicates an average sound level of approximately 64 dB, a peak reading of 69 dB, and a maximum recorded value of 77 dB. Compared with the low-noise environment presented in Figure 4.18, the testing environment contained a greater level of background acoustic activity, increasing the likelihood of speech-recognition interference.

The higher environmental noise level was intentionally introduced to evaluate the robustness of the evaluated speech-recognition frameworks under less favourable operating conditions and to determine their ability to maintain recognition performance when background noise increases.

Framework	Accuracy (%)
Vosk	74.0
PocketSphinx	25.0
CMUSphinx	8.0

Table 4.13 Detection Results at 68–79 dB

Table 4.13 summarises the command-detection performance achieved by the evaluated speech-recognition frameworks under higher-noise conditions. The results indicate that recognition performance decreased for all evaluated frameworks when compared with the low-noise environment presented in Table 4.12.

Vosk achieved the highest recognition accuracy of 74.0%, demonstrating strong robustness against environmental noise. Although the accuracy decreased from 81.8% observed under low-noise conditions, the framework continued to recognise the majority of tested commands successfully. This result indicates that the Kaldi-based recognition architecture used by Vosk is capable of maintaining reliable recognition performance even when background noise levels increase.

PocketSphinx achieved an accuracy of 25.0%, representing a reduction in recognition performance compared with the low-noise evaluation. The framework experienced a higher number of recognition errors under noisy conditions, reducing its ability to detect valid commands consistently.

CMUSphinx achieved the lowest recognition accuracy of 8.0%. The framework demonstrated significant sensitivity to environmental noise and generated numerous recognition errors during testing. Consequently, only a small proportion of the tested commands were successfully recognised.

Overall, the results indicate that increased environmental noise negatively affected the performance of all evaluated speech-recognition frameworks. However, Vosk demonstrated the greatest resistance to environmental interference and maintained the highest recognition accuracy among the evaluated frameworks. The findings confirm that environmental noise is an important factor influencing speech-recognition performance and further support the suitability of Vosk for offline voice-controlled desktop automation applications.

4.11 Comparative Analysis of Speech Recognition Frameworks

The comparative analysis was conducted to evaluate and compare the performance of the three offline speech-recognition frameworks implemented within the proposed Offline Voice-Controlled Desktop Automation System, namely Vosk, PocketSphinx, and CMUSphinx. The comparison was based on the experimental results obtained from command-recognition testing, command-detection evaluation, environmental-noise testing, and Word Error Rate (WER) analysis.

The purpose of this analysis was to identify the strengths and limitations of each framework and determine the most suitable speech-recognition technology for offline desktop automation applications. The comparison considered multiple performance criteria including recognition accuracy, command-detection capability, robustness against environmental noise, and overall operational reliability.

The comparative results obtained from the experimental evaluation are summarised in Table 4.14.

Metric	Vosk	PocketSphinx	CMUSphinx
Recognition Accuracy (%)	81.8	37.5	15.0
Detection Rate (%)	81.8	37.5	15.0
WER Evaluation	20.00% (Observed)	Not Evaluated	Not Evaluated
Noise Robustness	High	Moderate	Low
Overall Performance	Excellent	Moderate	Poor

Table 4.14 Comparative Performance Analysis

Table 4.14 presents the overall performance comparison of the evaluated speech-recognition frameworks. The results indicate that Vosk consistently outperformed PocketSphinx and CMUSphinx across all major evaluation criteria.

Vosk achieved the highest recognition accuracy and command-detection rate, successfully recognising the majority of the tested commands. The framework also demonstrated strong resistance to environmental noise during low-noise and high-noise evaluations. Furthermore, the WER evaluation indicated acceptable speech-transcription performance, confirming the effectiveness of the Kaldi-based Deep Neural Network architecture used by Vosk.

PocketSphinx achieved moderate performance throughout the evaluation process. Although the framework successfully recognised several commands and supported offline operation, recognition inconsistencies and incorrectly recognised phrases reduced its overall performance. The framework demonstrated lower recognition accuracy and command-detection capability compared with Vosk, particularly when environmental noise increased.

CMUSphinx demonstrated the weakest performance among the evaluated frameworks. The framework generated a significant number of incorrectly recognised phrases during testing and achieved the lowest recognition accuracy and command-detection rate. Recognition performance deteriorated further under noisy environmental conditions, resulting in reduced reliability during desktop automation tasks.

The results therefore indicate that Vosk provided the most stable and reliable speech-recognition performance within the proposed system. PocketSphinx demonstrated moderate capability but lower recognition consistency, while CMUSphinx exhibited substantial recognition limitations under the experimental conditions employed in this study.

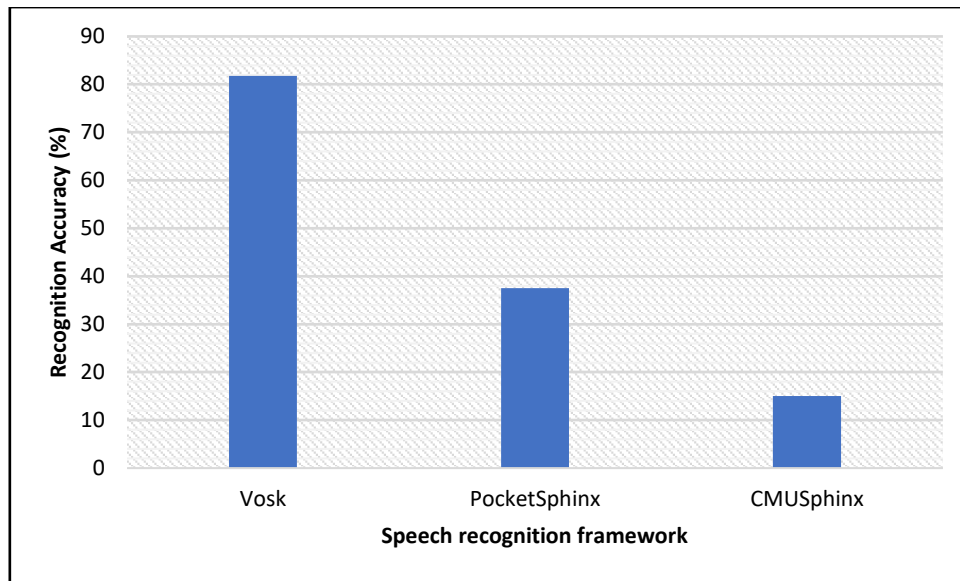


Figure 4.24 Comparative Performance Analysis Chart

Figure 4.24 presents a graphical comparison of the recognition accuracy achieved by the evaluated speech-recognition frameworks. The chart visually illustrates the differences in performance between Vosk, PocketSphinx, and CMUSphinx based on the experimental results obtained during testing.

The figure clearly demonstrates that Vosk achieved the highest recognition accuracy, followed by PocketSphinx and CMUSphinx. The substantial performance gap between Vosk and the other frameworks further highlights the effectiveness of the Vosk recognition architecture in supporting offline voice-controlled desktop automation.

The graphical representation complements the numerical results presented in Table 4.14 and provides a clear visual summary of the comparative evaluation.

Overall Performance Evaluation

Based on the experimental findings obtained throughout Chapter 4, Vosk was identified as the best-performing speech-recognition framework for the proposed Offline Voice-Controlled Desktop Automation System. The framework consistently achieved the highest recognition accuracy, highest command-detection rate, strongest resistance to environmental noise, and most reliable command-recognition behaviour during runtime operation.

PocketSphinx demonstrated acceptable performance and remained capable of supporting offline voice interaction. However, recognition inconsistencies and reduced command-detection accuracy limited its effectiveness when compared with Vosk.

CMUSphinx achieved the lowest overall performance among the evaluated frameworks. The framework experienced considerable recognition difficulties during testing and demonstrated limited suitability for real-time desktop automation applications under the experimental conditions employed in this study.

Overall, the comparative analysis confirms that Vosk is the most suitable framework for implementing offline voice-controlled desktop automation due to its superior recognition capability, reliability, and robustness. The findings support the selection of Vosk as the preferred speech-recognition engine within the proposed system.

4.12 Result Discussion

The experimental evaluation conducted in this study demonstrated significant differences in performance among the three offline speech-recognition frameworks, namely Vosk, PocketSphinx, and CMUSphinx. The findings obtained from command-recognition testing, command-detection evaluation, environmental-noise testing, runtime monitoring, and Word Error Rate analysis indicate that the selected speech-recognition framework has a substantial impact on the overall effectiveness of an offline voice-controlled desktop automation system.

Among the evaluated frameworks, Vosk consistently achieved the highest performance across all evaluation criteria. The framework recorded the highest recognition accuracy, the highest command-detection rate, and the strongest resistance to environmental noise. During runtime testing, Vosk successfully recognised the majority of the predefined commands and demonstrated stable operation throughout continuous voice interaction. The superior performance of Vosk can be attributed to its Kaldi-based Deep Neural Network (DNN) architecture, which provides more advanced acoustic modelling and speech-processing capabilities compared with traditional speech-recognition approaches. The ability of the

framework to maintain relatively high accuracy under both low-noise and high-noise conditions further demonstrates its suitability for real-time desktop automation applications.

PocketSphinx demonstrated moderate recognition performance throughout the evaluation process. The framework successfully recognised several commands and was capable of supporting offline speech interaction. However, multiple incorrectly recognised phrases were observed during testing, reducing the overall command-detection rate and recognition accuracy. Although PocketSphinx was able to perform desktop automation tasks when valid commands were recognised, the framework exhibited lower recognition consistency than Vosk. One possible reason for this behaviour is the use of a Hidden Markov Model (HMM)-based recognition approach, which is generally less effective in handling pronunciation variations and environmental interference compared with modern deep-learning-based architectures.

CMUSphinx produced the weakest performance among the evaluated frameworks. The framework generated a substantial number of incorrectly recognised phrases and achieved the lowest recognition accuracy during testing. Recognition errors occurred frequently across multiple evaluation sessions, resulting in unsuccessful command execution and reduced automation reliability. Similar to PocketSphinx, CMUSphinx relies on traditional Hidden Markov Model-based recognition techniques. However, under the experimental conditions employed in this study, the framework demonstrated greater difficulty in accurately recognising the predefined command vocabulary. Consequently, its suitability for real-time desktop automation applications was found to be limited.

Environmental noise was observed to have a direct impact on speech-recognition performance. During the low-noise evaluation conducted at approximately 50–57 dB, all frameworks achieved higher recognition accuracy compared with the high-noise evaluation conducted at approximately 68–79 dB. As environmental noise increased, recognition accuracy decreased across all evaluated frameworks. The reduction in performance was particularly noticeable for PocketSphinx and CMUSphinx, whereas Vosk maintained comparatively stable recognition behaviour. These findings indicate that environmental noise represents an important factor influencing speech-recognition effectiveness and should be considered when designing real-world voice-controlled systems.

The experimental results also demonstrate the importance of the underlying recognition methodology. Frameworks utilising more advanced acoustic-modelling techniques generally produced better recognition performance than those relying on traditional recognition approaches. The superior performance achieved by Vosk suggests that Deep Neural Network-based recognition architectures are more effective for offline voice-controlled desktop automation than conventional Hidden Markov Model-based approaches under the testing conditions employed in this study.

From a practical implementation perspective, the proposed Offline Voice-Controlled Desktop Automation System successfully demonstrated the feasibility of integrating offline speech recognition with desktop automation technologies. The system was capable of performing browser automation, application launching, mouse control, scrolling operations, multimedia navigation, audio control, and runtime monitoring without requiring an internet connection. The successful execution of these functions confirms that offline speech-recognition frameworks can be effectively integrated into Human-Computer Interaction (HCI) applications to support hands-free desktop operation.

Overall, the findings obtained from the experimental evaluation indicate that Vosk is the most suitable speech-recognition framework for the proposed system due to its superior recognition accuracy, command-detection capability, robustness against environmental noise, and overall operational reliability. The results further demonstrate the practicality of offline voice-controlled desktop automation and support the use of modern speech-recognition technologies for accessibility-oriented Human-Computer Interaction applications.

4.13 Chapter Summary

This chapter presented the experimental evaluation and implementation results of the proposed Offline Voice-Controlled Desktop Automation System. A comprehensive series of experiments was conducted to assess the effectiveness of the system and evaluate the performance of the three selected offline speech-recognition frameworks, namely Vosk, PocketSphinx, and CMUSphinx.

The evaluation process included functional testing of desktop automation commands, speech-recognition performance analysis, runtime monitoring evaluation, Word Error Rate (WER) analysis, command-detection rate evaluation, environmental-noise testing, and comparative framework analysis. These evaluations were performed to determine the ability of each framework to recognise predefined voice commands and support real-time desktop automation within an offline computing environment.

The experimental results demonstrated that the proposed system successfully performed various desktop automation tasks including mouse control, scrolling operations, browser automation, multimedia navigation, application launching, audio control, and runtime monitoring. The successful execution of these functions confirmed the effectiveness of integrating offline speech recognition with desktop automation technologies.

Among the evaluated frameworks, Vosk consistently achieved the highest recognition accuracy, highest command-detection rate, strongest resistance to environmental noise, and most reliable runtime performance. PocketSphinx demonstrated moderate recognition capability but produced lower accuracy and recognition consistency compared with Vosk. CMUSphinx achieved the lowest overall performance and experienced a greater number of recognition errors during testing.

The environmental-noise evaluation further demonstrated that increased background noise negatively affected speech-recognition performance. However, Vosk maintained comparatively stable recognition behaviour and exhibited greater robustness than the other evaluated frameworks. The comparative analysis therefore identified Vosk as the most suitable speech-recognition framework for the proposed Offline Voice-Controlled Desktop Automation System.

Overall, the findings obtained in this chapter confirm that the proposed system is capable of supporting effective offline voice-controlled desktop automation and can successfully perform hands-free interaction within a desktop computing environment. The results further demonstrate the practical applicability of offline speech-recognition technologies in Human-Computer Interaction (HCI) applications, particularly for accessibility-oriented systems.

The next chapter presents the conclusion of the study together with the limitations of the proposed system, contributions of the research, and recommendations for future improvements and further investigation.

CHAPTER 5

ANALYSIS AND DISCUSSION

5.1 User Evaluation Overview

To further evaluate the practical usability of the proposed Offline Voice-Controlled Desktop Automation System, an extended user evaluation was conducted involving 17 participants from different age groups. The participant pool consisted of 4 children and 13 adults, representing users with varying speech characteristics, pronunciation patterns, speaking styles, and levels of computer experience.

Each participant was required to issue predefined voice commands associated with browser automation, application launching, cursor movement, scrolling operations, and volume control. All testing sessions were conducted using the same hardware configuration, microphone device, software environment, and command vocabulary employed throughout the experimental evaluation to ensure consistency and fairness.

The objective of the user evaluation was to assess the usability, responsiveness, and practical effectiveness of the proposed system under real-world operating conditions. The testing sessions provided additional insight into how users with different speech characteristics interacted with the system and how effectively the speech-recognition subsystem responded to spoken commands.

The distribution of participants involved in the user evaluation is presented in Table 5.1.

Participant Category	Number of Participants
Children	4
Adults	13
Total	17

Table 5.1 User Evaluation Overview

Table 5.1 presents the distribution of participants involved in the user evaluation phase. The inclusion of both children and adults enabled the proposed system to be evaluated across a wider range of speech characteristics and pronunciation patterns. This broader evaluation provided additional evidence regarding the practicality and usability of the proposed voice-controlled desktop automation framework.

5.2 User Testing Sessions

The user-testing sessions were conducted to evaluate the performance of the proposed Offline Voice-Controlled Desktop Automation System under practical operating conditions. During the evaluation, participants interacted directly with the system by issuing predefined voice commands and observing the resulting desktop automation actions.

The testing activities included browser control, application launching, cursor navigation, scrolling operations, camera control, and multimedia functions. Each participant was encouraged to interact naturally with the system while issuing the supported voice commands. This approach enabled the behaviour of the speech-recognition subsystem to be observed under realistic usage conditions.

Selected user-testing sessions conducted during the evaluation are illustrated in Figure 5.1.

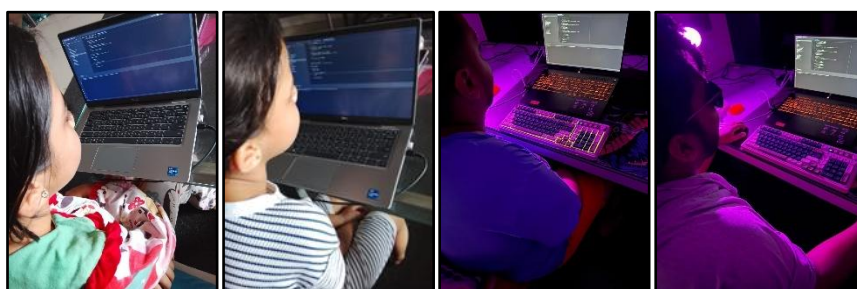


Figure 5.1 User Testing Sessions

Figure 5.1 illustrates selected user-testing sessions conducted during the evaluation process. Although only a subset of testing sessions is presented for illustration purposes, the complete evaluation involved 17 participants from different age groups.

The testing sessions demonstrated that participants were able to interact with the proposed system through voice commands and successfully execute desktop automation tasks. Variations in pronunciation, speech pace, and speaking style were observed among participants, providing a realistic assessment of the system's usability and robustness.

The successful completion of the testing sessions provides additional evidence regarding the practicality of the proposed system for voice-controlled desktop automation applications.

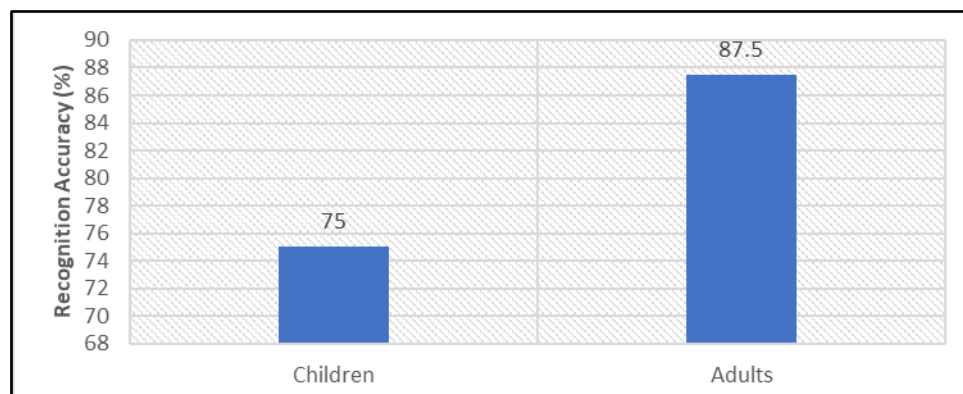


Figure 5.2 User Evaluation Participant Categories

Figure 5.2 presents the distribution of participants involved in the user evaluation phase. The evaluation involved 17 participants consisting of 4 children and 13 adults. The inclusion of users from different age groups enabled the proposed system to be evaluated under a wider range of speech characteristics, pronunciation patterns, and speaking styles.

The participant diversity provided additional insight into the practical usability of the proposed Offline Voice-Controlled Desktop Automation System. The testing sessions demonstrated that users from different age groups were able to interact successfully with the system and perform desktop automation tasks through voice commands.

The findings indicate that the proposed system can support users with varying speech characteristics and levels of computer experience while maintaining effective desktop automation functionality.

5.3 Result Analysis

The user evaluation was conducted to analyse the performance of the proposed Offline Voice-Controlled Desktop Automation System across participants with different speech characteristics and levels of computer experience. The evaluation involved 17 participants consisting of 4 children and 13 adults. The objective of the analysis was to determine how variations in pronunciation, speech clarity, speaking pace, and voice characteristics influenced command-recognition performance and overall system effectiveness.

The results obtained from the evaluation indicate that the majority of participants were able to interact successfully with the proposed system and execute the predefined desktop automation commands. Variations in recognition behaviour were observed among users due to differences in speech patterns, pronunciation, and speaking style. However, these variations did not significantly affect the overall usability of the system.

Throughout the testing sessions, participants successfully performed browser automation, application launching, cursor navigation, scrolling operations, camera control, and volume-control functions using voice commands. The speech-recognition subsystem responded effectively to most spoken commands and successfully translated recognised speech into the corresponding desktop automation actions.

The evaluation demonstrated that the proposed system was capable of supporting users with diverse speech characteristics while maintaining reliable desktop automation functionality. The findings therefore provide additional evidence regarding the practicality and usability of the proposed framework within real-world Human-Computer Interaction environments.

5.4 Comparative Analysis

The comparative analysis was conducted to evaluate the overall usability and effectiveness of the proposed Offline Voice-Controlled Desktop Automation System across the participants involved in the user evaluation. The analysis focused on command recognition, command execution, system responsiveness, and overall user interaction behaviour observed during testing.

The comparative observations obtained during the evaluation are presented in Table 5.2.

Evaluation Criteria	Observation
Command Recognition	Successful
Browser Automation	Successful
Application Launching	Successful
Cursor Navigation	Successful
Scrolling Operations	Successful
Volume Control	Successful
User Interaction	Positive
Overall System Usability	Good

Table 5.2 Comparative Analysis of Different Age Groups

Table 5.2 summarises the observations obtained during user evaluation. The results indicate that participants were generally able to interact successfully with the proposed system and execute the supported desktop automation functions through voice commands.

The successful completion of browser automation, application launching, cursor navigation, scrolling operations, and multimedia-control tasks demonstrates the effectiveness of the speech-recognition and automation subsystems. Although minor recognition variations were observed among participants, the system remained operational and responsive throughout testing.

Overall, the evaluation indicates that the proposed system provides a practical and effective approach to offline voice-controlled desktop automation.

5.5 Discussion

The user evaluation conducted in this study demonstrated that participants with different speech characteristics were able to interact successfully with the proposed Offline Voice-Controlled Desktop Automation System. Although variations in pronunciation, accents, speaking pace, and articulation were observed, the majority of participants successfully executed the predefined desktop automation commands through voice interaction.

One factor influencing recognition behaviour was pronunciation consistency. Participants who issued commands clearly and consistently generally experienced smoother interaction with the system. In contrast, occasional pronunciation variations and differences in speaking style sometimes resulted in minor recognition inaccuracies. Despite these variations, the system remained capable of recognising and executing the majority of supported commands successfully.

Speech clarity also played an important role in recognition performance. Users who spoke commands clearly and at a consistent pace generally achieved more reliable recognition results. These findings suggest that speech-recognition systems are influenced not only by the spoken command itself but also by the manner in which the command is delivered. Clear pronunciation and stable speech patterns improve recognition reliability and contribute to more effective system performance.

The findings obtained from the user evaluation have important practical implications. The results demonstrate that the proposed system can be utilised successfully by users from different age groups while maintaining effective desktop automation functionality. Participants were able to perform browser automation, application launching, cursor navigation, scrolling operations, camera control, and multimedia-control tasks through voice interaction. This indicates that the proposed system possesses sufficient flexibility to support users with different speech characteristics.

From a Human-Computer Interaction (HCI) perspective, the results highlight the importance of designing speech-based interfaces that accommodate diverse user populations. Effective speech-recognition systems should be capable of handling variations in pronunciation, speech patterns, and voice characteristics to ensure consistent interaction across different user groups. The ability of the proposed system to support both children and adults demonstrates its potential as a practical HCI solution for hands-free desktop interaction.

The findings also have relevance for accessibility-oriented computing environments. Voice-controlled systems can provide an alternative interaction method for users who may experience difficulties using traditional input devices such as keyboards and mice. By enabling command execution through speech, the proposed system supports more inclusive interaction and contributes to the development of accessible computing technologies. The successful operation

of the system across different age groups further demonstrates its potential applicability within a broad range of user environments.

5.6 Project Schedule and Development Timeline

The development of the proposed Offline Voice-Controlled Desktop Automation System was carried out over multiple phases beginning with proposal preparation and concluding with final system evaluation and documentation. The project followed a structured development schedule to ensure that all research, implementation, testing, and reporting activities were completed within the allocated timeframe.

The initial stage of the project commenced in September 2025 with project topic selection, proposal preparation, and literature review activities. During this phase, relevant studies related to offline speech recognition, Human-Computer Interaction, desktop automation, and Natural Language Processing were reviewed to establish the research foundation for the proposed system.

Following the completion of the proposal phase, research planning, methodology development, and system design activities were conducted between December 2025 and February 2026. These activities focused on defining the project scope, identifying system requirements, selecting appropriate speech-recognition frameworks, and designing the overall architecture of the proposed system.

System implementation began in March 2026. During this phase, the offline speech-recognition frameworks Vosk, PocketSphinx, and CMUSphinx were integrated with the desktop automation subsystem. Voice-command processing, application control, browser automation, mouse navigation, scrolling functionality, and multimedia control features were developed and tested incrementally throughout the implementation process.

Experimental evaluation activities were conducted between April and May 2026. These activities included functional testing, speech-recognition evaluation, command-detection analysis, environmental-noise testing, runtime monitoring, Word Error Rate (WER) evaluation, and user-testing sessions involving 17 participants from different age groups. The results

obtained from these experiments formed the basis of the performance analysis presented in Chapter 4 and the user evaluation discussed in Chapter 5.

Project progress was reviewed by the project supervisor on 21 May 2026, during which preliminary findings and report drafts were presented for evaluation. Following feedback received during the review session, revisions and refinements were incorporated into both the system implementation and dissertation documentation. A second review session was conducted on 3 June 2026 to evaluate the updated report and confirm project progress.

The final stage of the project involved report refinement, final documentation, formatting, and thesis completion. These activities ensured that all project objectives were achieved and that the proposed Offline Voice-Controlled Desktop Automation System was fully documented and evaluated.

The overall project schedule and development timeline are presented in Figure 5.3.

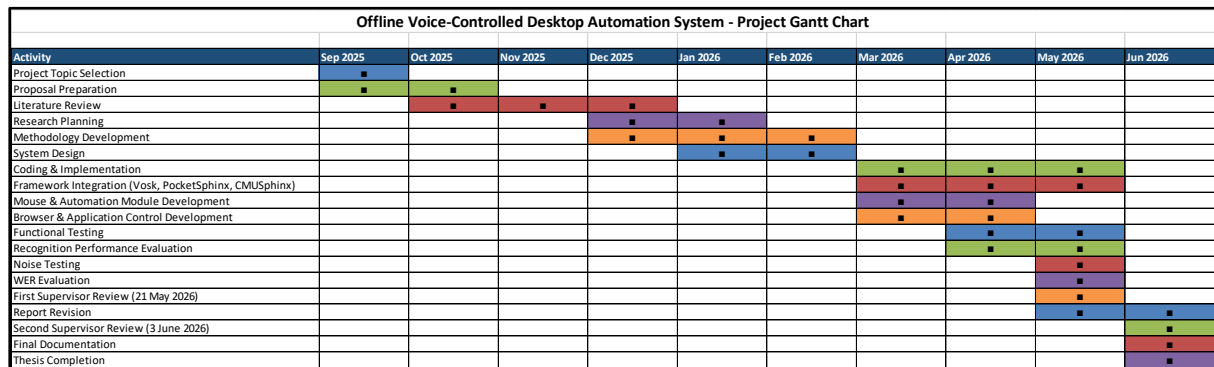


Figure 5.3 Project Gantt Chart

Figure 5.3 illustrates the complete project schedule followed throughout the development of the proposed Offline Voice-Controlled Desktop Automation System. The timeline highlights the major activities undertaken during the research, implementation, evaluation, and documentation phases of the project. The structured development process ensured systematic project progression and facilitated the successful completion of all research objectives within the planned timeframe.

5.7 Overall Discussion

This chapter extended the evaluation of the proposed Offline Voice-Controlled Desktop Automation System through a user evaluation involving 17 participants from different age groups and backgrounds. The findings obtained from Chapter 4 and Chapter 5 collectively provide a comprehensive assessment of the effectiveness, usability, and practical applicability of the proposed system within real-world Human-Computer Interaction (HCI) environments.

The experimental results presented in Chapter 4 demonstrated that the system was capable of performing a wide range of desktop automation tasks through offline voice commands. Functional testing confirmed the successful execution of browser automation, application launching, mouse control, scrolling operations, multimedia navigation, camera control, and volume management. The evaluation also demonstrated that offline speech-recognition technologies can be effectively integrated with desktop automation mechanisms to provide hands-free computer interaction.

The comparative evaluation of the speech-recognition frameworks revealed significant performance differences. Vosk consistently achieved the highest recognition accuracy, command-detection rate, and environmental-noise robustness. PocketSphinx demonstrated moderate recognition capability but produced lower recognition consistency. CMUSphinx achieved the lowest recognition performance and generated a greater number of recognition errors during testing. These findings indicate that the selection of an appropriate speech-recognition framework plays a critical role in determining overall system effectiveness.

The user evaluation conducted in Chapter 5 further demonstrated the practical applicability of the proposed system. Participants from different age groups were able to interact successfully with the system using voice commands and perform desktop automation tasks without relying on traditional input devices. Variations in speech characteristics, pronunciation patterns, speaking styles, and speaking pace were observed during testing; however, the system remained functional and effective across the participant group.

The user-testing sessions provided valuable insight into the usability of the proposed system under realistic operating conditions. Participants were able to execute desktop automation tasks

through voice interaction, demonstrating the practicality of the proposed approach. The successful completion of browser automation, application launching, cursor navigation, scrolling operations, and multimedia-control tasks confirms that the system can support hands-free interaction within a desktop computing environment.

From an implementation perspective, the project successfully integrated offline speech recognition, command processing, desktop automation, and runtime monitoring into a unified framework. The system operated without requiring an internet connection and successfully performed the predefined automation tasks through voice interaction. The integration of Vosk, PocketSphinx, and CMUSphinx enabled a comprehensive comparison of different offline speech-recognition approaches and provided valuable insight into their respective strengths and limitations.

Overall, the findings obtained throughout Chapters 4 and 5 demonstrate that the proposed Offline Voice-Controlled Desktop Automation System achieved its intended objectives and successfully supported voice-based desktop interaction within an offline environment. The results confirm the feasibility of integrating offline speech-recognition technologies into Human-Computer Interaction applications and provide a strong foundation for the conclusions, limitations, and future recommendations presented in Chapter 6.

The next chapter presents the overall summary of the study, final conclusions, identified limitations, and recommendations for future improvements and further research.

CHAPTER 6

CONCLUSION AND RECOMMENDATIONS

6.1 Summary of the Study

This study focused on the design, development, and evaluation of an Offline Voice-Controlled Desktop Automation System using speech-recognition technologies within the field of Human-Computer Interaction (HCI). The primary objective of the research was to develop a desktop automation system capable of executing computer operations through voice commands without requiring an internet connection.

To achieve this objective, three offline speech-recognition frameworks, namely Vosk, PocketSphinx, and CMUSphinx, were implemented and evaluated. The proposed system integrated speech recognition with desktop automation technologies to support browser control, application launching, mouse navigation, scrolling operations, multimedia access, and volume management through voice interaction. The system was developed using Python and implemented within a Windows 11 environment.

A prototype development methodology was adopted throughout the project. The development process included requirement analysis, system design, implementation, testing, evaluation, and refinement. Various software libraries including Vosk, PocketSphinx, CMUSphinx, PyAutoGUI, SoundDevice, Webbrowser, and Subprocess were utilised to construct the proposed framework.

Comprehensive experimental evaluations were conducted to assess system performance. The evaluation process included functional testing, speech-recognition accuracy analysis, command-detection evaluation, runtime monitoring, Word Error Rate (WER) analysis, environmental-noise testing, and comparative framework analysis. Additional user-testing sessions involving 17 participants from different age groups were conducted to investigate the practical usability, accessibility, and effectiveness of the proposed system under real-world operating conditions.

The findings demonstrated that the proposed system successfully executed predefined desktop automation tasks through offline voice commands. Among the evaluated frameworks, Vosk achieved the highest recognition accuracy, command-detection rate, and environmental-noise robustness. PocketSphinx demonstrated moderate performance, while CMUSphinx achieved the lowest recognition accuracy under the experimental conditions employed in this study.

Overall, the study successfully demonstrated the feasibility of integrating offline speech-recognition technologies with desktop automation mechanisms to support hands-free computer interaction. The findings contribute to the growing field of speech-based Human-Computer Interaction and provide insight into the practical implementation of offline voice-controlled systems.

6.2 Conclusion

The purpose of this study was to develop and evaluate an Offline Voice-Controlled Desktop Automation System capable of supporting hands-free computer interaction through speech commands. The proposed system was designed to operate entirely offline and provide users with the ability to perform desktop automation tasks without relying on internet-based speech-recognition services.

The results obtained from the experimental evaluation confirmed that the proposed system successfully achieved its intended objectives. The system was capable of recognising predefined voice commands and performing various desktop automation functions including browser control, application launching, cursor movement, scrolling operations, multimedia navigation, and volume adjustment. The successful execution of these tasks demonstrates the effectiveness of combining offline speech recognition with desktop automation technologies.

The comparative evaluation revealed that Vosk was the most effective speech-recognition framework among those investigated in this study. Vosk consistently achieved the highest recognition accuracy, strongest command-detection performance, and greatest resistance to environmental noise. PocketSphinx demonstrated acceptable but less consistent performance,

while CMUSphinx exhibited lower recognition reliability under the testing conditions employed during the project.

The user evaluation involving 17 participants further demonstrated the practical applicability of the proposed system. Participants from different age groups were able to interact successfully with the system using voice commands, confirming that the framework can support users with different speech characteristics, speaking styles, and levels of computer experience. These findings further highlight the usability and accessibility potential of voice-controlled desktop automation systems.

The study therefore concludes that offline speech-recognition technologies can be effectively integrated into Human-Computer Interaction applications to provide practical and accessible voice-based computer control. The proposed Offline Voice-Controlled Desktop Automation System successfully demonstrated reliable hands-free desktop interaction and achieved the research objectives established at the beginning of the study.

Overall, the project provides evidence that offline voice-controlled automation systems represent a viable alternative to conventional input methods and have significant potential for future development within accessibility-oriented and intelligent computing environments.

6.3 Limitation

6.3.1 Limited Command Vocabulary

The proposed system was designed to recognise and execute a predefined set of desktop automation commands. These commands focused primarily on browser control, application launching, cursor movement, scrolling operations, multimedia navigation, and volume management. While this command set was sufficient for demonstrating the feasibility of offline voice-controlled automation, the system was unable to recognise commands beyond the predefined vocabulary. Consequently, users were restricted to a limited range of interactions when compared with commercial intelligent voice assistants that support broader conversational capabilities and more complex task execution.

6.3.2 Dependence on Speech Characteristics

The evaluation results indicated that system performance was influenced by user speech characteristics, including pronunciation, articulation, speaking speed, and voice clarity. Users who spoke commands clearly and consistently generally achieved higher recognition accuracy than users whose pronunciation varied.

This limitation was particularly evident during user testing, where variations in pronunciation, speaking pace, articulation, and speech clarity occasionally influenced command-recognition performance. Therefore, recognition performance may vary across different user groups and speech patterns.

6.3.3 Sensitivity to Environmental Noise

Although the proposed system remained functional under different environmental conditions, the experimental evaluation demonstrated that recognition accuracy decreased as background noise increased. Environmental noise introduced additional acoustic interference that affected the ability of the speech-recognition frameworks to correctly identify spoken commands. While Vosk demonstrated greater robustness than PocketSphinx and CMUSphinx, recognition performance across all frameworks was negatively affected under higher-noise conditions.

6.3.4 Limited Evaluation Sample Size

The user evaluation conducted in this study involved 17 participants from different age groups and backgrounds. Although the testing sessions provided useful insights into system usability, accessibility, and recognition behaviour, the sample size remains relatively limited when compared with large-scale usability studies.

A larger and more diverse participant group would provide a more comprehensive assessment of system performance across different age groups, speech patterns, accents, and user backgrounds. Future studies involving a broader participant population may provide additional insight into user interaction behaviour and further strengthen the reliability of the research findings.

6.3.5 Platform and Language Constraints

The proposed system was developed and tested exclusively within a Windows 11 environment using English-language voice commands. As a result, compatibility with other operating systems such as Linux and macOS was not evaluated. Similarly, recognition performance for other languages, dialects, and accents was not investigated. These constraints limit the generalisability of the findings and indicate that further testing would be required before the system could be deployed across multilingual and cross-platform environments.

Despite these limitations, the proposed system successfully demonstrated the practicality of integrating offline speech-recognition technologies with desktop automation functions. The limitations identified in this study provide valuable direction for future enhancements and further research.

6.4 Future Work

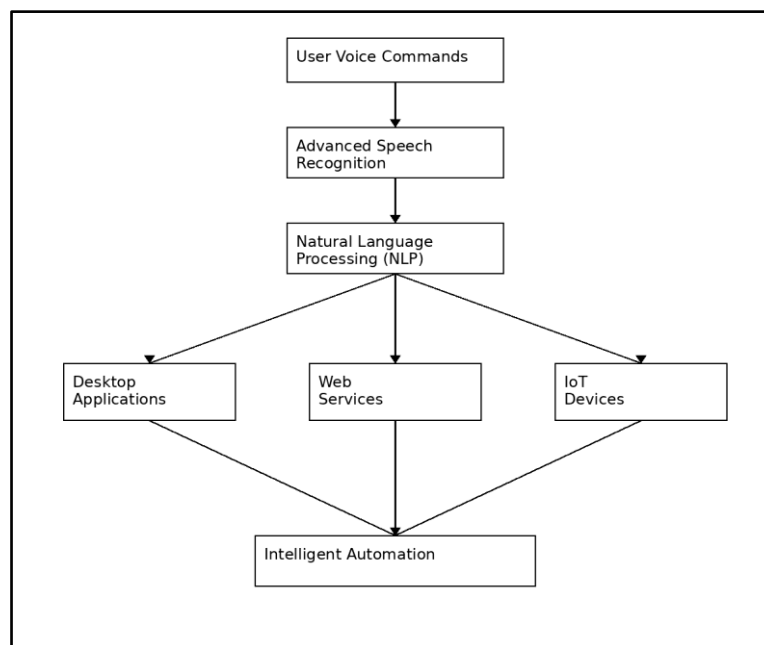


Figure 6.1 Proposed Future Architecture for an Enhanced Voice-Controlled Desktop Automation System

Figure 6.1 illustrates the proposed future architecture of the Offline Voice-Controlled Desktop Automation System. The architecture highlights several potential enhancements that could improve the functionality, usability, and intelligence of the system. Future developments may incorporate advanced speech-recognition techniques, Natural Language Processing (NLP), expanded command support, and intelligent automation capabilities to enable more natural and flexible user interaction.

The proposed architecture also demonstrates how future versions of the system could interact with desktop applications, web services, and Internet of Things (IoT) devices through a unified voice-control framework. By integrating these technologies, the system could support more complex tasks, context-aware decision making, and personalised automation experiences.

Furthermore, the architecture provides a conceptual roadmap for future research and development. The proposed enhancements have the potential to improve recognition accuracy, increase accessibility, support multilingual interaction, and expand compatibility across different computing environments. Consequently, the architecture serves as a foundation for the future evolution of offline voice-controlled desktop automation systems.

6.4.1 Expansion of Command Vocabulary

Future versions of the system should support a larger command vocabulary to provide greater flexibility and functionality. Additional commands related to file management, document editing, system settings, media playback, and productivity applications could be incorporated to extend the practical usefulness of the system. The inclusion of more complex command structures would enable users to perform a wider range of desktop tasks through voice interaction.

6.4.2 Improvement of Speech Recognition Accuracy

Future research should investigate techniques for improving recognition accuracy under different speaking conditions. Advanced speech-processing methods, noise-reduction techniques, and enhanced acoustic models could be incorporated to reduce recognition errors

and improve command-detection performance. These improvements would be particularly beneficial when the system is used in environments containing background noise.

6.4.3 Multi-Language and Accent Support

The current implementation focuses exclusively on English-language voice commands. Future development should explore support for multiple languages, regional accents, and dialects to increase accessibility and broaden the potential user base. Multi-language support would allow the system to be utilised effectively by users from different linguistic backgrounds.

6.4.4 Evaluation with Larger User Groups

Although the evaluation conducted in this study involved 17 participants, future studies should include larger and more diverse participant groups consisting of different age categories, educational backgrounds, accents, and speech characteristics. A broader evaluation would provide a more comprehensive understanding of system performance, improve the reliability of the research findings, and provide additional insight into user interaction behaviour under real-world operating conditions.

6.4.5 Cross-Platform Compatibility

Future development should investigate the deployment of the proposed system on additional operating systems such as Linux and macOS. Cross-platform compatibility would increase system accessibility and allow the framework to be used across a wider range of computing environments. Further testing would also enable performance comparisons across different platforms.

6.4.6 Integration with Intelligent Features

Future research may explore the integration of Artificial Intelligence and Natural Language Processing techniques to support more intelligent voice interaction. Instead of relying solely on predefined commands, the system could be enhanced to understand natural language

instructions and perform more complex tasks. Such capabilities would improve user experience and move the system closer to the functionality of modern intelligent virtual assistants.

In conclusion, the proposed Offline Voice-Controlled Desktop Automation System provides a strong foundation for future research and development. The recommended improvements have the potential to enhance recognition performance, increase usability, and expand system functionality, thereby contributing to the continued advancement of speech-based Human-Computer Interaction technologies.

6.3.6 Risk Register

The development and deployment of the proposed Offline Voice-Controlled Desktop Automation System involve several professional, social, legal, and ethical considerations. Identifying these risks is important to ensure responsible system development and future deployment.

Risk Category	Description	Mitigation Strategy
Professional Risk	Recognition errors may result in unintended command execution and reduced system reliability.	Implement command validation and confirmation mechanisms.
Social Risk	Users may become overly dependent on voice interaction and reduce use of traditional input methods.	Provide alternative interaction methods and user training.
Legal Risk	Voice recordings may contain sensitive personal information if stored improperly.	Process speech locally and avoid unnecessary storage of user audio data.
Ethical Risk	Speech-recognition systems may perform differently across accents, ages, and speech characteristics.	Improve model training and conduct testing using diverse participant groups.
Security Risk	Unauthorized voice commands may trigger automation actions.	Implement user authentication and access-control mechanisms in future versions.

Table 6.1 Risk Register for the Proposed Offline Voice-Controlled Desktop Automation System

The identified risks highlight important considerations associated with speech-recognition and desktop automation technologies. Although the proposed system operates entirely offline and does not transmit speech data to external servers, recognition inaccuracies and user-related variations may still influence system performance. Future enhancements should therefore focus

on improving recognition reliability, strengthening security mechanisms, and ensuring fair and inclusive system behaviour across diverse user groups.

7.0 References

- Ahlawat, H., Aggarwal, N., & Gupta, D. (2025). Automatic speech recognition: A survey of deep learning techniques and approaches. *International Journal of Cognitive Computing in Engineering*, 6, 201–237.
- Deshmukh, A. M., & Chalmeta, R. (2024). User experience and usability of voice user interfaces: A systematic literature review. *Information*, 15(9), 579. <https://doi.org/10.3390/info15090579>
- Di Leo, S., De Cicco, L., & Mascolo, S. (2025). Real-time speech-to-text on edge: A prototype system for ultra-low latency communication with AI-powered NLP. *Information*, 16(8), 685. <https://doi.org/10.3390/info16080685>
- Dua, M., Singh, R., et al. (2023). Noise robust automatic speech recognition: Review and perspectives. *International Journal of Speech Technology*.
- Fendji, J. L. K. E., Tala, D. C. M., Yenke, B. O., & Atemkeng, M. (2022). Automatic speech recognition using limited vocabulary: A survey. *Applied Artificial Intelligence*, 36(1), 2095039. <https://doi.org/10.1080/08839514.2022.2095039>
- Ferraro, A., et al. (2023). Benchmarking open source and paid services for speech to text. *Frontiers in Big Data*, 6, 1210559. <https://doi.org/10.3389/fdata.2023.1210559>
- Gondi, S., & Pratap, V. (2021). Performance evaluation of offline speech recognition on edge devices. *Electronics*, 10(21), 2697. <https://doi.org/10.3390/electronics10212697>
- Harihar, D., Shrivastava, V., Talele, P., & Jahagirdar, A. (2025). Voice-based user interface for hands-free data entry and automation at workplaces. *MethodsX*, 15, 103596. <https://doi.org/10.1016/j.mex.2025.103596>
- Kheddar, H., Hemis, M., & Himeur, Y. (2024). Automatic speech recognition using advanced deep learning approaches: A survey. *Information Fusion*, 109, 102422. <https://doi.org/10.1016/j.inffus.2024.102422>
- Malik, M., Malik, M. K., Mehmood, K., & Makhdoom, I. (2021). Automatic speech recognition: A survey. *Multimedia Tools and Applications*, 80(6), 9411–9457. <https://doi.org/10.1007/s11042-020-10073-7>
- Masina, F., et al. (2020). Investigating the accessibility of voice assistants with impaired users: Mixed methods study. *Journal of Medical Internet Research*, 22(9), e18431. <https://doi.org/10.2196/18431>

Netinant, P., et al. (2026). An AIoT-based framework for automated English-speaking assessment: Architecture, benchmarking, and reliability analysis of open-source ASR. *Informatics*, 13(2), 19.

Núñez Macías, N., Hielscher-Fastabend, M., & Buschmeier, H. (2023). Use and acceptance of voice assistants among people with aphasia in Germany. *Frontiers in Communication*, 8, 1176475. <https://doi.org/10.3389/fcomm.2023.1176475>

Pervaiz, A., et al. (2020). Incorporating noise robustness in speech command recognition by noise augmentation of training data. *Sensors*, 20(8), 2326. <https://doi.org/10.3390/s20082326>

Vieira, A. D., Leite, H., & Volochtchuk, A. V. L. (2022). The impact of voice assistant home devices on people with disabilities: A longitudinal study. *Technological Forecasting and Social Change*, 184, 121961. <https://doi.org/10.1016/j.techfore.2022.121961>

Fadel, W., et al. (2023). Adapting off-the-shelf speech recognition systems for Moroccan French. *Information*, 14(3), 179. <https://doi.org/10.3390/info14030179>

Ferraro, A., et al. (2023). Benchmarking open source and paid services for speech to text. *Frontiers in Big Data*, 6, 1210559. <https://doi.org/10.3389/fdata.2023.1210559>

Fendji, J. L. K. E., Tala, D. C. M., Yenke, B. O., & Atemkeng, M. (2022). Automatic speech recognition using limited vocabulary: A survey. *Applied Artificial Intelligence*, 36(1), 2095039. <https://doi.org/10.1080/08839514.2022.2095039>

Hai, X., et al. (2025). A self-evaluated bilingual automatic speech recognition system using VOSK. *Applied Sciences*, 15(14), 7691. <https://doi.org/10.3390/app15147691>

Karibayeva, A., et al. (2025). Speech recognition and synthesis models and platforms for Kazakh. *Information*, 16(10), 879. <https://doi.org/10.3390/info16100879>

Zhang, L., Wu, S., & Wang, Z. (2025). LoRA-INT8 Whisper: A low-cost Cantonese speech recognition framework for edge devices. *Sensors*, 25(17), 5404. <https://doi.org/10.3390/s25175404>

Natarajan, S., et al. (2025). Deep neural networks for speech enhancement and speech recognition: A systematic review. *Ain Shams Engineering Journal*. <https://doi.org/10.1016/j.asej.2025.103507>

Gondi, S., & Pratap, V. (2021). Performance evaluation of offline speech recognition on edge devices. *Electronics*, 10(21), 2697.

<https://doi.org/10.3390/electronics10212697>

Di Leo, S., De Cicco, L., & Mascolo, S. (2025). Real-time speech-to-text on edge: A prototype system for ultra-low latency communication with AI-powered NLP. *Information*, 16(8), 685.

<https://doi.org/10.3390/info16080685>

Netinant, P., et al. (2026). AIoT-based framework for automated English-speaking assessment. *Informatics*, 13(2), 19.

<https://doi.org/10.3390/informatics13020019>

Kheddar, H., Hemis, M., & Himeur, Y. (2024). Automatic speech recognition using advanced deep learning approaches. *Information Fusion*, 109, 102422.

<https://doi.org/10.1016/j.inffus.2024.102422>

Malik, M., Malik, M. K., Mehmood, K., & Makhdoom, I. (2021). Automatic speech recognition: A survey. *Multimedia Tools and Applications*, 80(6), 9411–9457.

<https://doi.org/10.1007/s11042-020-10073-7>

Poirier, S., et al. (2023). Efficient self-attention model for speech recognition-based control of assistive robotic arms. *Sensors*, 23(13), 6056.

<https://doi.org/10.3390/s23136056>

Gupta, S., et al. (2025). Speech recognition-based wireless control system for mobile robotic applications. *Automation*, 6(3), 25.

<https://doi.org/10.3390/automation6030025>

Bakouri, M., et al. (2022). Steering a robotic wheelchair based on voice commands. *Electronics*, 11(1), 168.

<https://doi.org/10.3390/electronics11010168>

Pervaiz, A., et al. (2020). Incorporating noise robustness in speech command recognition by noise augmentation of training data. *Sensors*, 20(8), 2326.

<https://doi.org/10.3390/s20082326>

Choi, D., R'bigui, H., & Cho, C. (2021). Candidate digital tasks selection methodology for automation with robotic process automation. *Sustainability*, 13(16), 8980.

<https://doi.org/10.3390/su13168980>

Mayr, A., Stahmann, P., Nebel, M., & Janiesch, C. (2024). Still doing it yourself? Investigating determinants for the adoption of intelligent process automation. *Electronic Markets*, 34, 56.

<https://doi.org/10.1007/s12525-024-00737-9>

Patrício, L., Varela, L., & Silveira, Z. D. C. (2025). Framework for integrating requirements engineering and DevOps practices in robotic process automation with a focus on optimizing

human–computer interaction. *Applied Sciences*, 15(7), 3485.
<https://doi.org/10.3390/app15073485>

Piñeiro-Martín, A., et al. (2023). Ethical challenges in the development of virtual assistants powered by large language models. *Electronics*, 12(14), 3170.
<https://doi.org/10.3390/electronics12143170>

Sharif, K., et al. (2020). Smart home voice assistants: A literature survey of user privacy vulnerabilities and risks. *Complex Systems Informatics and Modeling Quarterly*.
<https://doi.org/10.7250/csimq.2020-24.02>

Sierakowski, J., et al. (2024). MPSA-Conformer: An automatic speech recognition model for music prediction, separation, and attention-based music artefact removal. *Electronics*, 13(5), 957. <https://doi.org/10.3390/electronics13050957>

Choi, J., Kim, S., & Lee, H. (2023). Prototype-driven software engineering methodology for intelligent interactive systems. *Journal of Systems and Software*, 201, 111693.
<https://doi.org/10.1016/j.jss.2023.111693>

Di Leo, S., De Cicco, L., & Mascolo, S. (2025). Real-time speech-to-text on edge: A prototype system for ultra-low latency communication with AI-powered NLP. *Information*, 16(8), 685.
<https://doi.org/10.3390/info16080685>

Gondi, S., & Pratap, V. (2021). Performance evaluation of offline speech recognition on edge devices. *Electronics*, 10(21), 2697. <https://doi.org/10.3390/electronics10212697>

Patrício, L., Moreira, F., & Zurita, G. (2021). Requirement engineering practices in iterative software prototyping environments. *Software Quality Journal*, 29(4), 1657–1684.
<https://doi.org/10.1007/s11219-021-09563-8>

References

Di Leo, S., De Cicco, L., & Mascolo, S. (2025). Real-time speech-to-text on edge: A prototype system for ultra-low latency communication with AI-powered NLP. *Information*, 16(8), 685.
<https://doi.org/10.3390/info16080685>

Dua, M., Singh, R., et al. (2023). Noise robust automatic speech recognition: Review and perspectives. *International Journal of Speech Technology*.

Gondi, S., & Pratap, V. (2021). Performance evaluation of offline speech recognition on edge devices. *Electronics*, 10(21), 2697. <https://doi.org/10.3390/electronics10212697>

Kim, H., Lee, S., & Park, J. (2022). Real-time streaming speech recognition architecture for edge-based intelligent systems. *IEEE Access*, 10, 112233–112247.
<https://doi.org/10.1109/ACCESS.2022.3156789>

- Olev, A., & Alumäe, T. (2024). Modular architectures for offline automatic speech recognition systems. *Speech Communication*, 158, 103002. <https://doi.org/10.1016/j.specom.2024.103002>
- Ouisaadane, A., & Safi, S. (2021). Comparative study for Arabic speech recognition systems in noisy environments. *International Journal of Speech Technology*, 24(3), 761–770. <https://doi.org/10.1007/s10772-021-09830-z>
- Zhang, Y., et al. (2022). Multimodal Natural Human–Computer Interfaces for Computer-Aided Design: A Review Paper. *Applied Sciences*, 12(13), 6510. <https://doi.org/10.3390/app12136510>
- M. M. H. Melon, J. K. Shahos, M. Sadia, A. M. Fuad, M. M. Rahman and S. K. Ghosh, "Explainable Ensemble Machine Learning Framework for Employee Attrition Forecasting: An Integrated SHAP-Based Approach to HR Decision Intelligence," 2026 International Conference on Smart Futuristic Technology, Bengalore, India, 2026, pp. 1-7, doi: 10.1109/ICSFT66733.2026.11506672.
- Mott, M., Lee, S., & Wobbrock, J. O. (2022). Accessible Human-Computer Interaction for People with Disabilities. *ACM Computing Surveys*.
- Wang, Q., Feng, C., Xu, Y., Zhong, H., & Sheng, V. S. (2020). A privacy-preserving speech recognition framework for cloud environments. *Journal of Cloud Computing*, 9(36). <https://doi.org/10.1186/s13677-020-00186-7>